

פרויקט משחק דאבל וירטואלי

עבודת גמר תכנון ותכנות מערכות 883589

חלופה: מערכות טלפונים ניידים

מגיש: מידן חברוני סבח

ת.ז. 330738634

מנחה: אלון חימוביץ'

בית ספר תיכון ע"ש מנחם בגין, אילת



תאריך הגשה:

תוכן העניינים

4	מענה לדרישות משרד החינוך לפרויקט 5 יח"ל
5	מבוא
5	הרקע לפרויקט
5	מטרת המערכת
5	יכולות המערכת
6	תהליך המחקר
6	אתגרים מרכזיים
7	מדריך למשתמש
7	תרשים מסכים
8	תיאור המסכים
8	מסך פתיחה – התחברות:
9	מסך הרשמה:
10	תפריט ראשי:
11	מסך חדר המתנה:
12	מסך חדר מוכן:
13	מסך המשחק:
14	מסך סיום המשחק:
15	מסך היסטוריית המשחקים:
16	הוראות התקנה והרשאות
16	דרישות מיוחדות ומגבלות
16	גירסת Android מינימלית
16	מכשירים עליהם נבדקה המערכת
17	תיאור תחום הידע
17	אובייקטים נחוצים
17	סוגי נתונים
17	ייצוג מידע
19	מבנה/ארכיטקטורה
19	מחלקות הפרויקט וקשרים ביניהם
20	תרשים מחלקות
20	שיקולי תכן (design))
22	מחלקות – צד לקוח (Android))
22	מחלקה 1 - SocketManager
23	מחלקה 2 - GameActivity
25	מחלקה 3 - MatchHistoryAdapter
27	מחלקה 4 - LoginActivity
29	מחלקה 5 - RegisterActivity
31	מחלקה 6 - RoomActivity
33	מחלקה 7 - MatchHistoryActivity
35	מחלקה 8 - MatchItem
36	מחלקה 9 - MainActivity

38	GameOverActivity: - 10 מחלקה
40	CardDataManager: - 11 מחלקה
41	מחלקות – צד שרת (Python))
41	app.py: - 1 מחלקה
43	room_manager.py: - 2 מחלקה
45	game_manager.py: - 3 מחלקה
48	card_manager.py: - 4 מחלקה
50	auth_manager.py: - 5 מחלקה
53	stats_manager.py: - 6 מחלקה
56	Player: - 7 מחלקה
57	room.py: - 8 מחלקה
58	game.py: - 9 מחלקה
60	שמירת נתונים – צד לקוח (Android))
60	קבצים
60	Shared Preferences
62	שמירת נתונים – צד שרת
64	פרוטוקול תקשורת (עבור שרת-לקוח)
67	רפלקציה אישית
68	ביבליוגרפיה
69	נספחים
69	קוד המקור (source code))
69	CardDataManager מחלקה
72	GameActivity מחלקה
87	GameOverActivity מחלקה
90	LoginActivity מחלקה
93	MainActivity מחלקה
103	MatchHistoryActivity מחלקה
111	MatchHistoryAdapter מחלקה
114	MatchItem מחלקה
116	RegisterActivity מחלקה
121	RoomActivity מחלקה
128	SocketManager מחלקה
133	app.py מחלקה
141	auth_manager.py מחלקה
145	card_manager.py מחלקה
147	game_manager.py מחלקה
151	room_manager.py מחלקה
154	stats_manager.py מחלקה
156	game.py מחלקה
157	room.py מחלקה

מענה לדרישות משרד החינוך לפרויקט 5 יח"ל

פרק במחווה	נושאים שמומשו	מחלקה/ות בהן מומש הנושא
6	שימוש ב- RecyclerView/ ObservableCollection + CollectionsView	MatchHistoryActivity.java, MatchHistoryAdapter.java, item_match.xml
	הערה: מימוש יותר מנושא אחד מהרשימה, יחשב גם כ-2 נושאים מסעיף 10.	
7	אחסון וטיפול בנתונים (חובה שמירה ושליפה) בבסיס נתונים (Firebase, Firestore, MySQL, SQLite). שימוש בבסיס נתונים מרוחק לכתיבה וגם לקריאה כולל עדכון נתונים סינכרוני, יחשב גם כשני נושאים מסעיף 10.	LoginActivity.java, RegisterActivity.java, auth_manager.py, stats_manager.py
9	נושאים מתקדמים (חובה לבחור לפחות נושא אחד מסעיף זה):	
	אפליקציית רב משתתפים	SocketManager.java, RoomActivity.java, GameActivity.java
	רשתות מחשבים	SocketManager.java, RoomActivity.java, GameActivity.java
	פיתוח ושימוש ב API בצד שרת (אפשר בכל שפת תכנות)	app.py
10	שילוב של 2 נושאים מרשימה זו יחשבו ביחד לנושא מתקדם:	
	Shared Preference	LoginActivity.java, MainActivity.java

מבוא

הרקע לפרויקט

שם הפרויקט הוא "DOBBLE GAME". הפרויקט הוא אפליקציה למכשירי אנדרואיד המממשת את אחד ממשחקי הקלפים האהובים עלי – דאבל (Dobble) בגרסה שיכולה לקשר בין כמה משתתפים בצורה מקוונת בזמן אמת. המערכת כוללת לקוח (Client) שנכתב בשפת Java ב-Android Studio, ושרת שנכתב בשפת Python. השרת הוא זה שמנהל את המשחק, מחליט על החוקים, מסנכרן בין חדרים ובין המשתתפים ובודק את הלחיצות בזמן אמת באמצעות ספריית Socket.IO. המערכת מאפשרת למשתמשים ליצור משתמש למשחק, להתחבר לאחד, ליצור חדר משחק, להכניס חברים באמצעות קוד של החדר ולשחק ביחד כשהנתונים האישיים וההיסטוריה של המשחקים נשמרים ב-firebase. כל אחד מוזמן לבוא ולנסות את המשחק בין אם זה אנשים האוהבים משחקים קופסא, ילדים ומבוגרים שמחפשים משחק שהוא קל להבנה אך מהיר ותחרותי שאפשר לשחק עם חברים או משפחה מרחוק בלי הצורך לנוכחות פיזית ובחפיסת קלפים אמיתית. בחרתי בנושא הזה לפרויקט בגלל שדאבל זה אחד המשחקים קופסא האהובים עלי שיש לו רקע מעולם המתמטיקה. בין כל שני קלפים יש בדיוק סמל אחד זהה וזה מבוסס על עקרון גיאומטרי שרשום עליו גם בהוראות של המשחק האמיתי. בנוסף רציתי לעשות פרויקט שיהיה לי כמשחק שאני יכול לשתף עם חברים ועם הרעיון של דאבל זה היה מצויין שאני אעשה זאת כפרויקט סוף.

מטרת המערכת

המטרה של המערכת היא לתת לשחקנים להינות מהמשחק האהוב לא רק כשניפגשים פנים אל פנים עם קלפים ממשיים אלא גם להינות מהתחרותיות כאשר השחקנים נמצאים במקומות שונים. עבור המשתמש זה יוצר לו חווית משחק פשוטה שבה הוא יכול להתחבר למשתמש שלו ותוך שניות ליצור חדש ולהכניס את חבר שלו אליו בצורה קלה ומהירה. במהלך המשחק כאשר השחקנים בוחרים את הסמל המתאים בין שני הקלפים התשובה נקלטת בצורה מהירה ככה שלא יהיה תקיעות או ספק במי שניצח בסיבוב. המשחק עצמו:

1. חפישת קלפים: החפישה מורכבת ממספר קלפים שעל כל אחד מהם 8 סמלים שונים.
2. החוק המרכזי: בין כל שני קלפים בחפיסה קיים תמיד סמל אחד משותף (הסמל יכול להופיע בגדלים שונים, אך הוא לא תמיד סמל).
3. מהלך הסיבוב: על המסך מוצג קלף מרכזי וקלף אישי של השחקן.
4. המטרה: השחקן צריך להביט במהירות בשני הקלפים ולמצוא את הסמל המשותף היחיד ביניהם וללחוץ עליו.
5. נקודות וניצחון: השחקן הראשון שמזהה נכון את הסמל ולוחץ עליו מקבל נקודה והקלפים המרכזיים והאישיים משתנים. השחקן שזיהה את יותר פעמים את הסמל הזהה ראשון הוא המנצח.

יכולות המערכת

- ניהול משתמשים: אפשרות להרשמה, להתחברות ואימות משתמשים של השחקנים עם השרת באמצעות Firebase Authentication.
- מערכת חדרים: אפשרות ליצירת חדר והצטרפות של משתמשים אחרים באמצעות קוד מיוחד.
- סנכרון של המשתמשים במהלך המשחק: תמיכה במשחק של שני שחקנים על אותו חדר עם עדכוני מסך מיידיים.

- בדיקת ניחושים: השרת מוודא אם הסמל עליו לחץ השחקן הוא הסמל המשותף בשביל למנוע רמאויות או לחיצות שגויות.
- ניהול חפישת קלפים משתנה: השרת מגריל את הקלפים מתוך קובץ הנתונים ומחלק לכל שחקן את הקלף שלו.
- טבלת היסטורית משחקים: הצגת המשחקים של המשתמש בעזרת RecyclerView.
- מערכת שמירת נתונים (Session Persistence): שימוש ב- SharedPreferences לשמירת הנתונים של המשתמש על מנת להקל ולמנוע את הצורך להקליד את האימייל והסיסמה בכל פעם שהוא מפעיל את האפליקציה מחדש.

תהליך המחקר

1. מחקר על תחום הידע: הייתי צריך לחשוב איך אני לוקח את הקלפים הממשיים והופך אותם לצורה שבה השרת יוכל לקרוא. אחרי שבדקתי כמה צורות אפשריות החלטתי שהפתרון הכי מהיר וקל יהיה לחלק כל קלף ל-9 אזורים (למעלה-שמאלה, למעלה-אמצע, למעלה-ימינה, אמצע-ימינה...), כך שבכל אזור יהיה סמל אחר. בתוך קובץ json הכנסתי ידנית בכל אזור איזה סמל יש ככה שהשרת יכול לזהות בקלות את הסמלים ולהתאים ביניהם. בנוסף היה צריך למצוא פרוטוקול מתאים לתקשורת בין השרת והלקוח. HTTP לא מתאים משום שהוא בקשה ותגובה בלי תקשורת שמתאימה למשחק בעל כמה משתמשים זה למה חיפשתי פרוטוקל אחר שמתאים ומצאתי את ספריית Socket.IO.
2. עבודת שטח וסקירת שוק: בעולם קיימות הרבה אפליקציות לוח וקלפים ויש גם כאלה עם משחק הדאבל אבל האפליקציות האלה כבדות יחסית ועם פרסומות. שלי מתרכזת רק במשחק האהוב דאבל והיא מאוד קלה להפעלה ללא צורך במערכת חברים מורכבת. רק לשלוח קוד של חדר ולהתחיל את המשחק.
3. פירוט טכנולוגיות שהשתמשתי שאינן חלק מתכנית הלימודים:
 - Python – שפת תכנות שלא לומדים אותה בתוכנית הלימודים. השתמשתי בה לבניית השרת.
 - Socket.IO – ספרייה מתקדמת המבוססת על WebSockets. היא נותנת לנו את האפשרות לשלוח ולקבל הודעות, להתמודד עם ניתוקים ועוד בזמן אמת ולנהל חדרים בצד השרת.
 - Cloud Firestore – בשרת פייתון השתמשנו בו כדי לקרוא ולכתוב נתונים לתוך הענן לגבי משחקים שהיו למשתמשים.

אתגרים מרכזיים

במשחק דאבל מי שמדיר יותר הוא זה שמנצח והבעיה המרכזית הייתה מה יקרה אם שני השחקנים לחצו על הסמל הנכון כמעט באותה המילישנייה. אם היינו עושים שהלוגיקה הייתה מתנהלת בצד של הלקוח, כל אחד מהם היה יכול לחשוב שהוא ניצח.

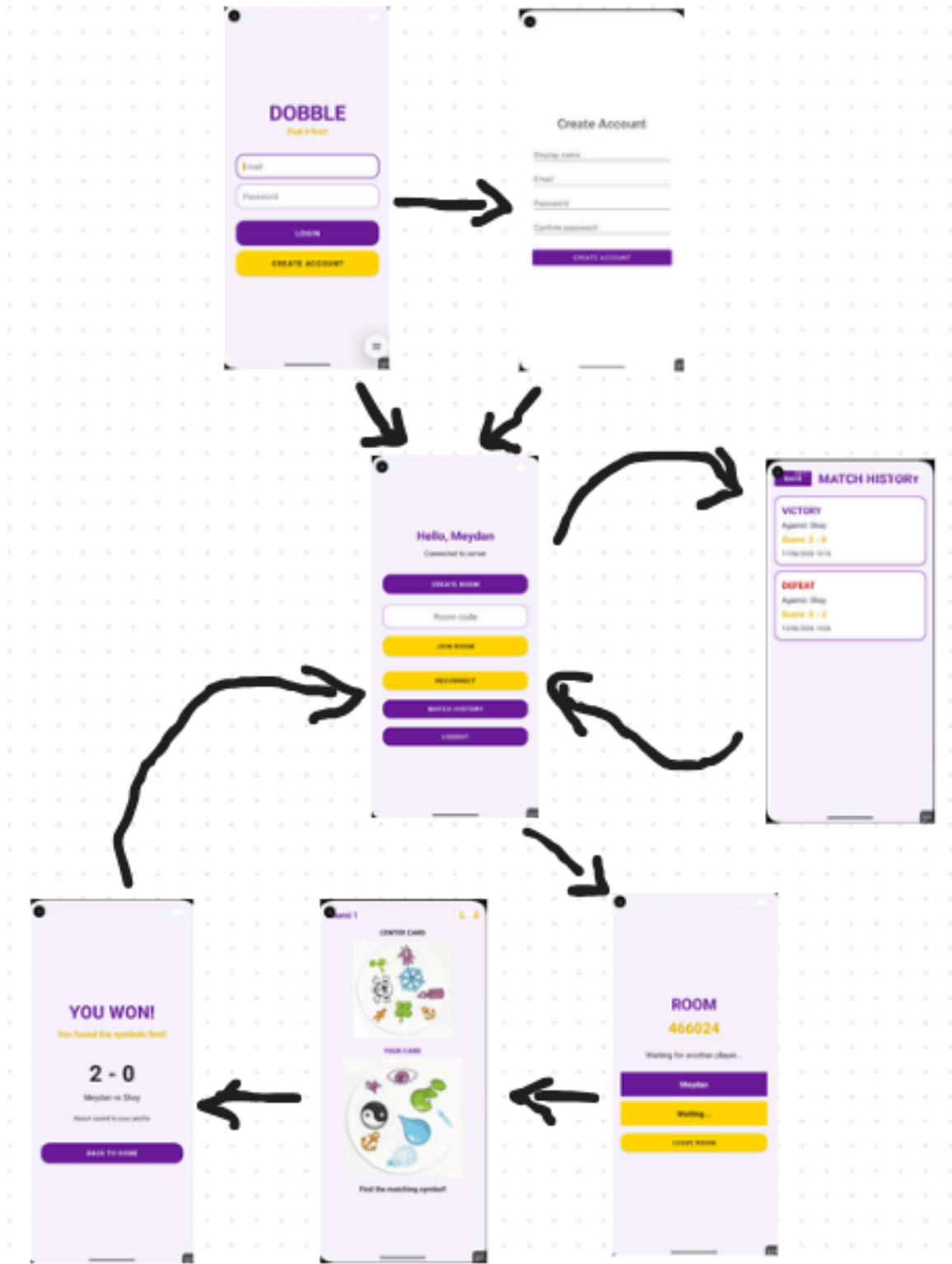
לכן עשיתי כך שהלקוח רק מדווח לשרת על הלחיצה של המשתמש והשרת קובע מי ניצח בצורה אמינה. הלוגיקה בשרת עובדת ככה שהיא מקבלת את הניחוש הראשון שמגיע אליה לא משנה מאיזה שחקן וחוסמת את הניחוש השני ככה שלא יהיה ספק מי ענה קודם.

פתרונות כמו לקוח אחד שמשמש כמנהל נפסלו בגלל שהבעיה המרכזית עדיין הייתה יכולה להתממש כי הלקוח שהוא גם השרת יכול לקלוט את התשובה שלו לפני כי לקח להודעה של הלקוח השני זמן להגיע. שימוש ב-Firebase גם לא היה מתאים כי עם שני הלקוחות ששולחים אליו הודעות זה היה לוקח הרבה זמן יחסית ולא מתאים למשחק מהיר.

בנוסף עיצבתי את הפרויקט בצורה שתואמת לצבעים האמיתיים של המשחק ונותנת חוויה טובה יותר למשתמש והקודים גם בשרת וגם בלקוח מחולקים בצורה נוחה וברורה ככה שאם בעתיד נרצה להוסיף פיצרים יהיה את האפשרות לעשות זאת בקלות יותר.

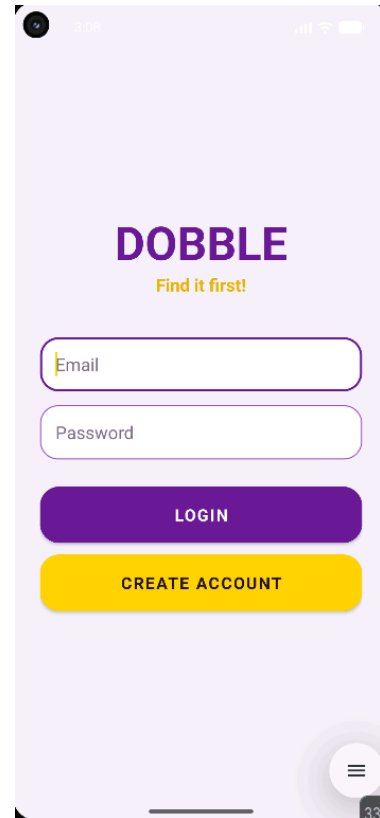
מדריך למשתמש

תרשים מסכים



תיאור המסכים מסך פתיחה – התחברות:

זהו המסך הראשון שהמשתמש רואה וממנו המשתמש נכנס לחשבון שלו או עובר לאפשרות שתיתן לו להירשם וליצור משתמש חדש. במסך מופיע הלוגו של המשחק, תיבות טקסט שבהם מכניסים את המייל והסיסמה של המשתמש, כפתור להתחברות וכפתור שמעביר אותך לאפשרות הרשמה אם אתה משתמש חדש. אם המשתמש כבר התחבר האפליקציה תשמור את פרטי ההתחברות שלו ותעביר אותו ישר לתפריט הראשי.



מסך הרשמה:

המסך הזה מאפשר למשתמש ליצור משתמש חדש על מנת שיוכל להתחיל את חווית המשחק. במסך מופיע המטרה של המסך (create account), תיבות טקסט שבהם המשתמש יכניס את פרטיו (שמו במשחק, מייל, סיסמה, ואימות הסיסמה), וכפתור ששולח את הנתונים למאגר מידע ומעביר את המשתמש לתפריט של המשחק.

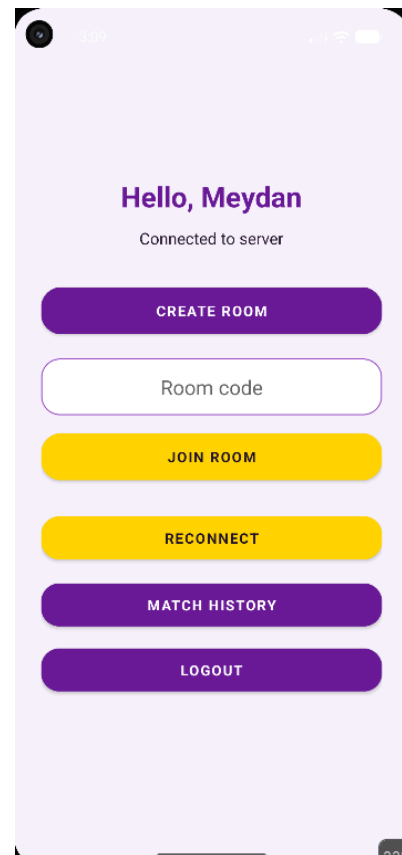


Create Account



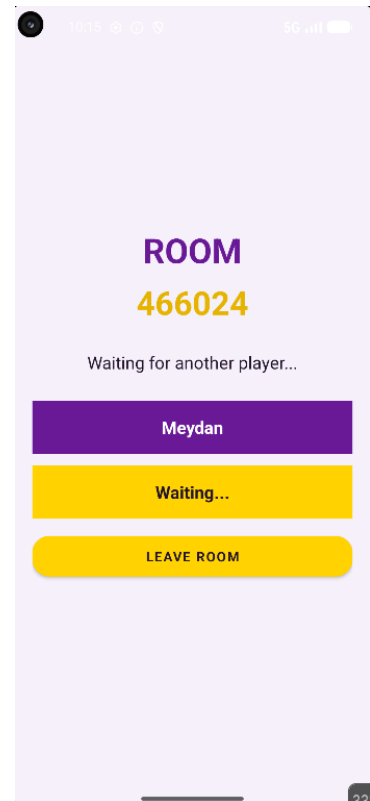
תפריט ראשי:

זהו התפריט הראשי של המשחק, כאשר משתמש נכנס לכאן הוא יכול להשתמש בכל הפיצרים שהמשחק מציע. האפליקציה מברכת את המשתמש בשלום ומתחברת לשרת שמנהל את המשחקים. התפריט כולל בו כפתור שיכול ליצור חדר בשביל משתמש, תיבת טקסט שאותה אפשר למלא עם קוד של חדר בשביל להתחבר אליו ולהתחיל משחק, כפתור "join room" שאחרי שמכניסים את הקוד של החדר לוחצים עליו והאפליקציה תעביר את המשתמש לחדר, כפתור "reconnect" שמאפשר למשתמש להתחבר מחדש לשרת אם עבר הרבה זמן בלי שהמשתמש ביצע פעולה, כפתור "match history" שמאפשר למשתמש לעבור למסך שבו הוא יראה את היסטוריית המשחקים שלו וכפתור "logout" שבעזרתו המשתמש יכול להתנתק מהחשבון שלו.



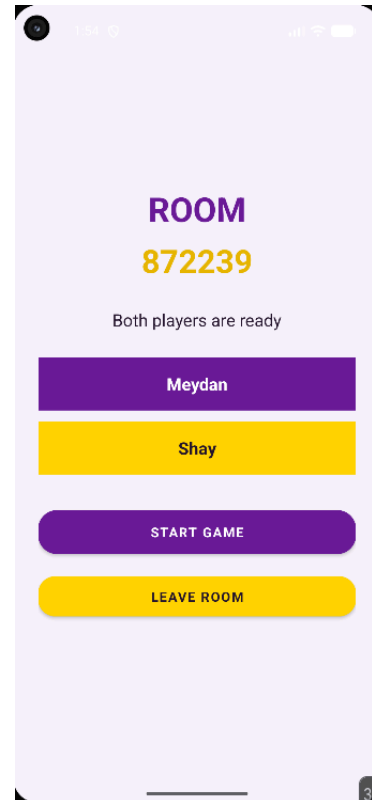
מסך חדר המתנה:

מסך זה המסך שבו המשתמש שפתח את החדר מחכה לשחקן השני שיכנס בעזרת הקוד שמופיע במרכז המסך. במסך מופיע השם של המשתמש, הודעה שמסמנת שממתינים לשחקן השני שיכנס לחדר "waiting" וכפתור "leave room" שמאפשר למשתמש לחזור לתפריט הראשי.



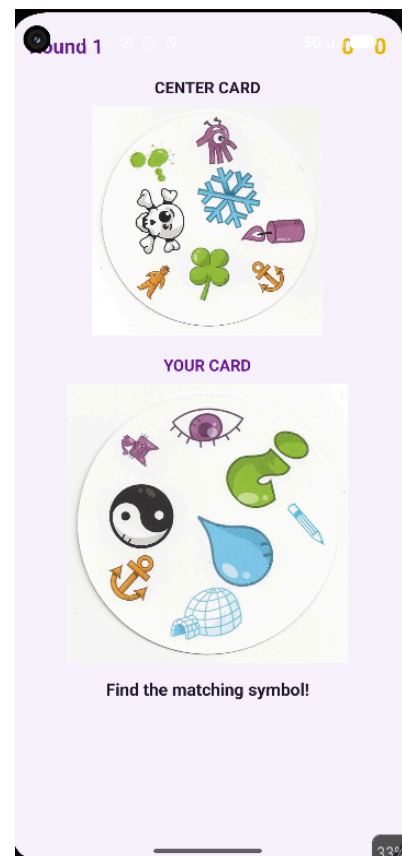
מסך חדר מוכן:

זה המסך שמראה כי שני השחקנים נמצאים בתוך החדר ואפשר להתחיל את המשחק. בנוסף למספר החדר האפליקציה מראה הודעה כי שני השחקנים מוכנים להתחיל את המשחק, את שמות השחקנים, כפתור "start game" שמופיע ליוצר של החדר ומאפשר לו להתחיל את המשחק וכפתור "leave room" שעדיין ניתן לו את האפשרות לצאת מהחדר ולחזור לתפריט הראשי.



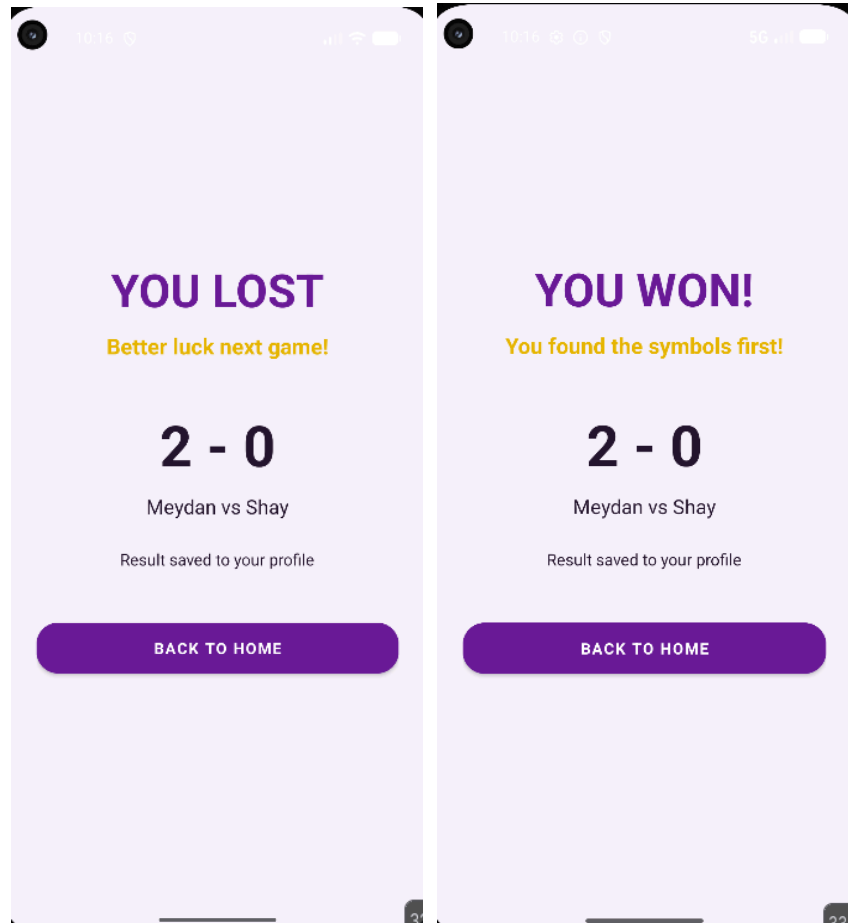
מסך המשחק:

זה המסך שבו המשתמש באמת משחק במשחק ומתחרה נגד השחקן השני. במסך למעלה מופיע מספר הסיבוב והניקוד של המשחק ואז האפליקציה מדמה את המשחק האמיתי. הקלף העליון הוא הקלף ששני השחקנים צריכים למצוא בו את הסמל המשותף שיש גם בקלף שלהם, והקלף התחתון הוא הקלף האישי של כל שחקן. כאשר השחקן מצא את הסמל המשותף לו ולקלף שנמצא במרכז השולחן הוא לוחץ על הסמל, אם הניחוש נכון אז האזור בו המשתמש לחץ מתמלא בירוק דבר המאשר כי השחקן צדק, מוסיף לו נקודה ומתחיל את הסיבוב הבא לשני השחקנים.



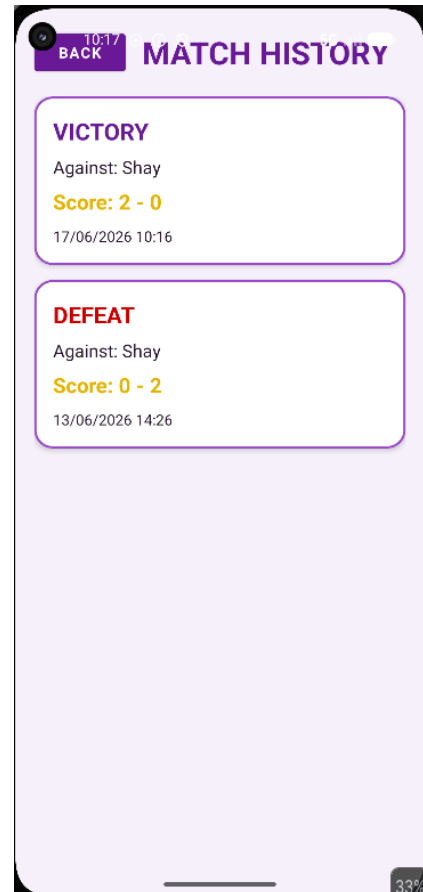
מסך סיום המשחק:

מסך זה מראה למשתמש את תוצאות המשחק והניקוד ביחד גם עם מי שניצח. בראש המסך רואים הודעה ("you lost" או "you won") שאומרת אם ניצחת או לא ביחד עם הודעת עידוד. לאחר מכן אפשר לראות את הניקוד של המשחק, שמות השחקנים והודעה שאומרת כי המשחק נשמר בפרופיל של המשתמש. בנוסף יש כפתור שמחזיר את המשתמש לדף הבית.



מסך היסטוריית המשחקים:

מסך זה מראה למשתמש את המשחקים אשר שיחק בעבר בצורה יפה ומעוצבת. למעלה רואים טקסט שמעיד על מטרותו של המסך וכפתור "back" שמאפשר למשתמש לחזור לתפריט הראשי. בקוביות המסך מסכם את המשחקים שהמשתמש שיחק בפורמט הבא: בראש הקוביה – תוצאת המשחק (ניצחון או הפסד), אחר כך שם השחקן נגדו המשתמש שיחק, הניקוד של המשחק ובסוף התאריך והשעה בו המשחק התקיים.



הוראות התקנה והרשאות

כדי שהאפליקציה תפעל בצורה תקינה ותוכל לתקשר עם שחקנים אחרים, יש לוודא את שני התנאים הבאים:

1. חיבור לאינטרנט: המכשיר חייב להיות מחובר לרשת אינטרנט פעילה (WI-FI או רשת סלולרית), מכיוון שהמשחק מבוסס על שרת ואימות מול Firebase.
2. הפעלת השרת (בשלב הפיתוח): מאחר והאפליקציה פונה לשרת עצמאי, שרת הפיתוח חייב לרוץ ולהיות נגיש בתכובת ה-IP שמוגדרת בקוד האנדרואיד.

דרישות מיוחדות ומגבלות

- דרישות תקשורת: המערכת מחייבת תקשורת אינטרנט רציפה וקבועה. בגלל שהמשחק מבוסס על מהירות תגובה בזמן אמת, מומלץ להשתמש בחיבור WI-FI יציב או ברשת סלולרית בדור חדש כדי למנוע תקיעות שעלולות לפגוע בחוויית המשחק.
- מגבלות חומרה וחיישנים: המשחק אינו דורש חומרה מיוחדת כמו רכיב GPS, חיישני תנועה או מצלמה.
- אמולטור מול מכשיר פיזי: האפליקציה יכולה לרוץ בצורה מלאה ותקינה על מכשירים פיזיים וגם על גבי אמולטורים, בתנאי שלמחשב המריץ את האמולטור יש חיבור אינטרנט פעיל המאפשר גישה לשרת ולענן.
- דרישות קבצים מקומיים: המערכת אינה מצפה לקבצים חיצוניים שישבו על זיכרון המכשיר (כמו תמונות או קבצי אחרים כמו שמע). כלל רכיבי המערכת כולל הלוגו, האייקונים וגרפיקת הסמלים של הקלפים – נמצאים ישירות בתוך משאבי האפליקציה (בתוך תיקיית res/drawable ותיקיית res/layout), מה שמאפשר התקנה נקייה ופשוטה.

גירסת Android מינימלית

גרסת Android מינימלית נדרשת: Android 8.0 (גרסת API 26) ומעלה. הגדרה זו מבטיחה תמיכה ברכיבי ממשק מודרניים ובספריות התקשורת של Socket.IO, ומאפשרת לאפליקציה לרוץ על מעל ל-95% ממכשירי האנדרואיד הפעילים בשוק כיום.

מכשירים עליהם נבדקה המערכת

המכשירים שעליהם נבדקה המערכת זה הטלפון pixel 4a באימולטור המובנה של Android Studio כאשר הטלפון מריץ גרסת Android 14 / API 36. מאחר וצריך בשביל המשחק לבדוק אותו עם כמה משתמשים הרצתי 2 טלפונים מהסוג הזה באימולטור וראיתי שהמשחק עובר כמו שצריך, כל סנכרון החדרים, חלוקת הקלפים ועדכון הנקודות עובדים בצורה מלאה ללא תקלות.

תיאור תחום הידע

אובייקטים נחוצים

- הפרויקט משלב בין העולם של המשחקים (ניהול לוח, ניהול קלפים) לבין עולם הרשתות אז אנחנו צריכים את האובייקטים של שני הנושאים. האובייקטים המרכזיים שמצויים בקוד (גם של השרת וגם של הלקוח) הם:
1. משתמש (User): מייצג משתמש רשום במערכת. מכיל את פרטיו, מזהה ייחודי, ואת הניקוד שלו בכל זמן במהלך משחק פעיל.
 2. חדר משחק (Room): מייצג אובייקט שמשמש כלובי שמקשר בין השחקנים לפני תחילת המשחק. כל חדר מבדיל את עצמו בין חדרים אחרים ומנהל את רשימת השחקנים שנמצאים בפנים.
 3. משחק פעיל (Game): מייצג משחק ומנהל את המצב (state) של המשחק בחגר מסויים לאחר שנלחץ כפתור ה-Start.
 4. קלף (Card): אובייקט המייצג קלף בודד של דאבל, המכיל קבוצה של סמלים המיוצגים כטקסט.
 5. פריט היסטוריית משחק (MatchItem): אובייקט המשמש להצגת היסטוריית המשחקים הישנה של המשתמש בתוך ה-RecycleView.

סוגי נתונים

- המשתנים ומבני הנתונים שישומו עבור האובייקטים הללו מבוססים על טיפוסים רגילים ומוכרים בשפות Java ו-Python:
- מחרוזות וטקסטים (str / String): משמשים לשמירת ה-id הייחודי של Firebase, שמות השחקנים, אימיילים, קוד החדר, שמות הסמלים שעל הקלפים ועוד.
 - מספרים שלמים (int / integer): משמשים לחישוב הניקוד של השחקנים בזמן אמת, ספירת מספר השחקנים בחדר ומספר הסיבוב הנוכחי.
 - ערכים בוליאניים (bool / boolean): משמשים לניהול מצבים במערכת, כמו: האם המשחק בחדר כבר התחיל או האם המשתמש מחובר כבר בהצלחה.
 - מילונים ומבנים דינמיים (List / dict / map): משמשים בצד השרת לייצוג מבנה הנתונים המורכב של חדרים ושחקנים בזיכרון, וכן בצד האנדרואיד לקבלת הודעות ה-JSON דרך הסוקטים.

ייצוג מידע

1. ייצוג קלף ולוח המשחק:
 - המבנה שנבחר: רשימה של מחרוזות טקסט <List<String> באנדרואיד, list בפייתון. כל קלף מיוצג כמערך חד-מימדי המכיל את שמות הסמלים שנמצאים עליו. חפיסת הקלפים כולה מיוצגת כרשימה של רשימות (מערך גמיש) הנטענת מתוך קובץ ה-Cards.json.
 - נימוק לבחירה: מאחר שמשחק דאבל אינו משחק מבוסס מיקום גיאומטרי או משבצות (כמו שחמט או איקס-עיגול הדורשים מערך דו-מימדי (3×3)), אין חשיבות למיקום הפיזי של הסמל על הקלף לצורך הלוגיקה של השרת. כל מה שהשרת צריך לבדוק זה האם איבר (סמל) מסוים קיים בשתי הרשימות במקביל (בקלף השחקן ובקלף המרכזי). חיפוש ומעבר על רשימת מחרוזות קטנה (כמה סמלים בודדים לקלף) מתבצע בזמן ריצה אפסי, מה שמבטיח מהירות תגובה מקסימלית ברשת.
2. ניהול החדרים והשחקנים בשרת:
 - המבנה שנבחר: מילון / טבלת גיבוב (Dictionary / Dict בפייתון). מפתח המילון (Key) הוא קוד החדר הייחודי או ה-uid, והערך (Value) הוא מופע של האובייקט (Room או Player).

- **נימוק לבחירה:** בזמן שהשרת מריץ עשרות משחקים במקביל, שחקנים שולחים בקשות בלי הפסקה. שימוש במילון מאפשר לשרת לבצע שליפה ישירה של החדר או השחקן המתאים לפי המזהה שלו בזמן קבוע ללא צורך בלולאות שיעברו על פני כל השחקנים בשרת.

פעולות על הנתונים: קריאה, הוספה, עדכון ומחיקה

במהלך מחזור החיים של האפליקציה מתבצעות כל פעולות ניהול המידע הללו, והן מחולקות בין זיכרון השרת הזמני (בזמן משחק) לבין בסיס הנתונים הקבוע בענן (לשמירה ארוכת טווח):

1. הוספה:

- **בזיכרון השרת:** כשמשמש לוחץ "Create Room", השרת מייצר אובייקט Room חדש ומוסיף אותו למילון החדרים הפעילים.
- בענן (Firestore): בעת הרשמה במסך RegisterActivity, מתווסף מסמך חדש לאוסף users. בסיום משחק, השרת מייצר ומוסיף מסמך חדש לאוסף match_history.

2. קריאה:

- בזיכרון השרת: השרת קורא ללא הרף את נתוני הקלפים מתוך Cards.json כדי לחלק אותם.
- בענן (Firestore): במסך ה-LoginActivity, השרת קורא את נתוני המשתמש כדי לאמת אותו ולשלוף את ה-displayName שלו. במסך ההיסטוריה, האנדרואיד מבצע קריאה של רשימת המשחקים הישנים כדי להציג אותם למשתמש.

3. עדכון:

- בזיכרון השרת: במהלך המשחק ב-GameActivity, השרת מעדכן את משתנה הניקוד (score) של השחקן בכל פעם שהוא לוחץ על סמל נכון, ומעדכן את קלף השולחן הנוכחי לקלף החדש.

4. מחיקה:

- בזיכרון השרת: ברגע שמשחק מסתיים או שכל השחקנים עוזבים את החדר, השרת מוחק באופן יזום את אובייקט ה-Room וה-Game ממילון הזיכרון שלו כדי לפנות מקום ב-RAM של המחשב ולא להעמיס על המערכת.

מבנה/ארכיטקטורה

מחלקות הפרוייקט וקשרים ביניהם

מבנה העצים והתיקיות בפרוייקט (Project Package Structure)

כדי לשמור על סדר, מודולריות והפרדת רשויות מלאה (Clean Architecture), הפרוייקט חולק לתתי-תיקיות (Packages) מוגדרות, הן בצד הלקוח והן בצד השרת:

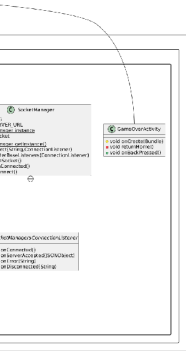
צד הלקוח (Android - Package: me.meydan.dobble):

- תיקיית השורש: מכילה את האקטיביטי המרכזי MainActivity.java שמנווט את המשתמש בתחילת הדרך.
- תיקיית התקשורת: מכילה את SocketManager.java – המחלקה הסינגלטונית שאחראית על כל ערוץ ה-WebSocket מול השרת.
- תיקיית מסכי המשחק (Activities):
 - LoginActivity.java ו-RegisterActivity.java (ניהול כניסה ורישום).
 - RoomActivity.java (ניהול הלובי והמתנה לשחקנים).
 - GameActivity.java (ניהול לוח המשחק הפעיל בזמן אמת).
 - MatchHistoryActivity.java (מסך הצגת המשחקים הקודמים).
- תיקיית העזר והרשימות (Adapters & Models):
 - MatchHistoryAdapter.java ו-MatchItem.java – לניהול ה-RecyclerView של היסטוריית המשחקים.
 - CardDataManager.java – עזר מקומי לניהול רכיבי הגרפיקה של הקלפים.

צד השרת (Python Server - dobble_server):

- app.py (קובץ ראשי): נקודת הכניסה (Entry Point). מאתחל את השרת, מקשיב לפורט ומנתב את אירועי ה-Socket.IO למנהלים המתאימים.
- תיקיית managers (הלוגיקה העסקית):
 - auth_manager.py – אימות טוקנים מול Firebase.
 - room_manager.py – יצירה, הצטרפות וניהול חדרים.
 - game_manager.py – מנוע ריצת המשחק, בדיקת הלחיצות וניהול התורים.
 - stats_manager.py – קריאה וכתיבה של היסטוריית משחקים ל-Firebase.
- תיקיית models (מבני הנתונים בזיכרון):
 - room.py – אובייקט שמחזיק את נתוני חדר המתנה.
 - game.py – אובייקט שמחזיק את מצב המשחק הפעיל ברגע נתון.
- תיקיית data: מכילה את קובץ ה-JSON הסטטי Cards.json שמגדיר את חפיסת הקלפים והסמלים המשותפים.

תרשים מחלקות



https://drive.google.com/file/d/1PWDUag3mz8HOI_IZSOgGuaGk_m1YnAOB/view?usp=sharing

חלוקת תפקידים בארכיטקטורת שרת-לקוח

הפעולה הלוגית	באחריות הלקוח (Android App)	באחריות השרת (Python Server)
אימות ורישום	קליטת אימייל וסיסמה, שליחה ל-Firebase Auth, והפקת ה-ID Token.	קבלת ה-Token מהאנדרואיד, אימותו מול ה-SDK של Firebase, ושליפת שם המשתמש מ-Firebase.
ניהול חדרים	הצגת כפתורי "צור חדר"/"הצטרף", וריענון רשימת השחקנים על המסך.	הגרלת קוד חדר ייחודי, שמירתו בזיכרון, שיוך שחקנים אליו, ו"דחיפת" עדכונים לכל חברי החדר.
חוקי המשחק	מאזין ללחיצה של המשתמש על הסמל, ושולח מיד את שם הסמל לשרת.	בודק האם הסמל שלחצו עליו אכן זהה, מעדכן ניקוד, ומגריל קלף חדש.
גרפיקה ותצוגה	ציור הקלפים, הצגת הסמלים, והפעלת רכיבי ממשק משתמש (UI).	לא מתעסק בגרפיקה. מעביר רק נתוני טקסט (JSON) המייצגים את שמות הסמלים.
שמירת נתונים קבועה	מציג את ההיסטוריה שמתקבלת מהרשת ב-RecyclerView.	כותב את מסמך סיכום המשחק ישירות ל-Firebase מול ה-Cloud API המאובטח.

שיקולי תכן (design)

במהלך תכנון ארכיטקטורת הפרויקט, נבחנו מספר חלופות תוכנה. עכשיו אציג את הנימוקים לבחירות המרכזיות שנעשו:

1. בחירה בארכיטקטורת שרת סמכותי (Authoritative Server) מול שרת טיפשי:
 - החלופה: לנהל את חוקי המשחק ואת הבדיקה האם השחקן צדק בתוך קוד ה-Java של האנדרואיד, ולשתף רק את הניקוד הסופי.

- ההחלטה והנימוק: הוחלט להשתמש בשרת סמכותי שבו כל הלוגיקה מנוהלת בפייתון. במשחק מהירות בזמן אמת, אם הבדיקה הייתה נעשית במכשיר הטלפון, שני שחקנים שלחצו במקביל היו יכולים "לנצח" מקומית לפני שהמידע הגיע לשחקן השני, מה שהיה גורם לחוסר סנכרון חמור (De-synchronization). ניהול ריכוזי בשרת מבטיח שמי שהודעת ה-Socket שלו הגיעה ראשונה לשרת – הוא המנצח הרשמי, והחוק נאכף בצורה שווה ומוחלטת ללא אפשרות לרמאות (Cheating) מצד הלקוח.

2. בחירה בתבנית עיצוב סינגלטון (Singleton Pattern) עבור SocketManager:

- החלופה: לפתוח חיבור Socket חדש בכל אקטיביטי בנפרד (במסך הלובי, במסך החדר, במסך המשחק).
- ההחלטה והנימוק: החלופה נפסלה מכיוון שפתיחת חיבור רשת מחדש בכל מעבר מסך מייצרת השהיה גבוהה (Overhead), עלולה לנתק את השחקן מהחדר, ומבזבזת משאבי סוללה ורשת. מחלקת SocketManager נכתבה כ-Singleton – היא מייצרת מופע יחיד בזיכרון האפליקציה ששומר על חיבור קבוע ורציף לאורך כל חיי האפליקציה, ומאפשרת לכל מסך להשתמש באותו הצינור הקיים בקלות.

3. חלוקת השרת למנהלים (Managers) ומודלים (Models):

- החלופה: כתיבת כל לוגיקת השרת בתוך קובץ ראשי אחד (app.py).
- ההחלטה והנימוק: כתיבת קובץ אחד מונעת יכולת לתחזק את הקוד או למצוא באגים (Spaghetti Code). החלוקה למנהלים ייעודיים (כמו card_manager.py האחראי אך ורק על הקלפים, ו-room_manager.py האחראי רק על הלובי) מאפשרת "הפרדת דאגות" (Separation of Concerns). אם יש באג במנגנון הקלפים, פותחים רק את המנהל הרלוונטי מבלי לפגוע במנגנון האימות או בניהול החדרים.

מחלקות – צד לקוח (Android)

מחלקה 1 - SocketManager:

תפקיד המחלקה:

מחלקה זו מהווה את תשתית התקשורת של האפליקציה כולה. היא ממומשת כתבנית עיצוב סינגלטון (Singleton) ומנהלת ערוץ תקשורת קבוע, יציב ודו-כיווני בזמן אמת (WebSocket) מול שרת הפיתוח העצמאי. היא מונעת פתיחת חיבורים מרובים ומאפשרת לכל מסכי האפליקציה להשתמש באותו צינור תקשורת.

תכונות המחלקה (Fields):

- `private static SocketManager instance` (סטטית): שומרת את המופע היחיד של המחלקה בזיכרון לאורך כל חיי האפליקציה.
- `private Socket socket`: אובייקט הצינור הפיזי של ספריית `Socket.IO` המנהל את שליחת וקבלת ההודעות בפועל.
- `private final String SERVER_URL`: מחרוזת קבועה המגדירה את כתובת ה-IP והפורט של השרת (לדוגמה `http://10.0.2.2:5000` עבור גישה מהאמולטור לשרת מקומי).

פעולות המחלקה (Methods):

- `private SocketManager()` (בנאי פרטי): מאתחל את הגדרות ה-`Socket`, מאפשר ניסיונות התחברות אוטומטיים במקרה של ניתוק רגעי (`reconnection = true`), ומקשר את כתובת השרת.
- `public static synchronized SocketManager getInstance()`: הפונקציה המרכזית המחזירה את המופע הייחודי של המחלקה. אם המשתנה `instance` ריק, היא מייצרת אותו פעם אחת; אחרת, היא מחזירה את הקיים.
- `public Socket getSocket()`: פונקציית גישה המאפשרת לכל אקטיביטי אחר "להאזין" לאירועים או לשלוח הודעות דרך ה-`Socket`.
- `public void connect()`: מפעילה פיזית את החיבור לשרת מול שרת הפיתוח אם הוא אינו מחובר עדיין.
- `public void disconnect()`: סוגרת ומנתקת בצורה יזומה ומסודרת את החיבור מול השרת (למשל בעת סגירת האפליקציה).

קוד והסבר של פעולה לוגית מרכזית (`getInstance`):

```
public static synchronized SocketManager getInstance() {
    if (instance == null) {
        instance = new SocketManager();
    }
    return instance;
}
```

הסבר: מילת המפתח `synchronized` מבטיחה שאפילו אם שתי פונקציות ברקע ינסו לגשת ל-`SocketManager` באותה המילישנייה, לא ייווצרו שני חיבורים שונים במקביל. התנאי `מוודא יצירה חד-פעמית` (Lazy Initialization) השומרת על יעילות משאבי המכשיר.

מחלקה 2 - `GameActivity`:

תפקיד המחלקה:

המחלקה החשובה ביותר המנהלת את מסך המשחק הפעיל בזמן אמת. היא אחראית על הצגת הגרפיקה של הקלפים (הקלף המרכזי והקלף האישי), האזנה ללחיצות המשתמש על הסמלים, וסנכרון מיידי מול הודעות הדחיפה שמגיעות מהשרת. תכונות המחלקה (Fields):

- `private ImageView centerCardImageView`: רכיב תצוגת תמונה המציג את קלף המשחק המרכזי שעל השולחן.
- `private ImageView playerCardImageView`: רכיב תצוגת תמונה המציג את קלף המשחק האישי שביד השחקן.
- `private TextView roundTextView`: רכיב טקסט המציג את מספר הסיבוב הנוכחי (למשל: "Round 3").
- `private TextView scoreTextView`: רכיב טקסט המציג את תוצאת המשחק והניקוד העדכני בין שני השחקנים.
- `private TextView gameStatusTextView`: רכיב טקסט המציג הודעות סטטוס ומנחה את המשתמש (למשל: "Find the matching symbol").
- `private String currentPlayerCardId`: מחרוזת טקסט השומרת את ה-ID (המזהה) של הקלף הנוכחי שנמצא בידו של השחקן.
- `private CardDataManager cardDataManager`: מופע של מחלקת העוזר המקומית האחראית לזהות איזה סמל נמצא בכל אזור (Region) של הקלף.
- `private boolean clickEnabled`: משתנה בוליאני המשמש כדגל אבטחה (Flag) למניעת לחיצות כפולות; הוא ננעל (false) ברגע שהמשתמש לוחץ על סמל, ומשתחרר רק לאחר קבלת תשובה מהשרת.
- `private final Emitter.Listener gameStateListener`: מאזין רשת פנימי המופעל ברגע שהשרת משדר אירוע `game_state`. הוא מקבל את נתוני ה-JSON ומעדכן את הלוח.
- `private final Emitter.Listener wrongSymbolListener`: מאזין רשת פנימי המופעל כשהשרת מעדכן על לחיצה שגויה (`wrong_symbol`). הוא צובע את הטקסט באדום ומשחרר את נעילת הלחיצות.
- `private final Emitter.Listener roundResultListener`: מאזין רשת פנימי המקבל את האירוע `round_result` ומציג לשחקנים על המסך מה היה הסמל הנכון של הסיבוב שהסתיים.
- `private final Emitter.Listener gameOverListener`: מאזין רשת פנימי המקבל את האירוע `game_over` ומפעיל את המעבר למסך סיום המשחק.

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המאתחלת את ממשק המשתמש (UI), מוצאת את הרכיבים מה-XML, מאתחלת את ה-`cardDataManager` ומזמנת את הגדרת האזורים והמאזינים.
- `private void setupCardRegions()`: פונקציית עזר הממפה את 9 אזורי הלחיצה הבלתי-נראים שיושבים על גבי תמונת הקלף (כגון: `top-left`, `center-center`, `bottom-right`).
- `private void setupRegion(int viewId, String regionName)`: פונקציה המצמידה מאזין לחיצה (`OnClickListener`) לכל אזור ספציפי בקלף. ברגע שלחיצה מתרחשת, היא שולפת מתוך ה-`cardDataManager` את שם הסמל המדויק שנמצא שם, ומבצעת הבזק ויזואלי ושליחה לשרת.
- `private void flashRegion(View regionView)`: פונקציית אנימציה המבצעת הבזק צבע צהוב-שקוף זמני על האזור שעליו המשתמש לחץ למשך 200 מילישניות כדי לתת לו חיווי ויזואלי.
- `private void sendSymbolClick(String symbol)`: הפונקציה הלוגית המרכזית שאורזת את הלחיצה ושולחת אותה לשרת הפיתוח באמצעות `socket.emit("symbol_click")` יחד עם פונקציית הודעת אישור (Ack) לבדיקת התשובה.

- (private void showToast(String message): פונקציית עזר המקפיצה הודעת Toast קופצת על גבי מסך המשתמש.
 - (private void registerSocketListeners): פונקציה המחברת ומפעילה את ארבעת המאזינים הפנימיים (gameStateListener, wrongSymbolListener, וכו') מול ה-Socket, ובנוסף משדרת בקשה ראשונית לשרת (get_game_state) כדי למשוך את מצב המשחק הנוכחי.
 - (private void openGameOver(JSONObject gameOver): פונקציה המפרקת את אובייקט סיום המשחק, שולפת את השמות והניקוד הסופי של שני השחקנים, ומעבירה אותם באמצעות Intent למסך ה-GameOverActivity תוך סגירת המשחק הנוכחי.
 - (private void updateGameState(JSONObject state): פונקציה המעדכנת את כל רכיבי המסך בבת אחת – משנה את מספרי הסיבוב, מעדכנת את לוח התוצאות הדינמי, וקוראת להצגת תמונות הקלפים החדשות.
 - (private void showCard(ImageView imageView, String cardId): פונקציה דינמית ההופכת את ה-ID של הקלף (למשל "5") לשם של קובץ drawable באפליקציה ("card_5"), מוצאת את ה-Identifier המספרי שלו ומציגה את תמונת הקלף על המסך.
 - (protected void onDestroy): מתודת מחזור חיים הסוגרת בצורה בטוחה ומסודרת את כל מאזיני ה-Socket באמצעות פקודת off() כדי למנוע זליגות זיכרון חמורות (Memory Leaks).
- קוד והסבר של פעולה לוגית מרכזית (sendSymbolClick):
- ```
private void sendSymbolClick(String symbol) {
 // ... (clickEnabled = false), קוד להגדרת ה-Socket ...
 // ... cardId-שנלחץ וה-symbol-עם ה JSON ויצירת ...
 socket.emit("symbol_click", new Object[]{data}, (Ack) args -> {
 // ... (Callback) עיבוד תגובת השרת ...
 // true-מוחזר ל clickEnabled, במידה והלחיצה שגויה או יש שגיאה
 });
}
```
- הסבר: פעולה זו מפעילה את מנגנון התקשורת הדו-כיווני, נועלת את האפשרות ללחיצות נוספות למניעת "ספאם", ושולחת את הסמל שנבחר. השרת מאשר את הלחיצה ומאפשר לשחקן לנסות שוב אם טעה.



### מחלקה 3 - MatchHistoryAdapter:

תפקיד המחלקה:

מחלקת תיווך (Adapter) המקשרת ומסנכרנת בין רשימת אובייקטי המידע של המשחקים הקודמים (MatchItem) לבין רכיב התצוגה הדינמי RecyclerView במסך היסטוריית המשחקים. המחלקה אחראית לנפח ויזואלית (Inflate) את קובץ העיצוב השורתי item\_match.xml, לשלוף את נתוני הניצחון/הפסד, הניקוד, שם היריב והתאריך, ולהציג אותם בשורות חוזרות וחסכוניות בזיכרון המכשיר.

תכונות המחלקה (Fields):

- `private final List<MatchItem> matches`: רשימה קבועה ודינמית (List) המכילה את כל אובייקטי נתוני המשחקים הקודמים שנשמרו בענן.

פעולות המחלקה (Methods):

- `public MatchHistoryAdapter(List<MatchItem> matches)`: בנאי המחלקה (Constructor) המקבל ומאתחל מבחוץ את רשימת המשחקים.
- `public MatchViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType)`: מתודה המנפחת ומייצרת פיזית את המראה הויזואלי של השורה מתוך קובץ ה-XML `R.layout.item_match` ומחזירה מופע חדש של ה-`ViewHolder` הפנימי.
- `public void onBindViewHolder(@NonNull MatchViewHolder holder, int position)`: הפעולה הלוגית המרכזית המקשרת ומזריקה את הנתונים של אובייקט ה-`MatchItem` במיקום ה-`position` אל תוך רכיבי הטקסט הויזואליים שנמצאים בשורה על המסך.
- `public int getItemCount()`: מחזירה את מספר פריטי המשחקים הקיימים ברשימה (`matches.size()`) כדי ליידע את ה-`RecyclerView` כמה שורות עליו לצייר.

קוד והסבר של פעולה לוגית מרכזית (`onBindViewHolder`):

@Override

```
public void onBindViewHolder(@NonNull MatchViewHolder holder, int position) {
 MatchItem match = matches.get(position);
 holder.resultTextView.setText(match.isWon() ? "VICTORY" : "DEFEAT");
 holder.resultTextView.setTextColor(
 holder.itemView.getContext().getColor(
 match.isWon() ? R.color.dobble_purple : android.R.color.holo_red_dark
)
);
 holder.opponentTextView.setText("Against: " + match.getOpponentName());
 holder.matchScoreTextView.setText("Score: " + match.getMyScore() + " - " +
match.getOpponentScore());

 if (match.getCreatedAt() != null) {
 SimpleDateFormat formatter = new SimpleDateFormat("dd/MM/yyyy HH:mm",
Locale.getDefault());
 holder.matchDateTextView.setText(formatter.format(match.getCreatedAt()));
 } else {
 holder.matchDateTextView.setText("");
 }
}
```

הסבר: הפונקציה שולפת את נתוני המשחק לפי המיקום הספציפי שלו ברשימה. היא משתמשת בתנאי מקוצר (Ternary Operator) כדי לבדוק האם השחקן ניצח (match.isWon): אם כן, היא קובעת את הטקסט ל-"VICTORY" וצובעת אותו בסגול; אם לא, הטקסט משתנה ל-"DEFEAT" ונצבע באדום. בנוסף, היא שולפת ומציגה את שם היריב ואת יחס הניקוד. לבסוף, היא לוקחת את רכיב הזמן (Timestamp) ובאמצעות SimpleDateFormat הופכת אותו למחרוזת טקסט קריאה של תאריך ושעה בפורמט מוכר (יום/חודש/שנה שעה:דקות).

מחלקה פנימית - static class MatchViewHolder extends RecyclerView.ViewHolder:

תפקיד המחלקה:

תת-מחלקה פנימית וסטטית המשמשת כמחסן זיכרון קבוע לרכיבי ה-XML של השורה, המונעת קריאות חוזרות ויקרות למעבד.

תכונות המחלקה (Fields):

- TextView resultTextView: רכיב להצגת כותרת תוצאת המשחק (ניצחון או הפסד).
- TextView opponentTextView: רכיב להצגת שם השחקן היריב שהתמודד מול המשתמש.
- TextView matchScoreTextView: רכיב להצגת יחס הנקודות הסופי של המשחק.
- TextView matchDateTextView: רכיב להצגת חותמת הזמן והתאריך שבו נערך המשחק.

פעולות המחלקה (Methods):

- public MatchViewHolder(@NonNull View itemView): הבנאי של מחלקת העזר, המוצא ומקשר באמצעות findViewById את ארבעת רכיבי הטקסט הפיזיים מתוך העיצוב של השורה הבודדת.

## מחלקה 4 - LoginActivity:

תפקיד המחלקה:

מחלקה זו מנהלת את מסך ההתחברות (Login) של האפליקציה. תפקידה לקלוט את פרטי הזיהוי של המשתמש (אימייל וסיסמה), לוודא את תקינות הקלט, ולבצע אימות אסינכרוני מול שירות הענן של Firebase Authentication. בנוסף, המחלקה מיישמת מנגנון כניסה אוטומטית (Auto-Login) – אם המשתמש כבר התחבר בעבר, היא מזהה זאת ומעבירה אותו ישירות למשחק ללא צורך בהקלדה מחדש.

תכונות המחלקה (Fields):

- `private EditText emailEditText`: רכיב קלט טקסט המיועד להקלדת כתובת האימייל של המשתמש.
- `private EditText passwordEditText`: רכיב קלט טקסט מאובטח המיועד להקלדת סיסמת המשתמש.
- `private ProgressBar progressBar`: רכיב ויזואלי של אינדיקטור טעינה המסתובב בזמן שהאפליקציה ממתינה לתשובת הרשת.
- `private FirebaseAuth firebaseAuth`: אובייקט ה-SDK הרשמי של Firebase המשמש לבצוע פעולות האימות והחיבור מול הענן.

פעולות המחלקה (Methods):

- `protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המאתחלת את המסך, מקשרת את העיצוב `activity_login.xml`, שולפת את מופעי ה-`FirebaseAuth` ומגדירה את מאזיני הלחיצה (Listeners) לכפתורי ההתחברות והמעבר להרשמה.
- `protected void onStart()`: מתודת מחזור חיים הפועלת רגע לפני שהמסך הופך לגלוי. היא בודקת האם קיים ששון (Session) פעיל של משתמש מחובר; אם כן, היא מדלגת על מסך ה-Login ומעבירה אותו קדימה.
- `private void login()`: הפעולה הלוגית המרכזית. קוראת את נתוני שדות הטקסט, מבצעת וולידציה (בדיקת תקינות) לשדות, מציגה את ה-`ProgressBar` ומבצעת את בקשת החיבור מול שרתי Firebase בענן.
- `private void setLoading(boolean loading)`: פונקציית עזר המשנה את המצב הויזואלי של ה-`ProgressBar` לגלוי (`View.VISIBLE`) או מוסתר (`View.GONE`) בהתאם לערך הבוליאני שמתקבל.
- `private void openMainActivity()`: פונקציית ניווט המייצרת `Intent` למעבר למסך ה-`MainActivity`, תוך שימוש בדגלים (`FLAG_ACTIVITY_CLEAR_TASK` ו-`FLAG_ACTIVITY_NEW_TASK`) שמנקים את מחסנית המסכים (Backstack) כדי שהמשתמש לא יוכל לחזור למסך ה-Login בלחיצה על כפתור החזור של הטלפון.

קוד והסבר של פעולה לוגית מרכזית (login):

```
private void login() {
 String email = emailEditText.getText().toString().trim();
 String password = passwordEditText.getText().toString();

 if (TextUtils.isEmpty(email)) {
 emailEditText.setError("Enter an email");
 return;
 }
 if (TextUtils.isEmpty(password)) {
 passwordEditText.setError("Enter a password");
 return;
 }

 setLoading(true);
 firebaseAuth.signInWithEmailAndPassword(email, password)
```

```

.addOnCompleteListener(task -> {
 setLoading(false);
 if (task.isSuccessful()) {
 openMainActivity();
 } else {
 String message = task.getException() != null
 ? task.getException().getMessage()
 : "Login failed";
 Toast.makeText(LoginActivity.this, message, Toast.LENGTH_LONG).show();
 }
});
}

```

הסבר:

1. הפונקציה שולפת את המחרוזות מרכיבי הקלט ומנקה רווחים מיותרים מהאימייל בעזרת `.trim()`.
2. היא משתמשת במחלקת העזר `TextUtils.isEmpty` כדי לוודא שאף שדה לא נשאר ריק. במידה ושדה ריק, היא מציגה שגיאה מקומית מובנית (`setError`) ועוצרת את הריצה בעזרת `.return`.
3. היא מפעילה את אנימצית הטעינה ומבצעת פנייה אסינכרונית לרשת בעזרת `.signInWithEmailAndPassword`.
4. היא מצמידה מאזין סיום (`addOnCompleteListener`) שממתין ברקע לתשובה מהשרת של גוגל. כשהתשובה מגיעה, ה-`ProgressBar` מוסתר. אם החיבור הצליח – המשתמש מועבר למסך הראשי. אם החיבור נכשל, הפונקציה מחלצת את סיבת השגיאה המדויקת מהמערכת ומציגה אותה למשתמש בהודעת `Toast` ברורה.

## מחלקה 5 - RegisterActivity:

תפקיד המחלקה:

מחלקה זו מנהלת את מסך רישום המשתמשים החדשים במערכת. תפקידה לקלוט שם תצוגה, אימייל, סיסמה ואימות סיסמה, לבצע וולידציה מקיפה על הקלט, ולייצר חשבון חדש בשרת האימות Firebase Authentication. מיד לאחר מכן, היא מעדכנת את פרופיל המשתמש בשם התצוגה שלו, ומייצרת עבורו רשומה קבועה ומאופסת בבסיס הנתונים Cloud Firestore לטובת ניהול סטטיסטיקות המשחק שלו בעתיד.

תכונות המחלקה (Fields):

- `private EditText nameEditText`: שדה קלט טקסט לקביעת שם התצוגה של המשתמש.
- `private EditText emailEditText`: שדה קלט טקסט להזנת כתובת האימייל.
- `private EditText passwordEditText`: שדה קלט טקסט מאובטח לקביעת הסיסמה החדשה.
- `private EditText confirmPasswordEditText`: שדה קלט טקסט מאובטח לאימות והקלדה שנייה של הסיסמה.
- `private ProgressBar progressBar`: רכיב ויזואלי המציג אינדיקטור טעינה בזמן תהליך הרישום מול הרשת.
- `private Button createAccountButton`: כפתור המפעיל את בדיקות התקינות ותהליך ההרשמה בעת לחיצה.
- `private FirebaseAuth firebaseAuth`: אובייקט ה-SDK של Firebase המשמש ליצירת חשבון המשתמש החדש בענן.
- `private FirebaseFirestore firestore`: אובייקט ה-SDK של Firestore המשמש לכתיבת מסמך המשתמש לבסיס הנתונים.

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המאתחלת את המסך, מקשרת את הרכיבים מקובץ ה-`activity_register.xml`, שולפת מופעים של ה-`Auth` וה-`Firestore`, ומגדירה מאזין לחיצה לכפתור הרישום.
- `(private void register)`: הפעולה הלוגית המרכזית. אוספת את הנתונים משדות הקלט, מבצעת סדרת בדיקות תקינות קפדנית, מפעילה את מנגנון יצירת המשתמש, ובמידה והצליח – מפנה לעדכון הפרופיל, כתיבה ל-Firebase ומעבר למסך הראשי.
- `(private void updateFirebaseProfile(FirebaseUser user, String displayName)`: פונקציה אסינכרונית המעדכנת את רכיב ה-`displayName` הפנימי של המשתמש בתוך שרת האימות של Firebase, ומצמידה מאזין כישלון להצגת הודעה במידה והשם לא נשמר.
- `(private void saveUserToFirestore(FirebaseUser user, String name, String email)`: פונקציה המייצרת מילון נתונים (Map) ומאתחלת את הנתונים הסטטיסטיים של השחקן (ניצחונות, הפסדים, ניקוד) לאפס, ושומרת את המסמך תחת ה-`UID` שלו ב-Firebase.
- `(private void setLoading(boolean loading)`: פונקציית עזר המשנה את נראות ה-`ProgressBar` ובו-זמנית נועלת או משחררת את כפתור הרישום כדי למנוע לחיצות כפולות בזמן פנייה לרשת.
- `(private void openMainActivity)`: פונקציית ניווט המעבירה את השחקן ל-`MainActivity` ומנקה את היסטוריית המסכים כדי למנוע חזרה למסך הרישום.

קוד והסבר של פעולה לוגית מרכזית (`register`):

(הקוד ארוך מידי, אפשר לראות אותו בנספחים בסוף)

הסבר:

הפונקציה (`register`) מבצעת את שלבי הרישום הבאים:

1. איסוף וולידציה: קריאת הנתונים משדות הקלט (`name, email, password`) וביצוע בדיקות תקינות - שדות ריקים או סיסמה קצרה מדי מציגים שגיאה.
2. יצירת משתמש: הפעלת (`setLoading(true)`) וביצוע `createUserWithEmailAndPassword` מול Firebase.
3. טיפול בתוצאות:

- במקרה של כישלון (למשל אימייל קיים), מוצגת הודעת Toast עם ה-Exception המתאים, וביטול הטעינה.
- במקרה של הצלחה, המשתמש נוצר, המערכת קוראת ל-updateFirebaseProfile (עדכון שם), saveUserToFirestore (התחלת נתוני משתמש), ומעבירה למסך הראשי.

## מחלקה 6 - RoomActivity:

תפקיד המחלקה:

מחלקה זו מנהלת את מסך "חדר ההמתנה" (Lobby Room) שלפני תחילת המשחק. תפקידה המרכזי הוא להציג את קוד החדר, להאזין בזמן אמת לשינויים ברשימת המשתתפים (כניסה או עזיבה של שחקנים) בעזרת פרוטוקול WebSockets, ולשקף זאת מיידית על גבי ממשק המשתמש. בנוסף, המחלקה אוכפת הרשאות ניהול: רק השחקן שהוגדר כיוצר ומנהל החדר (isOwner) רשאי לראות ולהקליק על כפתור התחלת המשחק מול השרת.

תכונות המחלקה (Fields):

- `private TextView roomCodeTextView`: רכיב טקסט להצגת הקוד הייחודי והסודי של החדר הנוכחי על גבי המסך.
- `private TextView statusTextView`: רכיב טקסט המציג הודעות סטטוס משתנות (למשל: "Waiting for...another player" או "Both players are ready").
- `private TextView playerOneTextView`: רכיב טקסט המציג את שם התצוגה של השחקן הראשון בחדר.
- `private TextView playerTwoTextView`: רכיב טקסט המציג את שם התצוגה של השחקן השני בחדר.
- `private Button startGameButton`: כפתור המשמש להתחלת המשחק (מוצג באופן דינמי רק עבור מנהל החדר).
- `private String roomCode`: מחרוזת טקסט השומרת את קוד החדר הנוכחי שנתקבל מהמסך הקודם.
- `private String currentUid`: מחרוזת השומרת את מזהה ה-UID הייחודי של המשתמש המחובר כרגע באפליקציה.
- `private boolean isOwner`: דגל בוליאני המגדיר האם המשתמש הנוכחי הוא מנהל ויוצר החדר (true) או שחקן אורח (false).
- `private final Emitter.Listener roomUpdatedListener`: מאזין רשת פנימי המופעל ברגע ששרת הפייתון משדר אירוע `room_updated`. הוא תופס את נתוני ה-JSONObject, מעביר את הריצה לחוט ה-UI המרכזי, ומעדכן את שמות השחקנים ומצב החדר.
- `private final Emitter.Listener gameStartedListener`: מאזין רשת פנימי המופעל כשהשרת משדר אירוע `game_started`. הוא מקפיץ מיד את השחקן אל מסך המשחק הפעיל (GameActivity).

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המאתחלת את המסך מקובץ ה-`activity_room.xml`, שולפת את נתוני ה-Intent (הקוד ונתוני החדר הראשוניים), מאמתת שקיים משתמש מחובר ב-Firebase, מקשרת את הכפתורים ורושמת את מאזיני ה-Socket.
- `private void registerSocketListeners()`: פונקציה השולפת את הצינור המרכזי מתוך ה-SocketManager ומחברת פיזית את המאזינים הפנימיים לאירועים `room_updated` ו-`game_started`.
- `private void updateRoomUi(JSONObject room)`: פונקציה המפרקת את אובייקט ה-JSON של החדר. היא משווה את ה-`ownerUid` למשתמש הנוכחי כדי לקבוע האם הוא המנהל, עוברת על מערך השחקנים (players), מעדכנת את שמותיהם על המסך, וקובעת את תנאי הנראות של כפתור ההתחלה.
- `private void startGame()`: הפעולה הלוגית המרכזית. מופעלת בלחיצה על כפתור ההתחלה על ידי מנהל החדר. היא מוודאת חיבור, מגדירה את יעד הניקוד לסיכום המשחק, ומשדרת אירוע `start_game` לשרת יחד עם מאזין אישור (Ack Callback) לטיפול בשגיאות.
- `private void leaveRoom()`: מופעלת בלחיצה על כפתור העזיבה. היא משדרת הודעת `leave_room` לשרת ובסיומה מחזירה את המשתמש למסך הראשי.
- `private void openMain()`: פונקציית ניווט המעבירה את השחקן בצורה בטוחה אל ה-MainActivity ומנקה את מחסנית המסכים.
- `private void showToast(String message)`: פונקציית עזר המציגה הודעת Toast ארוכה על גבי המסך.

- `protected void onDestroy()`: מתודת מחזור חיים המסירה ומכבה בצורה מוחלטת את המאזינים `room_updated` ו-`game_started` מה-`Socket` כדי למנוע זליגות זיכרון והרצות כפולות ברקע. קוד והסבר של פעולה לוגית מרכזית (`startGame`):

```
private void startGame() {
 if (!isOwner) { return; }
 Socket socket = SocketManager.getInstance().getSocket();
 if (socket == null || !socket.connected()) {
 showToast("Not connected to server");
 return;
 }
 // ... [start_game ושידור האירוע JSON קוד ליצירת]
 socket.emit("start_game", data, (Ack) args -> {
 // ... [לניהול שגיאות Callback-טיפול בתגובת השרת, כולל ה]
 });
}
```

הסבר:

- מתודה זו, הפעילה רק עבור מנהל החדר, בודקת חיבור תקין לשרת, מגדירה יעד ניקוד ומשיקה את המשחק.
1. אבטחה: וידוא שרק `isOwner` מריץ את הפעולה.
  2. חיבור ונתונים: בדיקת חיבור ל-`Socket` ויצירת `JSON` עם יעד הניקוד.
  3. שידור: שליחת אירוע `start_game` לשרת עם `Ack Callback` לטיפול בתשובה.
  4. טיפול בתשובה: אם השרת מחזיר שגיאה (`ok: false`), הפונקציה מציגה הודעת `Toast` מתאימה דרך `.runOnUiThread`.



## מחלקה 7 - MatchHistoryActivity:

תפקיד המחלקה:

מחלקה זו מנהלת את מסך היסטוריית המשחקים של המשתמש האפליקטיבי. תפקידה לבצע שאילתות מורכבות מול בסיס הנתונים Cloud Firestore כדי לשלוף את כל המשחקים שבהם המשתמש הנוכחי לקח חלק. מאחר והמידע על היריבים עשוי לדרוש השלמה, המחלקה מחלצת ומפענחת את שמות השחקנים היריבים, מחשבת אם המשתמש ניצח או הפסיד, ממירה ערכים מספריים לטיפוס אחיד, ומסדרת את הרשימה בסדר כרונולוגי יורד (מהמשחק החדש ביותר לישן ביותר) לצורך הצגה ב-RecyclerView.

תכונות המחלקה (Fields):

- `private final List<MatchItem> matches`: רשימה דינמית קבועה המאחסנת את אובייקטי המידע של המשחקים לאחר עיבודם וטעינתם, והיא זו שמוזנת לאדפטר.
- `private final Map<String, String> userNames`: מילון (Cache מקומי) המשמש למיפוי בין מזהה ה-UID של השחקן היריב לשם התצוגה שלו, כדי למנוע קריאות רשת כפולות ומיותרות ל-Firebase עבור אותו משתמש.
- `private MatchHistoryAdapter adapter`: מופע של האדפטר המקשר בין רשימת הנתונים לרכיב ה-RecyclerView הוויזואלי.
- `private ProgressBar historyProgressBar`: רכיב אינדיקטור טעינה המסתובב כל עוד מתבצעת שליפת נתונים ועיבודם מהרשת.
- `private TextView emptyHistoryTextView`: רכיב טקסט המוצג למשתמש רק במידה ואין לו אף משחק קודם מתועד במערכת.
- `private FirebaseFirestore firestore`: אובייקט ה-SDK הרשמי המאפשר גישה, קריאה וביצוע שאילתות מול מסמכי בסיס הנתונים בענן.
- `private int pendingMatches`: משתנה מונה (Counter) העוקב אחר מספר המשחקים שטרם סיימו את תהליך העיבוד האסינכרוני, ומאפשר לדעת מתי כל המידע מוכן להצגה סופית.

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המנפחת את העיצוב `activity_match_history.xml`, מאתחלת את ה-RecyclerView, יוצרת את האדפטר ומפעילה מאזין חזור לכפתור ה-Back המקומי שמסיים את האקטיביטי.
- `(private void loadMatches)`: מאמתת שקיים משתמש מחובר ומפעילה שאילתת סינון (`whereArrayContains`) על אוסף ה-`matches` בענן כדי לשווק את ההיסטוריה הרלוונטית בלבד.
- `(private void loadSingleMatch(QueryDocumentSnapshot document, String currentUid)`: מפרקת מסמך משחק בודד, מחלצת את ה-UID של היריב, את הניקוד האישי והיריב, בודקת האם קיים רישום קבוע של שם היריב במסמך המשחק, ובמידה ולא – קוראת לטעינת השם מקולקשן המשתמשים.
- `(private void loadOpponentName(String opponentUid, int myScore, int opponentScore, boolean won, Date createdAt)`: פונקציה אסינכרונית הבודקת קודם כל ב-Cache המקומי (`userNames`), ואם השם לא קיים שם, היא פונה עצמאית אל מסמך המשתמש הספציפי באוסף ה-`users` כדי לדלות את ה-`displayName` או ה-`email` כגיבוי.
- `(private void addMatch(String opponentName, int myScore, int opponentScore, boolean won, Date createdAt)`: מייצרת מופע חדש של `MatchItem` עם כל הנתונים המעובדים ומוסיפה אותו לרשימת ה-`matches` המרכזית.
- `(private void onMatchLoaded)`: מפחיתה את מונה ה-`pendingMatches`. ברגע שהמונה מגיע ל-0 (כלומר כל המשחקים עובדו לחלוטין), היא מפעילה אלגוריתם מיון מסוג Comparator המסדר את המשחקים לפי התאריך שלהם מהחדש לישן, וקוראת לסיום הטעינה.

- `private void finishLoading()`: מעלימה את ה-`ProgressBar` מהמסך, קובעת האם להציג הודעת "היסטוריה ריקה" בהתאם למצב הרשימה, ומבצעת `notifyDataSetChanged()` לאדפטר כדי לרענן את ה-`RecyclerView` באופן וירטואלי למשתמש.
- `private int numberToInt(Object value)`: פונקציית עזר בטיחותית המוודאת שערכי הניקוד המתקבלים מ-`Firestore` (העשויים להגיע כטיפוסים מספריים שונים כגון `Long`) יומרו בצורה תקינה ויציבה למשתנה מסוג `int` ללא גרימת קריסה.  
קוד והסבר של פעולה לוגית מרכזית (`loadMatches`):

```
private void loadMatches() {
 FirebaseAuth.getInstance().getCurrentUser();
 if (user == null) { finish(); return; }
 String currentUid = user.getId();

 firestore.collection("matches")
 .whereArrayContains("players", currentUid)
 .get()
 .addOnSuccessListener(querySnapshot -> {
 matches.clear();
 if (querySnapshot.isEmpty()) {
 finishLoading();
 return;
 }
 pendingMatches = querySnapshot.size();
 for (QueryDocumentSnapshot document : querySnapshot) {
 loadSingleMatch(document, currentUid);
 }
 })
 .addOnFailureListener(error -> {
 historyProgressBar.setVisibility(View.GONE);
 Toast.makeText(MatchHistoryActivity.this, error.getMessage(),
 Toast.LENGTH_LONG).show();
 });
}
```

הסבר:

הפונקציה ניגשת אל אוסף המשחקים ב-`Firestore` ומפעילה מסנן מובנה הבדוק האם מערך השחקנים (`players`) של המשחק מכיל בתוכו את ה-`UID` של המשתמש הנוכחי. השאילתה מתבצעת בצורה אסינכרונית ברקע. במידה והשליפה הצליחה, היא מנקה את הרשימה הישנה, מגדירה את כמות המשחקים לעיבוד (`pendingMatches`) ומריצה לולאת `for` העוברת מסמך מסמך ומפרקת אותו דרך `loadSingleMatch`. במידה ויש שגיאת רשת, המאזין המשני תופס אותה, מעלים את הבר ומקפיץ הודעת שגיאה מסודרת.

**מחלקה 8 - MatchItem:**

תפקיד המחלקה:

מחלקה זו משמשת כמחלקת מודל (Data Model) נקייה ופשוטה בתצורת POJO (Plain Old Java Object). תפקידה הבלעדי הוא לייצג ולשמור בזיכרון המכשיר ישות אחידה של נתוני משחק בודד מן העבר (היסטוריית משחק). המחלקה מאגדת את כלל המידע המעובד שנשאב מ-Firebase (תאריך, ניקוד, תוצאה ושם יריב), ומאפשרת ל-MatchHistoryAdapter לשלוף נתונים אלו בקלות ובמהירות עבור כל שורה ב-RecyclerView. תכונות המחלקה (Fields):

- `private final String opponentName`: מחרוזת טקסט קבועה השומרת את שם התצוגה היוזואלי של השחקן היריב שהתמודד מול המשתמש.
- `private final int myScore`: משתנה מספרי שלם המכיל את כמות הנקודות הסופית שצבר המשתמש הנוכחי באותו משחק.
- `private final int opponentScore`: משתנה מספרי שלם המכיל את כמות הנקודות הסופית שצבר השחקן היריב.
- `private final boolean won`: דגל בוליאני קבוע המגדיר האם המשתמש הנוכחי ניצח במשחק (true) או הפסיד (false).
- `private final Date createdAt`: אובייקט מסוג Date המאחסן את חותמת הזמן, התאריך והשעה המדויקים שבהם נערך והסתיים המשחק בענן.

פעולות המחלקה (Methods):

- `public MatchItem(String opponentName, int myScore, int opponentScore, boolean won, Date createdAt)`: בנאי המחלקה (Constructor) המשמש לאתחול והזרקה של כל חמשת ערכי התכונות של אובייקט המידע ברגע יצירתו בזיכרון המערכת. קוד והסבר של פעולה לוגית מרכזית (MatchItem):

```
public MatchItem(
 String opponentName,
 int myScore,
 int opponentScore,
 boolean won,
 Date createdAt
) {
 this.opponentName = opponentName;
 this.myScore = myScore;
 this.opponentScore = opponentScore;
 this.won = won;
 this.createdAt = createdAt;
}
```

הסבר:

הסבר מפורט: מילת המפתח `this` משמשת את המהדר (Compiler) כדי להבדיל בצורה מוחלטת בין הפרמטרים שמתקבלים מחוץ למחלקה בתוך סוגרי הפונקציה, לבין תכונות האובייקט הפנימיות של ה-Class עצמו. השימוש במילת המפתח `final` על גבי התכונות מבטיח הגנה על שלמות הנתונים (Immutability) – ברגע שהבנאי מאתחל את ערכי המשחק הישן, לא ניתן לשנות או לערוך אותם יותר לעולם, מה שמונע טעויות לוגיות ודריסת נתונים היסטוריים בזמן ריצת האפליקציה.

## מחלקה 9 - MainActivity:

תפקיד המחלקה:

מחלקה זו מהווה את מסך הבית והתפריט הראשי (Lobby / Dashboard) של האפליקציה לאחר התחברות מוצלחת. תפקידה המרכזי הוא לנהל את החיבור הראשוני והמאובטח מול שרת הפיתוח העצמאי באמצעות העברת ה-ID Token של Firebase. המחלקה טוענת את פרופיל השחקן מתוך ה-Firebase, מציגה סטטוס חיבור רשתי דינמי, ומאפשרת למשתמש לבצע פעולות ליבה: יצירת חדר משחק חדש, הצטרפות לחדר קיים באמצעות קוד בן 6 ספרות, מעבר למסך היסטוריית המשחקים, או ביצוע התנתקות מסודרת מהמערכת.

תכונות המחלקה (Fields):

- `private FirebaseAuth firebaseAuth`: אובייקט ה-SDK של Firebase לניהול המשתמש המחובר ואימותו.
- `private FirebaseFirestore firestore`: אובייקט ה-SDK לגישה ושליפת נתוני הפרופיל מבסיס הנתונים בענן.
- `private TextView userTextView`: רכיב טקסט המציג הודעת ברכה אישית עם שם השחקן (למשל: "Hello, Meydan").
- `private TextView serverStatusTextView`: רכיב טקסט המציג למשתמש את הסטטוס העדכני של החיבור מול השרת בזמן אמת.
- `private ProgressBar serverProgressBar`: רכיב אינדיקטור טעינה המסתובב בזמן קבלת טוקן או התחברות לשרת.
- `private EditText roomCodeEditText`: שדה קלט טקסט שבו המשתמש מזין את קוד החדר (בן 6 ספרות) כדי להצטרף לחברים.
- `private Button createRoomButton`: כפתור המפעיל את תהליך בקשת יצירת החדר מול השרת.
- `private Button joinRoomButton`: כפתור המפעיל את תהליך ההצטרפות לחדר על בסיס הקוד שהוקלד.
- `private Button reconnectButton`: כפתור המאפשר למשתמש לנסות להתחבר מחדש לשרת באופן דינמי במידה והחיבור נכשל.
- `private String displayName`: מחרוזת טקסט השומרת את שם התצוגה המעובד והרשמי של השחקן לצורך העברתו לשרת.

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המאתחלת את רכיבי המסך מ-`activity_main.xml`, מוודאת שקיים משתמש מחובר (ואם לא - מפנה ל-Login), טוענת את שם המשתמש, מחברת מאזיני לחיצה לכל הכפתורים ומשיקה את החיבור לשרת.
- `(private void loadDisplayName(FirebaseUser user)`: פונקציה אסינכרונית השולפת את ה-`displayName` של השחקן מתוך קולקשן ב-`users` ב-Firebase. היא כוללת מנגנוני גיבוי (Fallback) - אם אין שם ב-Firebase היא לוקחת את השם מאימות ה-Firebase, ואם גם שם אין, היא משתמשת בכתובת האימייל שלו.
- `(private void connectToServer()`: הפעולה הלוגית המרכזית הראשונה. שולפת בצורה אסינכרונית Token אבטחה מעודכן מ-Firebase, ומעבירה אותו ל-`SocketManager` לצורך התחברות מאובטחת ואימות מול שרת הפיתוח באמצעות ממשק מאזין פנימי.
- `(private void createRoom()`: פונקציה המופעלת בלחיצה על כפתור יצירת החדר. היא מוודאת חיבור רשת תקין, אורזת את ה-`displayName` בתוך `JSONObject`, ומשדרת אירוע `create_room` לשרת יחד עם Callback מסוג Ack לקבלת התשובה.
- `(private void joinRoom()`: הפעולה הלוגית המרכזית השנייה. קוראת את קוד החדר, מבצעת בדיקות תקינות (וולידציה שהקוד אינו ריק ואורכו בדיוק 6 תווים), ואם הכל תקין - משדרת אירוע `join_room` לשרת עם נתוני הקוד והשם.

- `(private void handleRoomResponse(Object responseObject)`: פונקציית עזר המעבדת את תשובת השרת (ה-Callback) עבור יצירה או הצטרפות לחדר. היא בודקת האם הפעולה הצליחה (`ok: true`), ואם כן - מחלצת את נתוני ה-JSON של החדר ומפנה לפתיחתו.
  - `(private void openRoom(String roomCode, String roomJson`: פונקציית ניווט המייצרת `Intent`, מצרפת אליו כנתונים נוספים (`putExtra`) את קוד החדר ואת נתוני החדר המלאים, ומעבירה את השחקן למסך ה-`RoomActivity`.
  - `(private void setConnectionLoading(boolean loading)`: משנה ויזואלית את נראות ה-`ProgressBar` ונועלת/משחררת את כפתור החיבור מחדש בהתאם למצב הרשת.
  - `(private void setRoomButtonsEnabled(boolean enabled)`: נועלת או משחררת את כפתורי יצירת והצטרפות לחדר, כדי למנוע מהמשתמש ללחוץ עליהם כל עוד אין חיבור רשת פעיל ומאושר מול שרת הפיתוח.
  - `(private void showToast(String message)`: פונקציית עזר המציגה הודעת `Toast` ארוכה על גבי מסך המשתמש.
  - `(private void logout)`: מנתקת בצורה מסודרת את חיבור ה-`Socket`, מבצעת `signOut` ל-`Firebase Auth` ומעבירה את המשתמש חזרה למסך ה-`Login`.
  - `(private void openLogin)`: פונקציית ניווט המעבירה את המשתמש למסך ה-`LoginActivity` ומנקה את מחסנית המסכים כדי למנוע חזרה במסך.
- קוד והסבר של פעולה לוגית מרכזית (`connectToServer`):
- הקוד ארוך ביותר ויהיה זמין בסוף הספר הסבר:
1. שלב א' (`Firebase Token`): האפליקציה פונה אסינכרונית לשרתי גוגל בעזרת `user.getIdToken(true)` כדי לחלץ את טוקן האבטחה המוצפן הייחודי של המשתמש הנוכחי.
  2. שלב ב' (שידור לשרת עצמאי): עם קבלת הטוקן, הוא מועבר לפקודת ה-`connect` של ה-`SocketManager`. הפונקציה מייצרת ממשק מאזין אנונימי בזמן אמת (`ConnectionListener`) הכולל 4 מצבים (חיבור, קבלה, שגיאה, וניתוק).
  3. ניהול חוטים (`Threading`): מאחר והודעות ה-`Socket` מתקבלות בחוט רקע (`Background Thread`), הפונקציה משתמשת ב-`runOnUiThread` בכל אחד מהמצבים כדי לעדכן בצורה בטוחה את רכיבי התצוגה הויזואליים ושחרור/נעילת הכפתורים על גבי המסך מבלי לגרום להתנגשות וקריסה של האפליקציה.

## מחלקה 10 - GameOverActivity:

תפקיד המחלקה:

מחלקה זו מנהלת את מסך סיום המשחק (Game Over) באפליקציה. תפקידה המרכזי הוא לקלוט את נתוני סיכום המשחק שהועברו אליה מתוך מסך המשחק, לזהות בצורה דינמית האם השחקן הנוכחי ניצח או הפסיד בהתאם ל-UID שלו מול ה-UID של המנצח הרשמי, ולהציג חיווי ויזואלי של התוצאות, שמות המשתתפים, הניקוד הסופי וסטטוס שמירת הנתונים בענן. בנוסף, המחלקה אוכפת ניווט בטוח חזרה למסך הבית תוך ניקוי היסטוריית המסכים.

תכונות המחלקה (Fields):

בקובץ קוד זה, רכיבי התצוגה והמאזינים אינם מוגדרים כתכונות גלובליות של המחלקה, אלא כמשתנים מקומיים (Local Variables) מוגדרים בתוך מתודת האתחול, דבר המייעל את ניהול הזיכרון כיוון שלא מתבצעות עליהם מניפולציות מחוץ לשלב טעינת המסך.

- `TextView resultTitleTextView`: רכיב טקסט המציג את כותרת התוצאה הראשית ("YOU WON") או ("YOU LOST").
- `TextView resultSubtitleTextView`: רכיב טקסט המציג כותרת משנה מנחמת או מברכת בהתאם לתוצאה.
- `TextView finalScoreTextView`: רכיב טקסט המציג את יחס הניקוד הסופי והמספרי של הסיבוב (לדוגמה: "7 - 5").
- `TextView playersTextView`: רכיב טקסט המציג את שמות שני השחקנים שהתמודדו זה מול זה (לדוגמה: "Meydan vs Player2").
- `TextView saveStatusTextView`: רכיב טקסט המציג אישור ויזואלי האם תוצאות המשחק נשמרו בהצלחה בבסיס הנתונים בענן Firestore.
- `Button backHomeButton`: כפתור המאפשר לשחקן לצאת ממסך הסיכום ולחזור לתפריט הראשי של האפליקציה.

פעולות המחלקה (Methods):

- `(protected void onCreate(Bundle savedInstanceState)`: מתודת מחזור חיים המנפחת את עיצוב ה-XML `activity_game_over.xml`, מאתרת את רכיבי הממשק, שולפת את חבילת הנתונים המורחבת (Intents Data) שהגיעה ממסך המשחק, מאמתת את השחקן מול Firebase Auth, ומבצעת את קביעת תוכן הטקסטים והמאזינים הפיזיים.
- `(private void returnHome())`: הפעולה הלוגית המרכזית. מנווטת את השחקן בצורה מאובטחת חזרה למסך התפריט הראשי, תוך שימוש בדגלי רשת שמנקים את מחסנית האקטיביטז כדי שלא יהיה ניתן לחזור אחורה למשחק שהסתיים.
- `(public void onBackPressed)`: דריסה (Override) של כפתור החזור המובנה של מכשיר הטלפון. במקום לאפשר למשתמש לחזור באופן לא חוקי למסך המשחק שכבר נסגר בשרת, היא מנתבת אותו ומפעילה את פונקציית ה-`(returnHome)` הרישמית.

קוד והסבר של פעולה לוגית מרכזית (`(returnHome)`):

```
private void returnHome() {
 Intent intent = new Intent(
 GameOverActivity.this,
 MainActivity.class
);
 intent.addFlags(
 Intent.FLAG_ACTIVITY_CLEAR_TOP
 | Intent.FLAG_ACTIVITY_NEW_TASK
);
 startActivity(intent);
}
```

```
finish();
}
```

הסבר:

1. הפונקציה מייצרת עצם מסוג Intent המגדיר מעבר ממסך הסיכום הנוכחי אל מסך התפריט הראשי MainActivity.
2. היא משלבת אופרטור לוגי מסוג OR בינארי כדי להצמיד שני דגלי מערכת קריטיים: FLAG\_ACTIVITY\_NEW\_TASK ו-FLAG\_ACTIVITY\_CLEAR\_TOP. דגלים אלו מנחים את מערכת ההפעלה אנדרואיד לסגור, להשמיד ולנקות מן הזיכרון את כל מסכי הביניים שהיו פתוחים בדרך (כמו מסך חדר המשחק או הלובי).
3. היא משיקה את המעבר הוויזואלי בעזרת startActivity ומסיימת סופית את חייה של האקטיביטי הנוכחית באמצעות finish() כדי לפנות משאבי מעבד וזיכרון RAM במכשיר.

## מחלקה 11 - CardDataManager:

תפקיד המחלקה:

מחלקה זו משמשת כמנהל משאבים מקומי (Data Asset Manager) באפליקציה, וממומשת בתצורת סינגלטון (Singleton) השומרת על מופע יחיד בזיכרון. תפקידה הקריטי הוא לטעון, לפענח ולשמור בזיכרון ה-RAM את קובץ המיפוי המבני Cards.json מתוך תיקיית ה-Assets של המכשיר. קובץ זה מגדיר אילו סמלים (מחרוזות) משויכים לכל אזור לחיצה ויזואלי (Region) בכל אחד מקלפי המשחק. המחלקה מאפשרת ל-GameActivity לבצע תרגום מהיר: כאשר משתמש לוחץ על קואורדינטה או אזור מסוים בקלף, היא מחזירה מיד את שם הסמל המדויק שנמצא שם, לצורך שליחתו לשרת.

תכונות המחלקה (Fields):

- `private static CardDataManager instance` (תכונה סטטית): שומרת את המופע היחיד והקבוע של המחלקה בזיכרון לאורך ריצת האפליקציה.
- `private final Map<String, Map<String, String>> cards` (Nested HashMap). המפתח החיצוני הוא מזהה הקלף (`cardId`), והערך הוא מילון פנימי הממפה בין שם אזור הלחיצה (`regionName`) לשם הסמל המדויק שנמצא באותו אזור.

פעולות המחלקה (Methods):

- `private CardDataManager(Context context)` (בנאי פרטי): מפעיל בצורה מוגנת ומיידידת את פונקציית טעינת הקלפים. מוגדר כפרטי כדי למנוע אפשרות ליצירת עצמים חיצוניים.
- `public static synchronized CardDataManager getInstance(Context context)`: הפונקציה הסטטית המרכזת המחזירה את המופע הייחודי של המחלקה. משתמשת ב-`context.getApplicationContext()` כדי למנוע זליגת זיכרון של אקטיביטיז, וכוללת הגנת סנכרון (`synchronized`) ברמת חוטים.
- `private void loadCards(Context context)`: הפעולה הלוגית המרכזת הראשונה. פותחת זרם קלט (`InputStream`) אל קובץ ה-JSON, קוראת אותו שורה אחר שורה, מפענחת את המבנה המקונן בעזרת איטרטורים של מפתחות, ומאכלסת את מבנה הנתונים `cards`.
- `public String getSymbol(String cardId, String regionName)`: הפעולה הלוגית המרכזת השנייה. משמשת כפונקציית שליפה מהירה (Query Method) המקבלת מזהה קלף ואזור, ומחזירה את שם הסמל המשוך.

קוד והסבר של פעולה לוגית מרכזית (`loadCards`):

הקוד ארוך ויהיה זמין בסוף הספר

הסבר:

פונקציה זו טוענת את המידע על הקלפים בזמן עליית האפליקציה:

1. קריאת הקובץ: פותחת את `Cards.json` מתיקיית ה-Assets וקוראת אותו לתוך `StringBuilder` באמצעות `BufferedReader`.
2. פענוח ומבנה: הופכת את הטקסט ל-`JSONObject` ומפענחת אותו (`Parsing`).
3. טעינה למפה: משתמשת בלולאות איטרטור (`while (cardIds.hasNext())`) כדי לעבור על כל הקלפים והאזורים, וממלאת את המפה המקוננת `cards`.
4. טיפול בשגיאות: כוללת בלוק `catch` שמבטיח זריקת `RuntimeException` אם הקובץ פגום או חסר.



## מחלקות – צד שרת (Python)

### מחלקה 1 - app.py:

תפקיד המחלקה:

קובץ זה מהווה את נקודת הכניסה הראשית (Entry Point) של שרת הפיתון ומנהל החומרים המרכזי שלו. תפקידו לאתחל את אפליקציית ה-Web בתצורת ASGI, להקים את שרת ה-Socket.IO האסינכרוני, ולטעון את כל מנהלי הלוגיקה העסקית (Managers) של המשחק. הקובץ מאזין לפורט קבוע ברשת (כברירת מחדל פועל על פורט 5000) ומנתב בזמן אמת את כל הודעות ה-WebSockets הדו-כיווניות שמגיעות ממכשירי האנדרואיד (כמו התחברות, ניהול חדרים, ולחיצות על סמלים) אל המנהלים המתאימים, תוך הפצת עדכוני מצב סינכרוניים לכל השחקנים בחדר.

תכונות המחלקה (Fields):

- `card_manager`: מופע של המחלקה `CardDataManager` האחראי על טעינת ופיענוח חפיסת הקלפים מקובץ ה-JSON.
  - `auth_manager`: מופע של המחלקה `AuthManager` האחראי על אימות השחקנים והטוקנים מול ה-Firebase.
  - `room_manager`: מופע של המחלקה `RoomManager` המנהל את מערך חדר המתנה (Lobbies) בזיכרון.
  - `game_manager`: מופע של המחלקה `GameManager` המנווט את לוגיקת ריצת המשחקים הפעילים.
  - `stats_manager`: מופע של המחלקה `StatsManager` האחראי על כתיבת תוצאות המשחקים ל-Firebase.
  - `sio`: אובייקט מסוג `AsyncServer` המנהל פיזית את פרוטוקול ה-WebSockets, פתיחת הששונים (Sessions) והפצת המידע.
  - `app`: מופע מסוג `ASGIApp` העוטף את שרת ה-Socket.IO ומאפשר לו לרוץ על גבי שרת ה-Uvicorn.
- פעולות המחלקה (Methods):
- `success(data)`: פונקציית עזר המייצרת ומחזירה מילון (Dictionary) המייצג תגובה מוצלחת ברשת הכוללת את הדגל `ok: true`.
  - `failure(message)`: פונקציית עזר המייצרת ומחזירה מילון המייצג תגובה שנכשלה הכוללת את הדגל `ok: false` והודעת השגיאה.
  - `emit_private_game_states(room)`: לולאה אסינכרונית העוברת על כל השחקנים בחדר, שולפת לכל אחד את מצב הקלפים הפרטי שלו, ודוחפת לו את זה ל-`sid` הייחודי שלו.
  - `connect(sid, environ, auth)`: מאזין המופעל בחיבור לקוח (`connect`). הוא מעביר את ה-Payload ל-`auth_manager`, שומר את נתוני המשתמש מהסשן ומחזיר חיבור מוצלח לאנדרואיד.
  - `disconnect(sid, reason)`: מאזין המופעל בניתוק לקוח (`disconnect`). הוא מוציא את השחקן מהחדר ב-`room_manager`, ואם המשחק היה פעיל – הוא מבטל אותו ומעדכן את היריב בהודעת ניתוק.
  - `ping_server(sid, data)`: מאזין המשמש לבדיקת יציבות הרשת (Heartbeat) ומחזיר הודעת "ping" מיידית ללקוח.
  - `create_room(sid, data)`: מושך את נתוני המשתמש מהסשן, קורא ל-`room_manager` ליצירת חדר, ומכניס את הלקוח לתוך קבוצת ה-Socket של אותו החדר.
  - `join_room(sid, data)`: קורא את קוד החדר שנשלח מהאנדרואיד, מאמת תקינות, מפעיל את ה-`room_manager` להצטרפות, ומצרף את הלקוח לקבוצת ה-Socket של החדר.
  - `leave_room(sid, data)`: מאפשר לשחקן לעזוב חדר באופן יזום, ומבטל את המשחק במידה והוא רץ באותו רגע.
  - `start_game(sid, data)`: מוודא שרק מנהל החדר מנסה להפעיל את המשחק, קורא ל-`game_manager` לאתחול חלוקת הקלפים הראשונית, ומשדר הודעת `game_started` לכל חברי החדר.
  - `get_game_state(sid, data)`: פונקציית שליפה המאפשרת למכשיר אנדרואיד לבקש ולקבל את מצב המשחק הפרטי והעדכני שלו.

- `async def symbol_click(sid, data)`: המאזין הלוגי המרכזי של המשחק. מקבל את שם הסמל ומזהה הקלף שלחצו עליו באנדרואיד, ומעביר אותם לבדיקה במנוע המשחק. קוד והסבר של פעולה לוגית מרכזית (`symbol_click`): הקוד ארוך במיוחד ויהיה זמין בסופו של הפרויקט. הסבר: כאשר שחקן לוחץ על סמל באנדרואיד, אירוע זה מופעל בשרת. הפונקציה מחלצת את הסמל ומזהה הקלף, מוודאת שהם אינם ריקים, ושולפת את אובייקט החדר והמשתמש לפי ה-`sid`. היא פונה ל-`game_manager.submit_symbol` שמכריע האם להחזיר נכונה.
- במקרה של טעות: השרת דוחף הודעת `wrong_symbol` אך ורק למכשיר הטלפון של השחקן שטעה (`to=sid`) כדי לצבוע את המסך שלו באדום.
- במקרה של נכונות: השרת מפיץ לכל החדר את ה-`round_result`. אם המשחק הגיע לסיומו (`gameFinished`), השרת פונה ל-`stats_manager` כדי לשמור את היסטוריית המשחק ב-Firestore ומפיץ לכולם הודעת `game_over` למעבר למסך הסיכום; אם המשחק נמשך, הוא מגריל קלפים חדשים ומפיץ לכל שחקן את המצב הפרטי המעודכן שלו.

## מחלקה 2 - room\_manager.py:

תפקיד המחלקה:

מחלקה זו אחראית על ניהול "הלובי" וחדרי המתנה בשרת. תפקידה המרכזי הוא לייצר קודים דינמיים וייחודיים בני 6 ספרות עבור חדרים חדשים, לעקוב אחר רשימת השחקנים ומזהי ה-Socket שלהם, לאפשר כניסה ועזיבה מסודרת, ולשמור על מיפוי קבוע בזיכרון ה-RAM כדי שהשרת ידע לשייך כל הודעת רשת לחדר המשחק המתאים.

תכונות המחלקה (Fields):

- `self.rooms`: מילון (Dictionary) שבו המפתח (Key) הוא קוד החדר הייחודי (מחרוזת), והערך (Value) הוא אובייקט מודל מסוג Room המכיל את נתוני החדר.

- `self.sid_to_room`: מילון עזר המשמש למיפוי מהיר, שבו המפתח הוא מזהי ה-Socket הייחודי של המכשיר (sid), והערך הוא קוד החדר שבו הוא נמצא. משמש לאיתור מהיר של החדר בעת ניתוק פתאומי.

פעולות המחלקה (Methods):

- `private str _generate_code()`: פונקציה פנימית המריצה לולאה של עד 100 ניסיונות להגרלת קוד חדר אקראי ומאובטח בן 6 ספרות (בין 100,000 ל-999,999) בעזרת ספריית secrets. הפונקציה מוודאת שהקוד אינו בשימוש קיים, ואם היא נכשלת - היא זורקת שגיאת מערכת.

- `public Room create_room(str uid, str sid, str display_name)`: מוודאת שהמשתמש אינו נמצא כבר בחדר אחר, מגרילה קוד חדר חדש, מייצרת אובייקט Room, מוסיפה את השחקן הנוכחי כמנהל ויוצר החדר, ומעדכנת את מילוני הזיכרון.

- `public Room join_room(str code, str uid, str sid, str display_name)`: הפעולה הלוגית המרכזית. מנהלת את תהליך ההצטרפות של שחקן שני לחדר קיים על בסיס קוד, תוך אכיפת מגבלות האבטחה וחוקי הלובי.

- `public Optional[Room] leave_by_sid(str sid)`: מופעלת כאשר שחקן עוזב את החדר או מתנתק. היא שולפת את החדר לפי ה-sid, מסירה את המשתמש ומנקה את המיפויים. אם החדר נשאר ריק לחלוטין - היא מוחקת את החדר מהזיכרון; אם נשאר שחקן אחר, והעוזב היה המנהל - היא מעבירה את סמכויות הניהול ל-sid של שחקן שנשאר. (owner\_uid)

- `public Optional[Room] get_room_for_sid(str sid)`: פונקציית שליפה מהירה (Getter) המחזירה את אובייקט החדר שבו השחקן נמצא על בסיס מזהי ה-sid שלו.

קוד והסבר של פעולה לוגית מרכזית (`join_room`):

```
def join_room(
 self,
 code: str,
 uid: str,
 sid: str,
 display_name: str,
) -> Room:
 if sid in self.sid_to_room:
 raise ValueError("You are already inside a room")

 room = self.rooms.get(code)
 if room is None:
 raise ValueError("Room not found")

 if room.status != "waiting":
 raise ValueError("The game has already started")
```

```

if len(room.players) >= 2:
 raise ValueError("Room is full")

if uid in room.players:
 raise ValueError("This user is already in the room")

room.players[uid] = Player(
 uid=uid,
 sid=sid,
 display_name=display_name,
)
self.sid_to_room[sid] = code
return room

```

הסבר:

הפונקציה מקבלת את פרטי החיבור של השחקן המבקש להצטרף ומבצעת סדרה קפדנית של 5 בדיקות תקינות (ווילדציה בארכיטקטורת שרת סמכותי):

1. מוודאת שהשחקן לא משויך כבר לחדר אחר במערכת.
2. בודקת האם הקוד שהוקלד קיים פיזית במילון החדרים.
3. מוודאת שסטוס החדר הוא בהמתנה (waiting) והמשחק עוד לא התחיל.
4. מוודאת שבחדר יש פחות מ-2 שחקנים (כדי לא לחרוג ממגבלת הדו-קרב).
5. מוודאת שה-uid שלו אינו רשום כבר בפנים.

במידה ואחת הבדיקות נכשלת – נזרק ValueError שמנוהל ב-app.py ומוחזר לאנדרואיד כהודעת שגיאה חלקה. אם הכל תקין, היא מייצרת מופע Player חדש, מאכלסת אותו בתוך רשימת שחקני החדר, מעדכנת את מילון ה-sid ומחזירה את החדר המעודכן.

### מחלקה 3 - game\_manager.py:

תפקיד המחלקה:

מחלקת הליבה האחראית על ניהול מחזור החיים המלא של המשחקים הפעילים בזמן אמת. תפקידה לאתחל את מצב המשחק, לערבב ולחלק קלפים דינמיים מתוך רשימת המפתחות, לנהל את מפת הניקוד, ולאכוף בצורה אבטחתית את בדיקת נכונות הלחיצות שמגיעות מהלקוח כדי למנוע מצבי מרוץ (Race Conditions). היא משמשת כפוסק הבלעדי של חוקי דובל ברשת ומנהלת את קידום הסיבובים או סיום המשחק.

תכונות המחלקה (Fields):

- self.card\_manager: מופע של המחלקה CardManager המשמש לשליפה ואימות מתמטי של סמלים משותפים.
- self.games: מילון (Dictionary) המנוהל בזיכרון ה-RAM שבו המפתח (Key) הוא קוד החדר, והערך (Value) הוא אובייקט מודל מסוג GameState המחזיק את נתוני הריצה הפעילים.

פעולות המחלקה (Methods):

- (public GameState start\_game(Room room, int target\_score = 10): מאמתת שלא רץ משחק בחדר, שיש בדיוק 2 שחקנים, ושערך יעד הניקוד תקין (בין 1 ל-50). היא שולפת את מפתחות הקלפים, מערבבת אותם רנדומלית, מקצה קלפים ראשונים לשחקנים ולקלף המרכזי, מאתחלת את הניקוד ל-0 ומשנה את סטטוס החדר ל-playing.
- (public GameState get\_game(str room\_code): פונקציית שליפה מהירה (Getter) המחזירה את אובייקט ה-GameState מתוך המילון. אם הוא לא קיים – היא זורקת ValueError המנוע המשך ריצה לא חוקית.
- (public dict get\_private\_state(Room room, str uid): מוודאת שהשחקן משויך לחדר, שולפת את המילון הציבורי המשותף (הניקוד וקלף השולחן), ומזריקה לתוכו באופן ספציפי ומאובטח את ה-playerCardId הייחודי שבידו של אותו משתמש.
- (public Tuple[bool, dict] submit\_symbol(Room room, str uid, str symbol, str card\_id): הפעולה הלוגית המרכזית. מנהלת את קבלת הלחיצה של השחקן, אימותה מול קלפי המשחק העדכניים, ועדכון המצב בהתאם.
- (private void \_advance\_round(GameState game, str round\_winner\_uid): פונקציה פנימית המקדמת את המשחק לסיבוב הבא. היא הופכת את קלף השחקן שניצח לקלף המרכזי החדש, ומפעילה אלגוריתם שליפה לחלוקת קלפים חדשים ושמירה על חדר פתוח.
- (private int \_find\_next\_unused\_index(GameState game, set[int] used\_indexes): פונקציית עזר אלגוריתמית המחשבת בצורה מעגלית ומחזירה את אינדקס הקלף הפנוי הבא בחפיסה שטרם נעשה בו שימוש בסיבוב הנוכחי, כדי למנוע כפילויות.
- (public Optional[GameState] cancel\_game(str room\_code): מוחקת בצורה יזומה ומסודרת את אובייקט המשחק מהמילון המקומי בעת סגירת חדר או עזיבת שחקן כדי למנוע דליפות זיכרון בשרת.

קוד והסבר של פעולה לוגית מרכזית (submit\_symbol):

```
def submit_symbol(
 self,
 room: Room,
 uid: str,
 symbol: str,
 card_id: str,
) -> Tuple[bool, dict]:
 game = self.get_game(room.code)
 if game.winner_uid:
 raise ValueError("The game has already ended")
 if not game.round_open:
```

```

 raise ValueError("This round is already closed")
 if uid not in room.players:
 raise ValueError("Player is not in this room")

 expected_player_card = game.player_card_id(uid)
 if str(card_id) != str(expected_player_card):
 raise ValueError("The submitted card does not match the current player card")

 common_symbol = self.card_manager.get_common_symbol(
 game.center_card_id,
 expected_player_card,
)

 normalized_submitted = symbol.strip().casefold()
 normalized_expected = common_symbol.casefold()

 if normalized_submitted != normalized_expected:
 return False, {
 "correct": False,
 "submittedSymbol": symbol,
 "expectedSymbol": common_symbol,
 }

 game.round_open = False
 game.scores[uid] += 1

 if game.scores[uid] >= game.target_score:
 game.winner_uid = uid
 room.status = "finished"
 return True, {
 "correct": True,
 "winnerUid": uid,
 "correctSymbol": common_symbol,
 "gameFinished": True,
 }

 self._advance_round(game, uid)
 return True, {
 "correct": True,
 "roundWinnerUid": uid,
 "correctSymbol": common_symbol,
 "gameFinished": False,
 }

```

הסבר:

הפונקציה מנהלת את ליבת האבטחה וחוקי דובל של המשחק:

1. בדיקות מקדימות (Gatekeepers): היא מוודאת שהמשחק לא הסתיים, שהסיבוב הנוכחי פתוח לקבלת לחיצות (מונע משחקן שני ללחוץ אם הראשון כבר צדק), ושהמשתמש רשום בחדר.
2. אימות קלף הלקוח: מוודאת שמשתנה ה-card\_id שהאנדרואיד דיווח עליו תואם בדיוק לקלף שהשרת יודע שנמצא אצלו ביד באמת (מניעת רמאויות וזיופי לקוח).
3. בדיקה מתמטית: פונה ל-card\_manager ומחלצת את הסמל המשותף האמיתי בין קלף השולחן לקלף השחקן. היא מנרמלת את שני הטקסטים (הסרת רווחים והתעלמות מאותיות קטנות/גדולות בעזרת casefold()) ומבצעת השוואה.
4. ניהול תוצאה שגויה: אם השחקן טעה – היא מחזירה מיד False יחד עם פירוט הסמל המצופה, מבלי לשנות את מצב המשחק.
5. ניהול הצלחה וסיום: אם השחקן צדק, השרת נועל זמנית את הסיבוב (game.round\_open = False) ומעלה את הניקוד שלו ב-1. אם הוא הגיע ליעד הניקוד – המשחק מוכרז כסגור ומוחזר חיווי של gameFinished: True. אם המשחק נמשך – היא מפעילה את פונקציית החלפת הקלפים ומקדמת סיבוב.

**מחלקה 4 - card\_manager.py:**

תפקיד המחלקה:

מחלקה זו אחראית על ניהול, טעינת ואבטחת נתוני קלפי המשחק בשרת. תפקידה המרכזי הוא לטעון את קובץ ה-JSON שמכיל את חפיסת הקלפים, לבצע וולידציה (בדיקת תקינות) קפדנית המוודאת שחפיסת הקלפים עומדת בדיוק בחוקים הגיאומטריים של משחק דובל (כל קלף מכיל סמלים ייחודיים ובין כל שני קלפים יש בדיוק סמל אחד משותף), ולספק מנגנון שליפה מהיר למציאת הסמל המשותף בזמן אמת.

תכונות המחלקה (Fields):

- VALID\_REGIONS (תכונה סטטית / קבועה): קבוצה (Set) המגדירה את 9 שמות האזורים הוויזואליים התקינים על גבי הקלף (כגון "top-left", "center-center" וכו').
- self.cards: מילון (Dictionary) השומר בזיכרון ה-RAM את כל מבנה חפיסת הקלפים, שבו המפתח הוא מזהה הקלף והערך הוא מיפוי האזורים והסמלים שלו.

פעולות המחלקה (Methods):

- (public \_\_init\_\_(Path cards\_path): בנאי המחלקה. פותח את קובץ ה-JSON בקידוד utf-8, טוען את המידע לתוך self.cards ומפעיל מיידית את פונקציית האימות המקיפה self.validate().
- (private Set[str] \_unique\_symbols(dict card): פונקציית עזר המחלצת ומחזירה אוסף של סמלים ייחודיים מתוך קלף, תוך התעלמות מערכים ריקים.
- (private None \_validate): הפעולה הלוגית המרכזית הראשונה. מריצה סדרת בדיקות מתמטיות קפדניות על החפיסה בזמן עליית השרת כדי למנוע שגיאות במשחק.
- (public dict get\_card(str card\_id): שולפת ומחזירה את מילון האזורים והסמלים של קלף ספציפי לפי ה-ID שלו. אם הקלף לא קיים – היא זורקת שגיאת מפתח (KeyError).
- (public List[str] get\_symbols(str card\_id): שולפת את כל הסמלים הייחודיים של קלף מסוים ומחזירה אותם כרשימה ממוינת.
- (public str get\_common\_symbol(str first\_card\_id, str second\_card\_id): הפעולה הלוגית המרכזית השנייה. מחשבת ומחלצת ברשת את הסמל המשותף היחיד הקיים בין שני קלפים שונים.

קוד והסבר של פעולה לוגית מרכזית (validate\_):

def \_validate(self) -&gt; None:

if len(self.cards) != 55:

raise ValueError(f"Expected 55 cards, found {len(self.cards)}")

for card\_id, regions in self.cards.items():

unknown\_regions = set(regions) - self.VALID\_REGIONS

if unknown\_regions:

raise ValueError(

f"Card {card\_id} contains invalid regions: {sorted(unknown\_regions)}")

)

symbols = self.\_unique\_symbols(regions)

if len(symbols) != 8:

raise ValueError(

f"Card {card\_id} must contain 8 unique symbols, found {len(symbols)}")

)

card\_ids = list(self.cards.keys())



```

for index, first_id in enumerate(card_ids):
 first_symbols = self._unique_symbols(self.cards[first_id])
 for second_id in card_ids[index + 1:]:
 second_symbols = self._unique_symbols(self.cards[second_id])
 common = first_symbols & second_symbols
 if len(common) != 1:
 raise ValueError(
 f"Cards {first_id} and {second_id} share "
 f"{len(common)} symbols: {sorted(common)}"
)

```

הסבר:

פונקציה זו מהווה את מנגנון בקרת האיכות המדעי של חפיסת המשחק ומבצעת שלושה שלבי הגנה:

1. בדיקת גודל ותקינות אזורים: מוודאת שיש בדיוק 55 קלפים בחפיסה, ושכל קלף מכיל אך ורק אזורים מתוך רשימת ה-VALID\_REGIONS המותרת.
2. בדיקת כמות סמלים לקלף: מוודאת שכל קלף בודד מכיל בדיוק 8 סמלים ייחודיים ושונים זה מזה.
3. אכיפת חוק הזהות המוחלט (הלולאה המקוננת): הפונקציה מבצעת השוואה מוצלבת בין כל זוג קלפים אפשרי בחפיסה. היא משתמשת באופרטור החיתוך של קבוצות (&) כדי למצוא את הסמלים המשותפים ביניהם ומאמתת שהתוצאה היא בדיוק סמל אחד משותף בלבד. במידה ויש זוג קלפים שמשותף 0 סמלים או 2 ומעלה – המערכת זורקת ValueError ומסרבת להעלות את השרת, מה שמבטיח משחק הוגן ולוגיקה חסינת באגים.

## מחלקה 5 - auth\_manager.py:

תפקיד המחלקה:

מחלקה זו אחראית על ניהול מנגנוני אבטחת המידע, הרישום ואימות המשתמשים בשרת. תפקידה המרכזי הוא לקבל את נתוני החיבור הראשוניים ממכשירי האנדרואיד (Payload), לאמת אסינכרונית את תקינות ה-ID Token הסודי של השחקן מול ה-SDK הרשמי של Firebase Authentication, ולמנוע חיבורים לא חוקיים או מזויפים. בנוסף, היא מתחברת לבסיס הנתונים Cloud Firestore כדי לשלוח את נתוני הפרופיל ושם התצוגה המעודכן של המשתמש לצורך סנכרונו בתוך חדר המשחק בזמן אמת.

תכונות המחלקה (Fields):

- `self.dev_skip_auth`: דגל בוליאני הקורא משתנה סביבה מתוך קובץ ה-`env`. במידה והוא מוגדר כ-`True`, השרת מדלג על אימות ה-Token הרשתי ומאפשר חיבורים מקומיים מהירים לצורך פיתוח ובדיקת באגים באמולטורים.
- `self.database`: מופע של הלקוח הרשמי של `Firestore (firestore.client)`, המשמש כצינור הגישה הישיר והמאובטח לטבלאות ואוספי הנתונים בענן של גוגל.

פעולות המחלקה (Methods):

- `(public __init__(Path project_root`: בנאי המחלקה. בודק האם מצב פיתוח פעיל. במידה ולא, הוא מאתר את קובץ האישורים הסודי `serviceAccountKey.json`, מוודא את קיומו בדיסק, מאתחל ומחבר את השרת בפעם הראשונה אל ה-API המרכזי של `Firebase`, ומנפק את אובייקט הגישה `self.database`.
- `(public Dict[str, str] verify_connection(Any auth_payload`: הפעולה הלוגית המרכזית. מקבלת את חבילת נתוני האימות שהלקוח שיגר, מאמתת את הטוקן, שולפת את נתוני המשתמש מ-`Firestore` ומחזירה מילון מעובד עם פרטי השחקן הרשמיים.

קוד והסבר של פעולה לוגית מרכזית (`verify_connection`):

```
def verify_connection(
 self,
 auth_payload: Any
) -> Dict[str, str]:
 if not isinstance(auth_payload, dict):
 raise ValueError("Missing Socket.IO authentication payload")

 if self.dev_skip_auth:
 uid = str(auth_payload.get("uid", "")).strip()
 if not uid:
 raise ValueError("DEV mode requires auth.uid")
 return {
 "uid": uid,
 "name": str(auth_payload.get("name", uid)),
 "email": str(auth_payload.get("email", "")),
 }

 token = auth_payload.get("token")
 if not isinstance(token, str) or not token.strip():
 raise ValueError("Missing Firebase ID token")

 decoded = auth.verify_id_token(token)
```

```

uid = decoded["uid"]
email = decoded.get("email", "")

display_name = None
try:
 document = (
 self.database
 .collection("users")
 .document(uid)
 .get()
)
 if document.exists:
 user_data = document.to_dict() or {}
 display_name = (
 user_data.get("displayName")
 or user_data.get("name")
 or user_data.get("username")
)
 except Exception as error:
 print(f"Failed to load Firestore user {uid}: {error}")

if display_name:
 display_name = str(display_name).strip()
if not display_name:
 display_name = decoded.get("name") or email or uid

return {
 "uid": uid,
 "name": display_name,
 "email": email,
}

```

הסבר:

- הפונקציה משמשת כמסנן האבטחה הראשי של ה-WebSockets ומבצעת את השלבים הבאים:
1. וולידציה ראשונית: מוודאת שנתוני ה-Payload הגיעו כמבנה נתונים מסוג מילון (Dictionary).
  2. נתיב עוקף פיתוח (DEV\_SKIP\_AUTH): במידה ומצב פיתוח מופעל בקובץ ההגדרות, היא שולפת מזהה משתמש (uid) גנרי מהלקוח, מדלגת על בדיקות האינטרנט ומאשרת מיידית את הכניסה כדי לחסוך זמן פיתוח.
  3. אימות טוקן קפדני (Production): שולפת את ה-token המוצפן שנשלח מהאנדרואיד ומעבירה אותו לפקודת verify\_id\_token. ספריית ה-Admin של גוגל מפענחת אותו ומאמתת רשתית שהמשתמש מחובר כחוק ומחלצת את ה-uid הייחודי שלו.
  4. שליפת שם תצוגה מ-Firestore: ניגשת אסינכרונית לאוסף users ומחפשת את המסמך התואם ל-uid. אם המסמך קיים, היא מנסה לחלץ שם תצוגה לפי סדר עדיפויות מוגדר: שדה displayName, שדה name, או שדה username.

5. מנגנון גיבוי (Fallback): אם שליפת ה-Firestore נכשלת או שהשדות ריקים, היא לוקחת כגיבוי את השם הרשום בתוך הטוקן המקורי, את כתובת האימייל, או פשוט מחזירה את מחרוזת ה-wid עצמה, ומאשרת את כניסת השחקן לשרת.

**מחלקה 6 - stats\_manager.py:**

תפקיד המחלקה:

מחלקה זו אחראית על עיבוד, אריזה ושמירה קבועה של נתוני המשחקים והתוצאות בסיום כל התמודדות. תפקידה המרכזי הוא להתממשק ישירות מול Cloud Firestore בעזרת ה-Firebase Admin SDK ולבצע עסקאות אטומיות מורכבות (Transactions / Batches) בענן. המחלקה מייצרת מסמכי סיכום קבועים באוסף המשחקים, ובמקביל מעדכנת ומקדמת בצורה מתמטית (Increment) את מוני הניצחונות, ההפסדים וסך כל הנקודות של כל שחקן במסמך הפרופיל האישי שלו, תוך הגנה על שלמות בסיס הנתונים.

תכונות המחלקה (Fields):

- `self.database`: מופע של הלקוח הרשמי של Firestore המאפשר גישה וכתובה אל אוספי בסיס הנתונים בענן של גוגל.

פעולות המחלקה (Methods):

- `__init__()`: בנאי המחלקה. מאתחל את המשתנה הרישמי ומחבר את המופע של ה-Database מול השרת.

- `public str save_match(str room_code, dict players, dict scores, str winner_uid, int target_score, int round_number)`: הפעולה הלוגית המרכזית. מקבלת את כלל נתוני הסיכום של המשחק שהסתיים, מכינה אובייקט מסמך מרוכז, מבצעת כתיבה סימולטנית בעזרת Write Batch ומעדכנת את רשומות המשתתפים.

קוד והסבר של פעולה לוגית מרכזית (save\_match):

```
def save_match(
 self,
 room_code: str,
 players: Dict[str, object],
 scores: Dict[str, int],
 winner_uid: str,
 target_score: int,
 round_number: int
) -> str:
 match_id = str(uuid4())
 player_uids = list(players.keys())
 player_names = {
 uid: players[uid].display_name
 for uid in player_uids
 }

 match_data = {
 "matchId": match_id,
 "roomCode": room_code,
 "players": player_uids,
 "playerNames": player_names,
 "scores": scores,
 "winnerUid": winner_uid,
 "targetScore": target_score,
 "roundsPlayed": round_number,
```

```

 "createdAt": datetime.now(timezone.utc)
}

batch = self.database.batch()
match_reference = (
 self.database
 .collection("matches")
 .document(match_id)
)
batch.set(match_reference, match_data)

for uid in player_uids:
 user_reference = (
 self.database
 .collection("users")
 .document(uid)
)
 score = scores.get(uid, 0)
 updates = {
 "gamesPlayed": firestore.Increment(1),
 "totalScore": firestore.Increment(score)
 }
 if uid == winner_uid:
 updates["wins"] = firestore.Increment(1)
 else:
 updates["losses"] = firestore.Increment(1)

 batch.set(
 user_reference,
 updates,
 merge=True
)

batch.commit()
return match_id

```

הסבר:

כאשר משחק דובל מגיע לסיומו הרשמי בשרת, פונקציה זו נקראת כדי להנציח את התוצאות ומבצעת את השלבים הבאים בארכיטקטורת רשת חסינת שגיאות:

1. הפקת מזהה ואריזה: מייצרת מזהה ייחודי גלובלי למשחק (`match_id`) בעזרת ספריית `uuid4`. היא מחלצת את רשימת מזהי השחקנים וממפה את שמותיהם כדי לארוז מסמך `match_data` מקיף הכולל חותמת זמן עולמית קבועה (`timezone.utc`).

2. שימוש ב-Write Batch (עסקה אטומית): השרת פותח רכיב batch. זהו כלי המאפשר לרכז מספר פעולות כתיבה ועדכון שונות, ולשלוח אותן לענן כאירוע אחד בלבד, מה שמבטיח שאם אחת הפעולות תיכשל (למשל בעיית רשת), אף פעולה לא תתבצע והדאטה לא יושחת (מניעת חצי-עדכון).
3. הכנת מסמך המשחק: השרת מגדיר מסמך חדש באוסף matches ומצרף את נתוני המשחק ל-Batch הקיים.
4. עדכון פרופילי שחקנים דינמי: לולאה עוברת על שני המשתתפים ומייצרת פקודות עדכון עבור המסמך שלהם באוסף users. במקום לקרוא את המסמך הישן, לערוך ולכתוב מחדש, השרת משתמש בפקודת firestore.Increment המורה ל-Firestore לקדם את השדות המתמטיים ישירות בענן (מעלה ב-1 את כמות המשחקים, ומעלה את סך הניקוד לפי מה שהושג). בעזרת תנאי, היא מוסיפה 1 למונה ה-wins של המנצח או 1 למוני ה-losses של המפסיד, ומצרפת את הפקודות ל-Batch עם ההגדרה merge=True המונעת דריסה של שדות קודמים (כמו אימייל או שם).
5. ביצוע מרוכז וסגירה: פקודת batch.commit() מוציאה לפועל את כל העדכונים יחד בענן בבת אחת, והפונקציה מחזירה את ה-ID של המשחק שנסגר.

## מחלקה 7 - Player:

תפקיד המחלקה:

מחלקה המוגדרת כ-@dataclass ומייצגת ישות שחקן רשת בודד בזיכרון השרת.

תכונות המחלקה (Fields):

- uid: מחרוזת טקסט המייצגת את מזהה ה-User הייחודי והקבוע של השחקן מ-Firebase Auth.
- sid: מחרוזת המאחסנת את מזהה ה-Session ID הזמני של ה-Socket.IO המקושר למכשיר.
- display\_name: מחרוזת המכילה את שם התצוגה הוירטואלי של השחקן לצורך הצגתו ליריבו.



## מחלקה 8 - room.py:

תפקיד המחלקה:

קובץ זה מחזיק את מחלקות מודל הנתונים (Data Models) הפשוטות המשמשות כמיכלי מידע מובנים בזיכרון השרת (RAM) בזמן הריצה. המחלקה Player שומרת את זהות ופרטי החיבור של שחקן מחובר. המחלקה Room מייצגת חלל לובי וירטואלי, מנהלת את רשימת השחקנים שנמצאים בפנים, שומרת את סוג חפיסת המשחק הנבחרת ואת הסטטוס הלוגי שלו. היא מספקת פונקציית סירוז (Serialization) הממירה את נתוני המצב למילון תקני כדי שיוכל להישלח כ-JSON ברשת אל האפליקציה באנדרואיד.

תכונות המחלקה (Fields):

- code: מחרוזת טקסט השומרת את קוד החדר הדינמי המגולל (בן 6 ספרות) המשמש להצטרפות.
- owner\_uid: מחרוזת המאחסנת את ה-UID של מנהל ויוצר החדר בעל סמכויות תחילת המשחק.
- players: מילון ([Dict[str, Player]) הממפה בין ה-uid של המשתמשים לאובייקט ה-Player שלהם.
- selected\_box: מחרוזת השומרת את סוג חפיסת הקלפים הנבחרת (ברירת מחדל נקבעת ל-"original").
- status: מחרוזת המגדירה את המצב הלוגי של הלובי (ברירת מחדל נקבעת ל-"waiting").

פעולות המחלקה (Methods):

- public to\_dict(): הפעולה הלוגית המרכזית. מפרקת וממירה את כל מאפייני החדר ואת רשימת השחקנים המקוננים בתוכו למילון פייתון פשוט על מנת לאפשר שידור JSON תקני ותואם ללקוח.

קוד והסבר של פעולה לוגית מרכזית (to\_dict):

```
def to_dict(self) -> dict:
```

```
 return {
 "code": self.code,
 "ownerUid": self.owner_uid,
 "selectedBox": self.selected_box,
 "status": self.status,
 "players": [
 {
 "uid": player.uid,
 "displayName": player.display_name,
 }
 for player in self.players.values()
],
 }
```

הסבר:

מאחר ופרוטוקול התקשורת Socket.IO אינו תומך בהעברת אובייקטים מורכבים של מחלקות מותאמות אישית ברשת, פונקציה זו מבצעת פירוק של המידע. היא משתמשת בלולאת רשימה מובנית (List Comprehension) העוברת על כל ערכי אובייקטי השחקנים שנמצאים במילון (self.players.values()), מחלצת מתוכם אך ורק את השדות הנחוצים לממשק המשתמש (uid ו-displayName) ומארגנת הכל בתוך מערך של אובייקטים. פלט זה מומר אוטומטית למחרוזת JSON נקייה, המאפשרת לאפליקציית האנדרואיד לקרוא את רשימת השחקנים ולרענן את מסך ה-RoomActivity בצורה חלקה.

## מחלקה 9 - game.py:

תפקיד המחלקה:

מחלקת מודל נתונים (Data Model Class) המבוססת על הארכיטקטורה של פייתון לניהול רשומות מידע מובנות (@dataclass). תפקידה הבלעדי הוא לייצג, לאגור ולשקף בזיכרון ה-RAM של השרת את המצב הלוגי והמספרי המדויק (State) של משחק פעיל בחדר ספציפי. היא עוקבת אחר סדר הקלפים שהוגרלו מחפיסת ה-JSOn, מנהלת את מיקומי האינדקסים של קלף השולחן וקלפי השחקנים, מתעדת את הסטטיסטיקה של מספר הסיבוב הנוכחי, מפת הניקוד הפעילה, ומצבי נעילת הסיבוב או זיהוי המנצח הרשמי.

תכונות המחלקה (Fields):

- room\_code: מחרוזת טקסט המייצגת את קוד החדר הייחודי שבו מתנהל המשחק.
- card\_order: רשימה ([List[str)]) המכילה את מזהי כלל הקלפים שהוקצו למשחק הנוכחי בסדר מוגדר (סדר חפיסה מעורבב).
- center\_index: משתנה מספרי (אינדקס) המצביע על מיקום קלף השולחן המרכזי הנוכחי מתוך רשימת ה-card\_order (ערך ראשוני קבוע הוא 0).
- player\_card\_indexes: מילון ([Dict[str, int)]) הממפה בין ה-UID הייחודי של כל שחקן לבין אינדקס המיקום של הקלף האישי שנמצא אצלו ביד כרגע.
- scores: מילון ([Dict[str, int)]) המנהל את טבלת הניקוד הפעילה וממפה בין ה-UID של השחקן לבין כמות הנקודות הנוכחית שצבר במשחק.
- round\_number: משתנה מספרי (שלם) העוקב ומציג באיזה סיבוב משחק נמצאים כרגע (ערך ראשוני קבוע הוא 1).
- round\_open: דגל בוליאני המשמש כנעילת אבטחה ברמת סיבוב. נקבע כ-True בכל סיבוב חדש, ומשתנה מיד ל-False ברגע ששחקן כלשהו לחץ על סמל נכון, כדי לחסום לחיצות סימולטניות נוספות.
- winner\_uid: משתנה השומר את מזהה ה-UID של השחקן שהגיע ראשון ליעד הניקוד והוכרז כמנצח המשחק (כל עוד המשחק רץ ערכו קבוע כ-None).
- target\_score: משתנה מספרי המגדיר את רף הניקוד הנדרש כדי לנצח ולסיים את ההתמודדות (נקבע כברירת מחדל ל-10 נקודות).

פעולות המחלקה (Methods):

- public str center\_card\_id (מאפיין / Property): פונקציית שליפה דינמית המחזירה את מזהה הקלף (ה-String האמיתי של הקלף, כגון "5") שנמצא כרגע במרכז השולחן על בסיס משתנה ה-center\_index הנוכחי.
- public str player\_card\_id(str uid): פונקציית גישה דינמית המקבלת מזהה שחקן ומחזירה את מזהה הקלף (ה-String האמיתי של הקלף) שנמצא אצלו ביד כרגע על בסיס מילון האינדקסים.
- public dict to\_public\_dict(dict players): הפעולה הלוגית המרכזית. מעבדת ומפרקת את מצב המשחק המופשט בשרת, וממירה אותו למילון תקני, ציבורי ומשותף המיועד להפצה מרוכזת (Broadcast) כ-JSON אל שני מכשירי הלקוח ברשת.

קוד והסבר של פעולה לוגית מרכזית (to\_public\_dict):

```
def to_public_dict(self, players: Dict[str, object]) -> dict:
 return {
 "roomCode": self.room_code,
 "roundNumber": self.round_number,
 "centerCardId": self.center_card_id,
 "scores": self.scores,
 "targetScore": self.target_score,
 "status": "finished" if self.winner_uid else "playing",
 "winnerUid": self.winner_uid,
```

```

"players": [
 {
 "uid": uid,
 "displayName": players[uid].display_name,
 "score": self.scores.get(uid, 0),
 }
 for uid in players
],
}

```

הסבר:

פונקציה זו משמשת לייצור ה-Payload הציבורי של המשחק במהלך ריצתו. היא אינה חושפת את קלפי היריב (כדי לשמור על אבטחת המשחק ומניעת רמאויות), אלא אורזת רק את הנתונים המשותפים ששני המכשירים צריכים לדעת: מהו מספר הסיבוב, מהו קלף השולחן המרכזי העדכני, ולוח הניקוד הכללי. באמצעות תנאי מובנה, היא קובעת דינמית את שדה ה-status ל-"finished" במידה ומשתנה ה-winner\_uid אוכלס, או ל-"playing" במידה והמשחק עדיין בריצה. בנוסף, היא מריצה לולאת מבנה מובנית (List Comprehension) המצליבה מידע בין מילון שחקני החדר לבין מפת הניקוד של המשחק, ומייצרת מערך מסודר המכיל את ה-uid, ה-displayName וה-score העדכני של כל משתתף, לצורך רענון מיידי של ה-scoreTextView במסך ה-GameActivity באנדרואיד.

## שמירת נתונים – צד לקוח (Android)

### קבצים

האפליקציה אינה מייצרת או כותבת קבצי טקסט דינמיים על זיכרון המכשיר (כגון קבצי .txt), אך היא משתמשת בקובץ נתונים מובנה המוגדר כנכס סטטי בתוך משאבי המערכת.

### :Cards.json

- סוג הקובץ: קובץ נתונים מובנה בפורמט JSON.
- מיקום הזיכרון: נשמר בזיכרון הפנימי של האפליקציה תחת תיקיית ה-Assets.
- תפקיד ומטרה: הקובץ מכיל את המיפוי הגיאומטרי והמבני המלא של 55 קלפי המשחק. הוא מגדיר בדיוק אילו 8 סמלים משויכים לכל קלף, ולאיזה אזור לחיצה גרפי (regionName) הם שייכים.
- מחלקות ופעולות המשתמשות בקובץ:
  - o `CardDataManager(Context context)` (בנאי המחלקה) – פותח את זרם הקלט לקובץ ב-Assets וטוען אותו לזיכרון ה-RAM.
  - o `CardDataManager.loadCards(Context context)` – מבצעת את פענוח ה-JSON (Parsing) ואת כלולת המפות המקוננות.

### מבנה קובץ ה-JSON (Schema):

הקובץ בנוי כאובייקט JSON מרכזי, שבו כל מפתח מייצג את מזהה הקלף, והערך הוא אובייקט פנימי הממפה בין האזור הפיזי לשם הסמל:

```
{
 "1": {
 "top-left": "Apple",
 "top-center": "Bird",
 "center-center": "Clock"
 },
 "2": {
 "top-left": "Apple",
 "top-right": "Dolphin",
 "center-center": "Exclamation"
 }
}
```

### Shared Preferences

האפליקציה מיישמת שימוש ברכיב ה-SharedPreferences המובנה באנדרואיד לצורך שמירת מצב פגישת (Session Persistence). המטרה היא להעניק חווית משתמש רציפה (Auto-Login) המונעת מהשחקן את הצורך להקליד אימייל וסיסמה בכל פעם שהוא פותח מחדש את האפליקציה.

- שם ה-Shared Preference: `com.me.meydan.dobble_preferences`
- תפקיד כללי: שמירת נתוני הזיהוי והטוקן המוצפן של השחקן המחובר על גבי קובץ XML פנימי ומוגן של המערכת.

- טבלת השדות והתפקידים בתוך הרכיב:

| <u>שם המפתח (Key)</u>      | <u>טיפוס הנתון (Type)</u> | <u>תפקיד השדה ושימוש באפליקציה</u>                                                         |
|----------------------------|---------------------------|--------------------------------------------------------------------------------------------|
| <b><u>remember_me</u></b>  | Boolean                   | דגל בוליאני המציין האם השחקן סימן את תיבת "זכור אותי" במסך ה-Login.                        |
| <b><u>cached_email</u></b> | String                    | שומר את כתובת הדואר האלקטרוני האחרונה שהתחברה בהצלחה, לצורך מילוי אוטומטי.                 |
| <b><u>auth_token</u></b>   | String                    | מחרוזת אבטחה מוצפנת (Firebase ID Token) המאפשרת דילוג על מסך הכניסה ישירות ל-MainActivity. |

## **שמירת נתונים – צד שרת**

בצד השרת, הנתונים מחולקים לשני סוגים: נתונים סטטיים בקבצים מקומיים, ונתונים דינמיים המנוהלים ישירות מול בסיס הנתונים Cloud Firestore המרוחק בענן של גוגל באמצעות ה-Firebase Admin SDK המאובטח.

1. קבצים בצד שרת:

- Cards.json: זהה לחלוטין במבנהו לקובץ הקיים באנדרואיד. השרת טוען אותו באמצעות card\_manager.py בזמן עליית המערכת ומבצע עליו סריקה מוצלבת. מטרתו היא לשמש כסמכות הבלעדית (Single Source of Truth) לבדיקת נכונות הלחיצות של השחקנים.
- env: קובץ הגדרות מקומי וסודי בשרת. מכיל משתני סביבה קריטיים כגון DEV\_SKIP\_AUTH (הדגל המאפשר לדלג על אימות רשת בזמן פיתוח) ופורטים של שרת ה-Websockets.

2. סכמת ה-Database בענן (Cloud Firestore Model):

בסיס הנתונים מבוסס על ארכיטקטורת NoSQL מבוססת מסמכים (Documents) ולא על טבלאות יחסית קשיחות (כמו Access או SQLite). מצורף תיעוד שני האוספים (Collections) והקשרים ביניהם במערכת:

### 📋 סכמת בסיס נתונים: אוסף המשחקים (matches)

| שם השדה במערכת (Field)      | טיפוס הנתון (Type) | תפקיד ושימוש השדה במערכת                                                          |
|-----------------------------|--------------------|-----------------------------------------------------------------------------------|
| <code>createdAt</code> 🕒    | Timestamp          | חותמת הזמן, התאריך והשעה המדויקים שבהם המשחק הסתיים (משמש למיזן ההיסטוריה).       |
| <code>matchId</code> 🆔      | String             | מזהה ייחודי קבוע (UUID) המופק עבור המשחק כדי למנוע התנגשויות בעק.                 |
| <code>playerNames</code> 👤  | Map (Key-Value)    | מילון הממפה בין ה-UID של כל שחקן לבין שם התצוגה ( <code>displayName</code> ) שלו. |
| <code>players</code> 👤      | Array (Strings)    | מערך המכיל את מזהי ה-UID של שני המשתתפים (משמש לשאילתת הסינון באנדרואיד).         |
| <code>roomCode</code> 🏠     | String             | קוד חדר המשחק בן 6 הספרות שבו נערכה ההתמודדות בשרת הפייתון.                       |
| <code>roundsPlayed</code> 🔄 | Integer            | מספר שלם המציין כמה סיבובים של החלפת קלפים שוחקו בפועל במהלך המשחק.               |
| <code>scores</code> 📊       | Map (Key-Value)    | מילון הממפה בין ה-UID של כל שחקן לבין כמות הנקודות הסופית שהוא צבר במשחק.         |
| <code>targetScore</code> 🎯  | Integer            | רף הנקודות שהוגדר לתנאי עצירת המשחק (בצילום המסך שלך הערך הוא 2).                 |
| <code>winnerUid</code> 🏆    | String             | מזהה ה-UID המדויק של השחקן שהגיע ראשון ליעד והוכרז כמנצח הרשמי.                   |

### 📋 סכמת בסיס נתונים: אוסף המשתמשים (users)

| שם השדה במערכת (Field)     | טיפוס הנתון (Type) | תפקיד ושימוש השדה במערכת                                                                      |
|----------------------------|--------------------|-----------------------------------------------------------------------------------------------|
| <code>createdAt</code> 🕒   | Timestamp          | חותמת הזמן, התאריך והשעה המדויקים שבהם המשתמש נרשם לראשונה באפליקציה.                         |
| <code>displayName</code> 👤 | String             | שם התצוגה הציבורי של השחקן (השם שיופיע לחברים שלו) בחדר ההמתנה ובמשחק.                        |
| <code>email</code> ✉       | String             | כתובת הדואר האלקטרוני של המשתמש, המשמשת לזיהוי ולביצוע ה-Login למערכת.                        |
| <code>gamesPlayed</code> 🎮 | Integer            | ממד מספרי המציג את סך כל המשחקים שהשחקן שיחק מאז פתיחת החשבון ( <code>wins + losses</code> ). |
| <code>losses</code> ❌      | Integer            | מונה מספר ההפסדים הכולל של השחקן. מקודם אוטומטית ב-1 על ידי השרת בסיום משחק.                  |
| <code>totalScore</code> 🏆  | Integer            | סך כל הנקודות הצבורות שהשחקן השיג לאורך כל המשחקים שלו במערכת יחד.                            |
| <code>uid</code> 🆔         | String             | מזהה המשתמש הייחודי והמאובטח שנוצר אוטומטית על ידי מערכת Firebase Authentication.             |
| <code>wins</code> 🏆        | Integer            | מונה מספר הניצחונות הכולל של השחקן. מקודם אוטומטית ב-1 על ידי השרת בסיום משחק.                |

### 3. פעולות ומחלקות המתקשרות עם ה-Database בשרת:

ניהול ה-Database מתבצע בצורה ריכוזית דרך מחלקות הניהול בשרת:

1. `AuthManager.verify_connection(auth_payload)`: מה הפעולה עושה: מבצעת קריאה (`get()`) של מסמך המשתמש מתוך אוסף `users` על בסיס ה-UID שפוענח מהטוקן, כדי לשלוף את שמו הרשמי ולהחזירו ללובי המשחק.
2. `StatsManager.save_match(room_code, players, scores, winner_uid, ...)`: מה הפעולה עושה: מבצעת עסקת כתיבה מרוכזת (`Write Batch`). היא מוסיפה מסמך חדש לחלוטין לאוסף `matches`, ובמקביל מעדכנת (`merge=True`) את מסמכי השחקנים באוסף `users` באמצעות פקודות `firestore.Increment` המקדמות ומעדכנות את מוני הניצחונות, ההפסדים וסך הנקודות ישירות בשרת הענן, ללא דריסת שדות אחרים.





## 2. הצטרפות לחדר קיים (join\_room):

- כיוון התקשורת: לקוח (Android) אל שרת (Python).
- מבנה ההודעה (Payload): אובייקט JSON המכיל את קוד החדר שהוקלד באפליקציה:  

```
{ "roomId": "584912" }
```
- מה קורה במערכת: השרת בודק האם הקוד קיים, האם המשחק כבר התחיל והאם יש מקום פנוי בחדר. אם הכל תקין, הוא מוסיף את השחקן האורח לרשימת החדר, ומחזיר תשובה חיובית לקליינט המבקש.
- 3. עדכון מצב החדר (room\_updated):
- כיוון התקשורת: שרת (Python) אל שידור מרוכז (Broadcast) לכל חברי החדר.
- מבנה ההודעה (Payload): אובייקט JSON המכיל את רשימת השחקנים העדכנית בלובי:

```
{
 "code": "584912",
 "ownerUid": "Usr_DGNp2...",
 "status": "waiting",
 "players": [
 { "uid": "Usr_DGNp2...", "displayName": "Meydan" },
 { "uid": "Usr_1dsmF...", "displayName": "Player2" }
]
}
```

- מה קורה במערכת: בכל פעם שמשתמש נכנס לחדר או עוזב אותו, השרת "דוחף" (Emit) באופן יזום את ההודעה הזו לשני המכשירים במקביל, מה שמפעיל באנדרואיד את רענון רכיבי הטקסט והצגת השמות במסך ה-RoomActivity.

## 3. אירועי מהלך המשחק הפעיל (Gameplay Events):

1. התחלת משחק (start\_game):
- כיוון התקשורת: לקוח (מנהל החדר בלבד) אל שרת (Python).
- מבנה ההודעה (Payload): אובייקט JSON המגדיר את יעד הניקוד שהאפליקציה קבעה למשחק (למשל 10 נקודות):  

```
{ "targetScore": 10 }
```
- מה קורה במערכת: השרת מעביר את סטטוס החדר למצב "playing", מערבת את חפיסת הקלפים, מקצה קלפים ראשונים, ומשגר לשני המכשירים הודעת מערכת בשם game\_started שגורמת למסכי האנדרואיד לעבור אוטומטית ל-GameActivity.
2. משיכת מצב המשחק הפרטי (get\_game\_state):
- כיוון התקשורת: לקוח (Android) אל שרת (Python).
- מה קורה במערכת: מסך המשחק עולה באנדרואיד ומבקש מהשרת את מצב הקלפים הנוכחי שלו.
- תגובת השרת (Private Data Pushing): השרת מחזיר הודעה מאובטחת המכילה את נתוני המשחק הכלליים, בתוספת מזהה הקלף האישי אך ורק של אותו משתמש ספציפי שביקש (כדי למנוע מהטלפון להכיר את קלפי היריב):

```
{
 "roomId": "584912",
 "roundNumber": 1,
 "centerCardId": "5",
 "playerCardId": "12",
 "status": "playing",

```

```
"scores": {"Usr_DGNp2...": 0, "Usr_1dsmF...": 0}
}
```

3. שליחת לחיצה על סמל (symbol\_click):

- כיוון התקשורת: לקוח (Android) אל שרת (Python).
- מבנה ההודעה (Payload): אובייקט JSON האורז את שם הסמל עליו המשתמש הקליק במסך, יחד עם מזהה הקלף שהיה לו ביד באותו שבריר שנייה:

```
{
 "symbol": "Apple",
 "cardId": "12"
}
```

- מה קורה במערכת: מנוע השרת הריכוזי קורא את ההודעה, מוודא מול קובץ ה-Cards.json המקומי שלו האם הסמל "Apple" באמת מופיע גם בקלף המרכז ("5") וגם בקלף המשתמש ("12").
- o במקרה של טעות: השרת מחזיר ללקוח הספציפי תשובת wrong\_symbol והאפליקציה משחררת את נעילת המסך לניסיון נוסף.
- o במקרה של נכונות: השרת נועל את הסיבוב, מעלה נקודה לשחקן, מעביר את קלפו למרכז ומחלק לו קלף חדש. השרת דוחף הודעה מרוכזת לכל חברי החדר המכילה את המצב המעודכן, והאנדרואיד מרענן את התמונות של הקלפים ב-ImageView ומקדם את מספר הסיבוב.

4. סיום המשחק (game\_over):

- כיוון התקשורת: שרת (Python) אל שידור מרוכז (Broadcast) לכל חברי החדר.
- מבנה ההודעה (Payload): אובייקט JSON חגיגי המסכם את תוצאות ההתמודדות:

```
{
 "winnerUid": "Usr_DGNp2...",
 "finalScores": {"Usr_DGNp2...": 10, "Usr_1dsmF...": 7},
 "statsSaved": true
}
```

- מה קורה במערכת: מופעל אוטומטית על ידי השרת ברגע שמשתמש חצה את רף ה-targetScore. השרת כותב את המידע ל-Firebase, ומשגר את ההודעה הזו ללקוחות. אפליקציית האנדרואיד קולטת את האירוע ומעבירה את השחקנים למסך ה-GameOverActivity להצגת התוצאה הסופית.

## רפלקציה אישית

כתיבת הפרויקט הייתה דבר מסובך ופרויקט בסדר גודל שעדיין לא יצא לי להתמודד איתו. הוא נמשך לאורך חודשים ארוכים וזה דרש ממני לעבור שלב שלב מהתכנון התיאורטי ועד להתמודדויות עם כתיבת קוד במערכת מורכבת בזמן אמת. עכשיו אני אסכם את האתגרים, התובנות והמסקנות שלי על הפרויקט שביצעתי.

תהליך העבודה חולק לבניית ממשק המשתמש (UI) ב-Java באנדרואיד סטודיו, ובניית מנוע המשחק בפייתון.

- ההצלחה הגדולה ביותר הייתה הרגע שבו חיברתי שני מכשירים שונים לאותו החדר, והצלחתי לשחק בצורה סינכרונית חלקה ובמהירות תגובה מיידית.
- האתגר המרכזי היה "מצבי מרוץ" (Race Conditions) – מה קורה כששני שחקנים מקליקים על הסמל הנכון באותה המילישנייה? בתחילה, ניהול הלוגיקה בטלפון גרם להתנגשויות וקריסות.
- דרך הפתרון הייתה מעבר לארכיטקטורת שרת סמכותי (Authoritative Server). העברתי את כל כובד המשקל הלוגי לפייתון. האנדרואיד הפך לרכיב תצוגה בלבד שמדווח על לחיצות, והשרת קובע מי לחץ ראשון, נועל את הסיבוב ומעדכן את כולם במקביל. בנוסף, ניתוק יזום של מאזיני ה-Socket ב-JonDestroy פתר בעיות של זליגות זיכרון.

הפרויקט אילץ אותי ללמוד הרבה חומר לבד ולממש אותו בכוחות עצמי:

1. שפת פייתון וארכיטקטורת שרתים: הקמת שרת וניהול אובייקטים בזיכרון ה-RAM (מילונים ומודלים).
2. פרוטוקול WebSockets ו-Socket.IO: הבנת ערוצי תקשורת דו-כיווניים (Full-Duplex) בזמן אמת לעומת HTTP סטנדרטי.
3. ניהול קוד אסינכרוני (Threading): שימוש ב-runOnUiThread לעדכון בטוח של ממשק המשתמש מתוך חוט רקע.

בראייה לאחור, אני חושב שעשיתי עבודה טובה באמת. הצלחתי לבנות מאפס אפליקציה ושרת שעובדים ומאפשרים לשני אנשים לשחק אחד נגד השני בטלפון בלי תקיעות. בכל זאת, אם הייתי מתחיל את הפרויקט מחדש, הדבר המרכזי שהייתי משנה זה להקדיש הרבה יותר זמן לתכנון של הקוד והמחלקות בהתחלה. בגלל שרציתי לראות את המשחק עובד כמה שיותר מהר, לא תכננתי הכל עד הסוף, וזה גרם לי לכך שבאמצע הפיתוח הייתי חייב לעצור הכל, לקחת צעד אחורה ולהסתכל על הפרויקט שוב מלמעלה כדי לעשות סדר בבלאגן. אם הייתי מתכנן את הארכיטקטורה והסדר של המחלקות מראש, זה היה חוסך לי המון זמן ומקל עלי מאוד במהלך העבודה.

## ביבליוגרפיה

- /Socket.io Official - <https://socket.io/docs/v4/server-api>  
 Math Stack Exchange - <https://math.stackexchange.com/questions/464932/dobble-card-game-mathematical-background>  
 AsyncIO & python-socketio - <https://python-socketio.readthedocs.io/en/stable/client.html>  
 Android Developers RecyclerView Guide - <https://developer.android.com/develop/ui/views/layout/recyclerview>  
 Firebase Admin SDK Setup - <https://firebase.google.com/docs/admin/setup>

## נספחים

### קוד המקור (source code)

#### מחלקה CardDataManager

```
package me.meydan.dobble;

import android.content.Context;

import org.json.JSONObject;

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

public class CardDataManager {

 private static CardDataManager instance;

 private final Map<String, Map<String, String>> cards =
 new HashMap<>();

 private CardDataManager(Context context) {
 loadCards(context);
 }

 public static synchronized CardDataManager getInstance(
 Context context
) {
 if (instance == null) {
 instance = new CardDataManager(
 context.getApplicationContext()
);
 }

 return instance;
 }

 private void loadCards(Context context) {
```

```
try {
 InputStream inputStream =
 context.getAssets().open("Cards.json");

 BufferedReader reader = new BufferedReader(
 new InputStreamReader(inputStream)
);

 StringBuilder builder = new StringBuilder();
 String line;

 while ((line = reader.readLine()) != null) {
 builder.append(line);
 }

 reader.close();

 JSONObject root =
 new JSONObject(builder.toString());

 Iterator<String> cardIds = root.keys();

 while (cardIds.hasNext()) {
 String cardId = cardIds.next();

 JSONObject regionsObject =
 root.getJSONObject(cardId);

 Map<String, String> regions =
 new HashMap<>();

 Iterator<String> regionNames =
 regionsObject.keys();

 while (regionNames.hasNext()) {
 String regionName =
 regionNames.next();

 regions.put(
 regionName,
 regionsObject.getString(regionName)
);
 }
 }
}
```

```
 cards.put(cardId, regions);
 }

 } catch (Exception error) {
 throw new RuntimeException(
 "Could not load Cards.json",
 error
);
 }
}

public String getSymbol(
 String cardId,
 String regionName
) {
 Map<String, String> regions =
 cards.get(cardId);

 if (regions == null) {
 return null;
 }

 return regions.get(regionName);
}
}
```

**מחלקה GameActivity**

```

package me.meydan.dobble;

import android.content.Intent;
import android.graphics.Color;
import android.os.Bundle;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import org.json.JSONObject;

import io.socket.client.Ack;
import io.socket.client.Socket;
import io.socket.emitter.Emitter;

public class GameActivity extends AppCompatActivity {

 private ImageView centerCardImageView;
 private ImageView playerCardImageView;

 private TextView roundTextView;
 private TextView scoreTextView;
 private TextView gameStatusTextView;
 private String currentPlayerCardId;

 private CardDataManager cardDataManager;

 private boolean clickEnabled = true;

 private final Emitter.Listener gameStateListener = args -> {
 if (args.length == 0 || args[0] == null) {
 return;
 }

 try {
 JSONObject gameState;

 if (args[0] instanceof JSONObject) {

```



```

 gameState = (JSONObject) args[0];
 } else {
 gameState = new JSONObject(args[0].toString());
 }

 runOnUiThread() -> updateGameState(gameState));

} catch (Exception error) {
 runOnUiThread() ->
 Toast.makeText(
 GameActivity.this,
 "Invalid game state",
 Toast.LENGTH_SHORT
).show()
);
}
};

private final Emitter.Listener wrongSymbolListener = args ->
 runOnUiThread() -> {
 clickEnabled = true;

 gameStatusTextView.setText(
 "Wrong symbol!"
);

 gameStatusTextView.setTextColor(
 getColor(
 android.R.color.holo_red_dark
)
);

 gameStatusTextView.postDelayed(
 () -> {
 gameStatusTextView.setText(
 "Find the matching symbol!"
);

 gameStatusTextView.setTextColor(
 getColor(
 R.color.dobble_text_dark
)
);
 }
);
 }
};

```

```

 },
 800
);
});

private final Emitter.Listener roundResultListener = args -> {
 if (args.length == 0 || args[0] == null) {
 return;
 }

 try {
 JSONObject result;

 if (args[0] instanceof JSONObject) {
 result = (JSONObject) args[0];
 } else {
 result = new JSONObject(args[0].toString());
 }

 String correctSymbol =
 result.optString("correctSymbol", "");

 runOnUiThread() -> {
 gameStatusTextView.setText(
 "Correct symbol: " + correctSymbol
);

 gameStatusTextView.setTextColor(
 getColor(android.R.color.holo_green_dark)
);
 });

 } catch (Exception ignored) {
 }
};

private final Emitter.Listener gameOverListener = args -> {
 if (args.length == 0 || args[0] == null) {
 return;
 }

 try {
 JSONObject gameOver;

```

```

 if (args[0] instanceof JSONObject) {
 gameOver = (JSONObject) args[0];
 } else {
 gameOver = new JSONObject(
 args[0].toString()
);
 }

 runOnUiThread() ->
 openGameOver(gameOver)
);

 } catch (Exception error) {
 runOnUiThread() ->
 showToast("Invalid game-over data")
);
 }
};

@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_game);

 centerCardImageView =
 findViewById(R.id.centerCardImageView);

 playerCardImageView =
 findViewById(R.id.playerCardImageView);

 roundTextView =
 findViewById(R.id.roundTextView);

 scoreTextView =
 findViewById(R.id.scoreTextView);

 gameStatusTextView =
 findViewById(R.id.gameStatusTextView);
 cardDataManager =
 CardDataManager.getInstance(this);

 setupCardRegions();

```

```
registerSocketListeners();
}
private void setupCardRegions() {
 setupRegion(
 R.id.regionTopLeft,
 "top-left"
);

 setupRegion(
 R.id.regionTopCenter,
 "top-center"
);

 setupRegion(
 R.id.regionTopRight,
 "top-right"
);

 setupRegion(
 R.id.regionCenterLeft,
 "center-left"
);

 setupRegion(
 R.id.regionCenterCenter,
 "center-center"
);

 setupRegion(
 R.id.regionCenterRight,
 "center-right"
);

 setupRegion(
 R.id.regionBottomLeft,
 "bottom-left"
);

 setupRegion(
 R.id.regionBottomCenter,
 "bottom-center"
);
}
```

```
setupRegion(
 R.id.regionBottomRight,
 "bottom-right"
);
}

private void setupRegion(
 int viewId,
 String regionName
) {
 View regionView = findViewById(viewId);

 regionView.setOnClickListener(v -> {
 if (!clickEnabled) {
 return;
 }

 if (currentPlayerCardId == null) {
 return;
 }

 String symbol =
 cardDataManager.getSymbol(
 currentPlayerCardId,
 regionName
);

 if (symbol == null || symbol.trim().isEmpty()) {
 return;
 }

 flashRegion(regionView);
 sendSymbolClick(symbol);
 });
}

private void flashRegion(View regionView) {
 regionView.setBackgroundColor(
 Color.argb(
 110,
 255,
 214,
```

```

 0
)
);

regionView.postDelayed(
 () -> regionView.setBackgroundColor(
 Color.TRANSPARENT
),
 200
);
}

private void sendSymbolClick(String symbol) {
 Socket socket =
 SocketManager.getInstance().getSocket();

 if (socket == null || !socket.isConnected()) {
 showToast("Not connected to server");
 return;
 }

 clickEnabled = false;

 JSONObject data = new JSONObject();

 try {
 data.put(
 "cardId",
 currentPlayerCardId
);

 data.put(
 "symbol",
 symbol
);

 } catch (Exception error) {
 clickEnabled = true;
 return;
 }

 socket.emit(
 "symbol_click",

```

```

new Object[]{data},
(Ack) args -> {
 if (args.length == 0 || args[0] == null) {
 runOnUiThread(() -> {
 clickEnabled = true;
 showToast("Empty server response");
 });

 return;
 }

 try {
 JSONObject response;

 if (args[0] instanceof JSONObject) {
 response =
 (JSONObject) args[0];
 } else {
 response =
 new JSONObject(
 args[0].toString()
);
 }

 boolean ok =
 response.optBoolean(
 "ok",
 false
);

 boolean correct =
 response.optBoolean(
 "correct",
 false
);

 if (!ok) {
 String error =
 response.optString(
 "error",
 "Click failed"
);
 }
 }
}

```

```

 runOnUiThread() -> {
 clickEnabled = true;
 showToast(error);
 });

 return;
 }

 if (!correct) {
 runOnUiThread() ->
 clickEnabled = true
);
 }

} catch (Exception error) {
 runOnUiThread() -> {
 clickEnabled = true;
 showToast(
 "Invalid server response"
);
 });
}

);
}

private void showToast(String message) {
 Toast.makeText(
 GameActivity.this,
 message,
 Toast.LENGTH_LONG
).show();
}

private void registerSocketListeners() {
 Socket socket =
 SocketManager.getInstance().getSocket();

 if (socket == null || !socket.connected()) {
 Toast.makeText(
 this,
 "Not connected to server",
 Toast.LENGTH_LONG

```



```

).show();

finish();
return;
}

socket.on("game_state", gameStateListener);
socket.on("wrong_symbol", wrongSymbolListener);
socket.on("round_result", roundResultListener);
socket.on("game_over", gameOverListener);
socket.emit(
 "get_game_state",
 new Object[]{new JSONObject()},
 (Ack) args -> {
 if (args.length == 0 || args[0] == null) {
 return;
 }

 try {
 JSONObject response;

 if (args[0] instanceof JSONObject) {
 response = (JSONObject) args[0];
 } else {
 response =
 new JSONObject(args[0].toString());
 }

 if (!response.optBoolean("ok", false)) {
 return;
 }

 JSONObject game =
 response.optJSONObject("game");

 if (game != null) {
 runOnUiThread(() ->
 updateGameState(game)
);
 }

 } catch (Exception ignored) {
 }
 }

```

```

 }
);
}

private void openGameOver(JSONObject gameOver) {
 try {
 JSONObject scores =
 gameOver.getJSONObject("scores");

 org.json.JSONArray players =
 gameOver.getJSONArray("players");

 JSONObject firstPlayer =
 players.getJSONObject(0);

 JSONObject secondPlayer =
 players.getJSONObject(1);

 String firstUid =
 firstPlayer.getString("uid");

 String secondUid =
 secondPlayer.getString("uid");

 String firstName =
 firstPlayer.optString(
 "displayName",
 "Player 1"
);

 String secondName =
 secondPlayer.optString(
 "displayName",
 "Player 2"
);

 int firstScore =
 scores.optInt(firstUid, 0);

 int secondScore =
 scores.optInt(secondUid, 0);

 Intent intent = new Intent(

```

```

 GameActivity.this,
 GameOverActivity.class
);

 intent.putExtra(
 "winnerUid",
 gameOver.optString("winnerUid")
);

 intent.putExtra("firstUid", firstUid);
 intent.putExtra("secondUid", secondUid);

 intent.putExtra("firstName", firstName);
 intent.putExtra("secondName", secondName);

 intent.putExtra("firstScore", firstScore);
 intent.putExtra("secondScore", secondScore);

 Object statsSavedValue =
 gameOver.opt("statsSaved");

 boolean statsSaved =
 Boolean.TRUE.equals(statsSavedValue)
 || "true".equalsIgnoreCase(
 String.valueOf(statsSavedValue)
)
 || gameOver.has("matchId");

 intent.putExtra(
 "statsSaved",
 statsSaved
);

 startActivity(intent);
 finish();

} catch (Exception error) {
 showToast(
 "Could not open game-over screen"
);
}
}

```

```
private void updateGameState(JSONObject state) {
 String centerCardId =
 state.optString("centerCardId");

 String playerCardId =
 state.optString("playerCardId");

 currentPlayerCardId = playerCardId;
 clickEnabled = true;

 int roundNumber =
 state.optInt("roundNumber", 1);

 roundTextView.setText(
 "Round " + roundNumber
);

 JSONObject scores =
 state.optJSONObject("scores");

 if (scores != null) {
 String[] keys = new String[2];
 int index = 0;

 java.util.Iterator<String> iterator =
 scores.keys();

 while (iterator.hasNext() && index < 2) {
 keys[index] = iterator.next();
 index++;
 }

 if (index == 2) {
 int firstScore =
 scores.optInt(keys[0], 0);

 int secondScore =
 scores.optInt(keys[1], 0);

 scoreTextView.setText(
 firstScore + " - " + secondScore
);
 }
 }
}
```

```

 }
}

showCard(
 centerCardImageView,
 centerCardId
);

showCard(
 playerCardImageView,
 playerCardId
);

gameStatusTextView.setText(
 "Find the matching symbol!"
);

gameStatusTextView.setTextColor(
 getColor(R.color.dobble_text_dark)
);
}

private void showCard(
 ImageView imageView,
 String cardId
) {
 String resourceName =
 "card_" + cardId;

 int drawableId =
 getResources().getIdentifier(
 resourceName,
 "drawable",
 getPackageName()
);

 if (drawableId == 0) {
 imageView.setImageDrawable(null);

 Toast.makeText(
 this,
 "Missing image: " + resourceName,
 Toast.LENGTH_LONG

```

```
).show();

 return;
 }

 imageView.setImageResource(drawableId);
}

@Override
protected void onDestroy() {
 super.onDestroy();

 Socket socket =
 SocketManager.getInstance().getSocket();

 if (socket != null) {
 socket.off("game_state", gameStateListener);
 socket.off("wrong_symbol", wrongSymbolListener);
 socket.off("round_result", roundResultListener);
 socket.off("game_over", gameOverListener);
 }
}
}
```

## מחלקה GameOverActivity

```
package me.meydan.dobble;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

public class GameOverActivity extends AppCompatActivity {

 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_game_over);

 TextView resultTitleTextView =
 findViewById(R.id.resultTitleTextView);

 TextView resultSubtitleTextView =
 findViewById(R.id.resultSubtitleTextView);

 TextView finalScoreTextView =
 findViewById(R.id.finalScoreTextView);

 TextView playersTextView =
 findViewById(R.id.playersTextView);

 TextView saveStatusTextView =
 findViewById(R.id.saveStatusTextView);

 Button backHomeButton =
 findViewById(R.id.backHomeButton);

 String winnerUid =
 getIntent().getStringExtra("winnerUid");

 String firstUid =
```

```
 getIntent().getStringExtra("firstUid");

String secondUid =
 getIntent().getStringExtra("secondUid");

String firstName =
 getIntent().getStringExtra("firstName");

String secondName =
 getIntent().getStringExtra("secondName");

int firstScore =
 getIntent().getIntExtra("firstScore", 0);

int secondScore =
 getIntent().getIntExtra("secondScore", 0);

boolean statsSaved =
 getIntent().getBooleanExtra(
 "statsSaved",
 false
);

FirebaseUser currentUser =
 FirebaseAuth.getInstance().getCurrentUser();

boolean didWin =
 currentUser != null
 && currentUser.getId().equals(winnerUid);

if (didWin) {
 resultTitleTextView.setText("YOU WON!");
 resultSubtitleTextView.setText(
 "You found the symbols first!"
);
} else {
 resultTitleTextView.setText("YOU LOST");
 resultSubtitleTextView.setText(
 "Better luck next game!"
);
}

finalScoreTextView.setText(
```



```

 firstScore + " - " + secondScore
);

 playersTextView.setText(
 firstName + " vs " + secondName
);

 saveStatusTextView.setText(
 statsSaved
 ? "Result saved to your profile"
 : "The result could not be saved"
);

 backHomeButton.setOnClickListener(v ->
 returnHome()
);
}

private void returnHome() {
 Intent intent = new Intent(
 GameOverActivity.this,
 MainActivity.class
);

 intent.addFlags(
 Intent.FLAG_ACTIVITY_CLEAR_TOP
 | Intent.FLAG_ACTIVITY_NEW_TASK
);

 startActivity(intent);
 finish();
}

@Override
public void onBackPressed() {
 returnHome();
}
}

```

## מחלקה LoginActivity

```

package me.meydan.dobble;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;

public class LoginActivity extends AppCompatActivity {

 private EditText emailEditText;
 private EditText passwordEditText;
 private ProgressBar progressBar;

 private FirebaseAuth firebaseAuth;

 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_login);

 firebaseAuth = FirebaseAuth.getInstance();

 emailEditText = findViewById(R.id.emailEditText);
 passwordEditText = findViewById(R.id.passwordEditText);
 progressBar = findViewById(R.id.progressBar);

 Button loginButton = findViewById(R.id.loginButton);
 Button registerButton = findViewById(R.id.registerButton);

 loginButton.setOnClickListener(v -> login());

 registerButton.setOnClickListener(v -> {
 Intent intent = new Intent(

```

```

 LoginActivity.this,
 RegisterActivity.class
);
 startActivity(intent);
});
}

@Override
protected void onStart() {
 super.onStart();

 if (firebaseAuth.getCurrentUser() != null) {
 openMainActivity();
 }
}

private void login() {
 String email = emailEditText.getText().toString().trim();
 String password = passwordEditText.getText().toString();

 if (TextUtils.isEmpty(email)) {
 emailEditText.setError("Enter an email");
 return;
 }

 if (TextUtils.isEmpty(password)) {
 passwordEditText.setError("Enter a password");
 return;
 }

 setLoading(true);

 firebaseAuth.signInWithEmailAndPassword(email, password)
 .addOnCompleteListener(task -> {
 setLoading(false);

 if (task.isSuccessful()) {
 openMainActivity();
 } else {
 String message = task.getException() != null
 ? task.getException().getMessage()
 : "Login failed";
 }
 });
}

```

```
 Toast.makeText(
 LoginActivity.this,
 message,
 Toast.LENGTH_LONG
).show();
 }
});
}

private void setLoading(boolean loading) {
 progressBar.setVisibility(loading ? View.VISIBLE : View.GONE);
}

private void openMainActivity() {
 Intent intent = new Intent(
 LoginActivity.this,
 MainActivity.class
);

 intent.addFlags(
 Intent.FLAG_ACTIVITY_NEW_TASK
 | Intent.FLAG_ACTIVITY_CLEAR_TASK
);

 startActivity(intent);
 finish();
}
}
```

## MainActivity מחלקה

```
package me.meydan.dobble;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

import org.json.JSONException;
import org.json.JSONObject;

import io.socket.client.Ack;
import io.socket.client.Socket;

public class MainActivity extends AppCompatActivity {

 private FirebaseAuth firebaseAuth;
 private FirebaseFirestore firestore;

 private TextView userTextView;
 private TextView serverStatusTextView;
 private ProgressBar serverProgressBar;
 private EditText roomCodeEditText;

 private Button createRoomButton;
 private Button joinRoomButton;
 private Button reconnectButton;

 private String displayName;

 @Override
```

```

protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);

 firebaseAuth = FirebaseAuth.getInstance();
 firestore = FirebaseFirestore.getInstance();

 userTextView = findViewById(R.id.userTextView);
 serverStatusTextView = findViewById(R.id.serverStatusTextView);
 serverProgressBar = findViewById(R.id.serverProgressBar);
 roomCodeEditText = findViewById(R.id.roomCodeEditText);

 createRoomButton = findViewById(R.id.createRoomButton);
 joinRoomButton = findViewById(R.id.joinRoomButton);
 reconnectButton = findViewById(R.id.reconnectButton);
 Button logoutButton = findViewById(R.id.logoutButton);
 Button matchHistoryButton =
 findViewById(R.id.matchHistoryButton);

 FirebaseUser user = firebaseAuth.getCurrentUser();

 if (user == null) {
 openLogin();
 return;
 }

 displayName = null;
 userTextView.setText("Loading profile...");

 setRoomButtonsEnabled(false);

 loadDisplayName(user);

 createRoomButton.setOnClickListener(v -> createRoom());
 joinRoomButton.setOnClickListener(v -> joinRoom());
 reconnectButton.setOnClickListener(v -> connectToServer());
 logoutButton.setOnClickListener(v -> logout());
 matchHistoryButton.setOnClickListener(v -> {
 Intent intent = new Intent(
 MainActivity.this,
 MatchHistoryActivity.class
);
 });

```

```

 startActivity(intent);
 });

 setRoomButtonsEnabled(false);
 connectToServer();
}

private void loadDisplayName(FirebaseUser user) {
 firestore.collection("users")
 .document(user.getUid())
 .get()
 .addOnSuccessListener(document -> {
 String savedName =
 document.getString("displayName");

 if (savedName != null
 && !savedName.trim().isEmpty()) {

 displayName = savedName.trim();

 } else if (user.getDisplayName() != null
 && !user.getDisplayName().trim().isEmpty()) {

 displayName =
 user.getDisplayName().trim();

 } else {
 displayName = user.getEmail();
 }

 userTextView.setText(
 "Hello, " + displayName
);

 if (SocketManager.getInstance().isConnected()) {
 setRoomButtonsEnabled(true);
 }
 })
 .addOnFailureListener(error -> {
 if (user.getDisplayName() != null
 && !user.getDisplayName().trim().isEmpty()) {

 displayName =

```

```

 user.getDisplayName().trim();

 } else {
 displayName = user.getEmail();
 }

 userTextView.setText(
 "Hello, " + displayName
);

 if (SocketManager.getInstance().isConnected()) {
 setRoomButtonsEnabled(true);
 }
});
}

private void connectToServer() {
 FirebaseUser user = firebaseAuth.getCurrentUser();

 if (user == null) {
 openLogin();
 return;
 }

 setConnectionLoading(true);
 setRoomButtonsEnabled(false);

 serverStatusTextView.setText("Getting Firebase token...");

 user.getIdToken(true)
 .addOnSuccessListener(result -> {
 String token = result.getToken();

 if (token == null || token.trim().isEmpty()) {
 setConnectionLoading(false);
 serverStatusTextView.setText("Firebase token is empty");
 return;
 }

 serverStatusTextView.setText("Connecting to server...");

 SocketManager.getInstance().connect(
 token,

```



```

new SocketManager.ConnectionListener() {
 @Override
 public void onConnected() {
 runOnUiThread() -> {
 setConnectionLoading(false);
 if (displayName != null) {
 setRoomButtonsEnabled(true);
 }
 serverStatusTextView.setText(
 "Connected to server"
);
 });
 }

 @Override
 public void onServerAccepted(
 JSONObject userData
) {
 runOnUiThread() -> {
 setConnectionLoading(false);
 if (displayName != null) {
 setRoomButtonsEnabled(true);
 }
 serverStatusTextView.setText(
 "Connected to server"
);
 });
 }

 @Override
 public void onError(String message) {
 runOnUiThread() -> {
 setConnectionLoading(false);
 setRoomButtonsEnabled(false);

 serverStatusTextView.setText(
 "Connection failed:\n" + message
);
 });
 }

 @Override
 public void onDisconnected(String reason) {

```

```

 runOnUiThread() -> {
 setConnectionLoading(false);
 setRoomButtonsEnabled(false);

 serverStatusTextView.setText(
 "Disconnected:\n" + reason
);
 });
 }
}

);
})
.addOnFailureListener(error -> {
 setConnectionLoading(false);
 setRoomButtonsEnabled(false);

 serverStatusTextView.setText(
 "Failed to get Firebase token:\n"
 + error.getMessage()
);
});
}

private void createRoom() {
 Socket socket = SocketManager.getInstance().getSocket();

 if (socket == null || !socket.connected()) {
 Toast.makeText(
 this,
 "Not connected to server",
 Toast.LENGTH_SHORT
).show();
 return;
 }

 setRoomButtonsEnabled(false);

 JSONObject data = new JSONObject();

 try {
 data.put("displayName", displayName);
 } catch (JSONException error) {
 setRoomButtonsEnabled(true);
 }
}

```

```

 return;
 }

 socket.emit("create_room", data, (Ack) args -> {
 if (args.length == 0) {
 runOnUiThread() -> {
 setRoomButtonsEnabled(true);
 showToast("Empty server response");
 };
 return;
 }

 handleRoomResponse(args[0]);
 });
}

private void joinRoom() {
 String roomCode =
 roomCodeEditText.getText().toString().trim();

 if (TextUtils.isEmpty(roomCode)) {
 roomCodeEditText.setError("Enter room code");
 return;
 }

 if (roomCode.length() != 6) {
 roomCodeEditText.setError("Room code must contain 6 digits");
 return;
 }

 Socket socket = SocketManager.getInstance().getSocket();

 if (socket == null || !socket.connected()) {
 showToast("Not connected to server");
 return;
 }

 setRoomButtonsEnabled(false);

 JSONObject data = new JSONObject();

 try {
 data.put("roomCode", roomCode);

```

```

 data.put("displayName", displayName);
 } catch (JSONException error) {
 setRoomButtonsEnabled(true);
 return;
 }

 socket.emit("join_room", data, (Ack) args -> {
 if (args.length == 0) {
 runOnUiThread() -> {
 setRoomButtonsEnabled(true);
 showToast("Empty server response");
 };
 return;
 }

 handleRoomResponse(args[0]);
 });
}

private void handleRoomResponse(Object responseObject) {
 try {
 JSONObject response;

 if (responseObject instanceof JSONObject) {
 response = (JSONObject) responseObject;
 } else {
 response = new JSONObject(responseObject.toString());
 }

 boolean ok = response.optBoolean("ok", false);

 if (!ok) {
 String error = response.optString(
 "error",
 "Room operation failed"
);

 runOnUiThread() -> {
 setRoomButtonsEnabled(true);
 showToast(error);
 };

 return;
 }
 }
}

```

```

 }

 JSONObject room = response.getJSONObject("room");
 String roomCode = room.getString("code");
 String roomJson = room.toString();

 runOnUiThread() -> openRoom(roomCode, roomJson));

} catch (Exception error) {
 runOnUiThread() -> {
 setRoomButtonsEnabled(true);
 showToast("Invalid server response");
 });
}
}

private void openRoom(String roomCode, String roomJson) {
 Intent intent = new Intent(
 MainActivity.this,
 RoomActivity.class
);

 intent.putExtra("roomCode", roomCode);
 intent.putExtra("roomJson", roomJson);

 startActivity(intent);
}

private void setConnectionLoading(boolean loading) {
 serverProgressBar.setVisibility(
 loading ? View.VISIBLE : View.GONE
);

 reconnectButton.setEnabled(!loading);
}

private void setRoomButtonsEnabled(boolean enabled) {
 createRoomButton.setEnabled(enabled);
 joinRoomButton.setEnabled(enabled);
}

private void showToast(String message) {
 Toast.makeText(

```

```
 MainActivity.this,
 message,
 Toast.LENGTH_LONG
).show();
}

private void logout() {
 SocketManager.getInstance().disconnect();
 firebaseAuth.signOut();
 openLogin();
}

private void openLogin() {
 Intent intent = new Intent(
 MainActivity.this,
 LoginActivity.class
);

 intent.addFlags(
 Intent.FLAG_ACTIVITY_NEW_TASK
 | Intent.FLAG_ACTIVITY_CLEAR_TASK
);

 startActivity(intent);
 finish();
}
}
```

**MatchHistoryActivity מחלקה**

```

package me.meydan.dobble;

import android.os.Bundle;
import android.view.View;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.firebase.Timestamp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;

import java.util.ArrayList;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class MatchHistoryActivity extends AppCompatActivity {

 private final List<MatchItem> matches = new ArrayList<>();
 private final Map<String, String> userNames = new HashMap<>();

 private MatchHistoryAdapter adapter;
 private ProgressBar historyProgressBar;
 private TextView emptyHistoryTextView;

 private FirebaseFirestore firestore;

 private int pendingMatches = 0;

 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_match_history);

```

```

 firestore = FirebaseFirestore.getInstance();

 findViewById(R.id.backButton)
 .setOnClickListener(v -> finish());

 RecyclerView matchesRecyclerView =
 findViewById(R.id.matchesRecyclerView);

 historyProgressBar =
 findViewById(R.id.historyProgressBar);

 emptyHistoryTextView =
 findViewById(R.id.emptyHistoryTextView);

 adapter = new MatchHistoryAdapter(matches);

 matchesRecyclerView.setLayoutManager(
 new LinearLayoutManager(this)
);

 matchesRecyclerView.setAdapter(adapter);

 loadMatches();
}

private void loadMatches() {
 FirebaseUser user =
 FirebaseAuth.getInstance().getCurrentUser();

 if (user == null) {
 finish();
 return;
 }

 String currentUid = user.getUid();

 firestore.collection("matches")
 .whereArrayContains("players", currentUid)
 .get()
 .addOnSuccessListener(querySnapshot -> {
 matches.clear();

```



```

 if (querySnapshot.isEmpty()) {
 finishLoading();
 return;
 }

 pendingMatches = querySnapshot.size();

 for (QueryDocumentSnapshot document : querySnapshot) {
 loadSingleMatch(document, currentUid);
 }
 })
 .addOnFailureListener(error -> {
 historyProgressBar.setVisibility(View.GONE);

 Toast.makeText(
 MatchHistoryActivity.this,
 error.getMessage(),
 Toast.LENGTH_LONG
).show();
 });
}

private void loadSingleMatch(
 QueryDocumentSnapshot document,
 String currentUid
) {
 List<String> players =
 (List<String>) document.get("players");

 Map<String, Object> scores =
 (Map<String, Object>) document.get("scores");

 Map<String, Object> playerNames =
 (Map<String, Object>) document.get("playerNames");

 String winnerUid =
 document.getString("winnerUid");

 if (players == null
 || scores == null
 || players.size() < 2) {

 onMatchLoaded();
 }
}

```

```
 return;
}

String opponentUid =
 players.get(0).equals(currentUid)
 ? players.get(1)
 : players.get(0);

int myScore =
 numberToInt(scores.get(currentUid));

int opponentScore =
 numberToInt(scores.get(opponentUid));

Timestamp timestamp =
 document.getTimestamp("createdAt");

Date createdAt =
 timestamp != null
 ? timestamp.toDate()
 : null;

boolean won =
 currentUid.equals(winnerUid);

String storedOpponentName = null;

if (playerNames != null
 && playerNames.get(opponentUid) != null) {

 storedOpponentName =
 String.valueOf(
 playerNames.get(opponentUid)
);
}

if (storedOpponentName != null
 && !storedOpponentName.trim().isEmpty()
 && !storedOpponentName.equals(opponentUid)) {

 addMatch(
 storedOpponentName,
 myScore,
```

```

 opponentScore,
 won,
 createdAt
);

 return;
}

loadOpponentName(
 opponentUid,
 myScore,
 opponentScore,
 won,
 createdAt
);
}

private void loadOpponentName(
 String opponentUid,
 int myScore,
 int opponentScore,
 boolean won,
 Date createdAt
) {
 if (userNames.containsKey(opponentUid)) {
 addMatch(
 userNames.get(opponentUid),
 myScore,
 opponentScore,
 won,
 createdAt
);

 return;
 }

 firestore.collection("users")
 .document(opponentUid)
 .get()
 .addOnSuccessListener(document -> {
 String opponentName =
 document.getString("displayName");

```

```

 if (opponentName == null
 || opponentName.trim().isEmpty()) {

 opponentName =
 document.getString("email");
 }

 if (opponentName == null
 || opponentName.trim().isEmpty()) {

 opponentName = "Unknown player";
 }

 userNames.put(
 opponentUid,
 opponentName
);

 addMatch(
 opponentName,
 myScore,
 opponentScore,
 won,
 createdAt
);
 })
 .addOnFailureListener(error ->
 addMatch(
 "Unknown player",
 myScore,
 opponentScore,
 won,
 createdAt
)
);
}

```

```

private void addMatch(
 String opponentName,
 int myScore,
 int opponentScore,
 boolean won,
 Date createdAt

```

```

) {
 matches.add(
 new MatchItem(
 opponentName,
 myScore,
 opponentScore,
 won,
 createdAt
)
);

 onMatchLoaded();
 }

 private void onMatchLoaded() {
 pendingMatches--;

 if (pendingMatches > 0) {
 return;
 }

 matches.sort((first, second) -> {
 if (first.getCreatedAt() == null) {
 return 1;
 }

 if (second.getCreatedAt() == null) {
 return -1;
 }

 return second.getCreatedAt()
 .compareTo(first.getCreatedAt());
 });

 finishLoading();
 }

 private void finishLoading() {
 historyProgressBar.setVisibility(View.GONE);

 emptyHistoryTextView.setVisibility(
 matches.isEmpty()
 ? View.VISIBLE

```

```
 : View.GONE
);

 adapter.notifyDataSetChanged();
}

private int numberToInt(Object value) {
 if (value instanceof Number) {
 return ((Number) value).intValue();
 }

 return 0;
}
}
```

## מחלקה MatchHistoryAdapter

```

package me.meydan.dobble;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.text.SimpleDateFormat;
import java.util.List;
import java.util.Locale;

public class MatchHistoryAdapter
 extends RecyclerView.Adapter<MatchHistoryAdapter.MatchViewHolder> {

 private final List<MatchItem> matches;

 public MatchHistoryAdapter(List<MatchItem> matches) {
 this.matches = matches;
 }

 @NonNull
 @Override
 public MatchViewHolder onCreateViewHolder(
 @NonNull ViewGroup parent,
 int viewType
) {
 View view = LayoutInflater.from(parent.getContext())
 .inflate(
 R.layout.item_match,
 parent,
 false
);

 return new MatchViewHolder(view);
 }

 @Override
 public void onBindViewHolder(

```

```

 @NonNull MatchViewHolder holder,
 int position
) {
 MatchItem match = matches.get(position);

 holder.resultTextView.setText(
 match.isWon() ? "VICTORY" : "DEFEAT"
);

 holder.resultTextView.setTextColor(
 holder.itemView.getContext().getColor(
 match.isWon()
 ? R.color.dobble_purple
 : android.R.color.holo_red_dark
)
);

 holder.opponentTextView.setText(
 "Against: " + match.getOpponentName()
);

 holder.matchScoreTextView.setText(
 "Score: "
 + match.getMyScore()
 + " - "
 + match.getOpponentScore()
);

 if (match.getCreatedAt() != null) {
 SimpleDateFormat formatter =
 new SimpleDateFormat(
 "dd/MM/yyyy HH:mm",
 Locale.getDefault()
);

 holder.matchDateTextView.setText(
 formatter.format(match.getCreatedAt())
);
 } else {
 holder.matchDateTextView.setText("");
 }
 }
}

```



```
@Override
public int getItemCount() {
 return matches.size();
}

static class MatchViewHolder
 extends RecyclerView.ViewHolder {

 TextView resultTextView;
 TextView opponentTextView;
 TextView matchScoreTextView;
 TextView matchDateTextView;

 public MatchViewHolder(@NonNull View itemView) {
 super(itemView);

 resultTextView =
 itemView.findViewById(
 R.id.resultTextView
);

 opponentTextView =
 itemView.findViewById(
 R.id.opponentTextView
);

 matchScoreTextView =
 itemView.findViewById(
 R.id.matchScoreTextView
);

 matchDateTextView =
 itemView.findViewById(
 R.id.matchDateTextView
);
 }
}
```

## מחלקה MatchItem

```
package me.meydan.dobble;

import java.util.Date;

public class MatchItem {

 private final String opponentName;
 private final int myScore;
 private final int opponentScore;
 private final boolean won;
 private final Date createdAt;

 public MatchItem(
 String opponentName,
 int myScore,
 int opponentScore,
 boolean won,
 Date createdAt
) {
 this.opponentName = opponentName;
 this.myScore = myScore;
 this.opponentScore = opponentScore;
 this.won = won;
 this.createdAt = createdAt;
 }

 public String getOpponentName() {
 return opponentName;
 }

 public int getMyScore() {
 return myScore;
 }

 public int getOpponentScore() {
 return opponentScore;
 }

 public boolean isWon() {
 return won;
 }
}
```

```
public Date getCreatedAt() {
 return createdAt;
}
}
```

## מחלקה RegisterActivity

```

package me.meydan.dobble;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.auth.UserProfileChangeRequest;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.HashMap;
import java.util.Map;

public class RegisterActivity extends AppCompatActivity {

 private EditText nameEditText;
 private EditText emailEditText;
 private EditText passwordEditText;
 private EditText confirmPasswordEditText;
 private ProgressBar progressBar;
 private Button createAccountButton;

 private FirebaseAuth firebaseAuth;
 private FirebaseFirestore firestore;

 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_register);

 firebaseAuth = FirebaseAuth.getInstance();
 firestore = FirebaseFirestore.getInstance();

```

```

nameEditText = findViewById(R.id.nameEditText);
emailEditText = findViewById(R.id.emailEditText);
passwordEditText = findViewById(R.id.passwordEditText);
confirmPasswordEditText =
 findViewById(R.id.confirmPasswordEditText);
progressBar = findViewById(R.id.progressBar);
createAccountButton =
 findViewById(R.id.createAccountButton);

createAccountButton.setOnClickListener(v -> register());
}

private void register() {
 String name = nameEditText.getText().toString().trim();
 String email = emailEditText.getText().toString().trim();
 String password = passwordEditText.getText().toString();
 String confirmPassword =
 confirmPasswordEditText.getText().toString();

 if (TextUtils.isEmpty(name)) {
 nameEditText.setError("Enter a display name");
 return;
 }

 if (TextUtils.isEmpty(email)) {
 emailEditText.setError("Enter an email");
 return;
 }

 if (password.length() < 6) {
 passwordEditText.setError(
 "Password must contain at least 6 characters"
);
 return;
 }

 if (!password.equals(confirmPassword)) {
 confirmPasswordEditText.setError(
 "Passwords do not match"
);
 return;
 }
}

```

```

setLoading(true);

firebaseAuth.createUserWithEmailAndPassword(email, password)
 .addOnCompleteListener(task -> {
 if (!task.isSuccessful()) {
 setLoading(false);

 String message = task.getException() != null
 ? task.getException().getMessage()
 : "Registration failed";

 Toast.makeText(
 RegisterActivity.this,
 message,
 Toast.LENGTH_LONG
).show();

 return;
 }

 FirebaseUser user = firebaseAuth.getCurrentUser();

 if (user == null) {
 setLoading(false);

 Toast.makeText(
 RegisterActivity.this,
 "Could not load the new user",
 Toast.LENGTH_LONG
).show();

 return;
 }

 updateFirebaseProfile(user, name);
 saveUserToFirestore(user, name, email);

 // כדי לעבור מסך Firestore-לא מחכים ל
 openMainActivity();
 });
}

```

```

private void updateFirebaseProfile(
 FirebaseUser user,
 String displayName
) {
 UserProfileChangeRequest profileUpdates =
 new UserProfileChangeRequest.Builder()
 .setDisplayName(displayName)
 .build();

 user.updateProfile(profileUpdates)
 .addOnFailureListener(error ->
 Toast.makeText(
 RegisterActivity.this,
 "Profile name was not saved",
 Toast.LENGTH_SHORT
).show()
);
}

private void saveUserToFirestore(
 FirebaseUser user,
 String name,
 String email
) {
 Map<String, Object> userData = new HashMap<>();

 userData.put("uid", user.getId());
 userData.put("displayName", name);
 userData.put("email", email);
 userData.put("wins", 0);
 userData.put("losses", 0);
 userData.put("gamesPlayed", 0);
 userData.put("totalScore", 0);
 userData.put("createdAt", FieldValue.serverTimestamp());

 firestore.collection("users")
 .document(user.getId())
 .set(userData)
 .addOnFailureListener(error ->
 Toast.makeText(
 RegisterActivity.this,
 "Account created, but profile data was not saved: "
 + error.getMessage(),

```

```
 Toast.LENGTH_LONG
).show()
);
}

private void setLoading(boolean loading) {
 progressBar.setVisibility(
 loading ? View.VISIBLE : View.GONE
);

 createAccountButton.setEnabled(!loading);
}

private void openMainActivity() {
 Intent intent = new Intent(
 RegisterActivity.this,
 MainActivity.class
);

 intent.addFlags(
 Intent.FLAG_ACTIVITY_NEW_TASK
 | Intent.FLAG_ACTIVITY_CLEAR_TASK
);

 startActivity(intent);
 finish();
}
}
```



## מחלקה RoomActivity

```
package me.meydan.dobble;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

import org.json.JSONArray;
import org.json.JSONObject;

import io.socket.client.Ack;
import io.socket.client.Socket;
import io.socket.emitter.Emitter;

public class RoomActivity extends AppCompatActivity {

 private TextView roomCodeTextView;
 private TextView statusTextView;
 private TextView playerOneTextView;
 private TextView playerTwoTextView;
 private Button startGameButton;

 private String roomCode;
 private String currentUid;
 private boolean isOwner = false;

 private final Emitter.Listener roomUpdatedListener = args -> {
 if (args.length == 0 || args[0] == null) {
 return;
 }

 try {
 JSONObject room;
```

```

 if (args[0] instanceof JSONObject) {
 room = (JSONObject) args[0];
 } else {
 room = new JSONObject(args[0].toString());
 }

 runOnUiThread() -> updateRoomUi(room));

 } catch (Exception error) {
 runOnUiThread() ->
 Toast.makeText(
 RoomActivity.this,
 "Invalid room update",
 Toast.LENGTH_SHORT
).show()
);
 }
};

private final Emitter.Listener gameStartedListener = args ->
 runOnUiThread() -> {
 Intent intent = new Intent(
 RoomActivity.this,
 GameActivity.class
);

 startActivity(intent);
 });

@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_room);

 roomCode = getIntent().getStringExtra("roomCode");
 String roomJson = getIntent().getStringExtra("roomJson");

 if (roomCode == null || roomCode.trim().isEmpty()) {
 finish();
 return;
 }

 FirebaseUser user =

```

```

 FirebaseAuth.getInstance().getCurrentUser();

 if (user == null) {
 finish();
 return;
 }

 currentUid = user.getId();

 roomCodeTextView =
 findViewById(R.id.roomCodeTextView);
 statusTextView =
 findViewById(R.id.statusTextView);
 playerOneTextView =
 findViewById(R.id.playerOneTextView);
 playerTwoTextView =
 findViewById(R.id.playerTwoTextView);
 startGameButton =
 findViewById(R.id.startGameButton);

 Button leaveRoomButton =
 findViewById(R.id.leaveRoomButton);

 roomCodeTextView.setText(roomCode);
 if (roomJson != null && !roomJson.trim().isEmpty()) {
 try {
 JSONObject initialRoom = new JSONObject(roomJson);
 updateRoomUi(initialRoom);
 } catch (Exception error) {
 Toast.makeText(
 RoomActivity.this,
 "Could not load room details",
 Toast.LENGTH_SHORT
).show();
 }
 }

 startGameButton.setOnClickListener(v ->
 startGame()
);

 leaveRoomButton.setOnClickListener(v ->
 leaveRoom()

```

```
);

registerSocketListeners();
}

private void registerSocketListeners() {
 Socket socket = SocketManager.getInstance().getSocket();

 if (socket == null) {
 finish();
 return;
 }

 socket.on("room_updated", roomUpdatedListener);
 socket.on("game_started", gameStartedListener);
}

private void updateRoomUi(JSONObject room) {
 String ownerId = room.optString("ownerId");
 isOwner = currentUid.equals(ownerId);

 JSONArray players = room.optJSONArray("players");

 if (players == null) {
 return;
 }

 if (players.length() >= 1) {
 JSONObject first = players.optJSONObject(0);

 if (first != null) {
 playerOneTextView.setText(
 first.optString("displayName", "Player 1")
);
 }
 }

 if (players.length() >= 2) {
 JSONObject second = players.optJSONObject(1);

 if (second != null) {
 playerTwoTextView.setText(
 second.optString("displayName", "Player 2")
);
 }
 }
}
```

```

);
}

statusTextView.setText("Both players are ready");

startGameButton.setVisibility(
 isOwner ? View.VISIBLE : View.GONE
);

} else {
 playerTwoTextView.setText("Waiting...");
 statusTextView.setText(
 "Waiting for another player..."
);
 startGameButton.setVisibility(View.GONE);
}
}

private void startGame() {
 if (!isOwner) {
 return;
 }

 Socket socket = SocketManager.getInstance().getSocket();

 if (socket == null || !socket.connected()) {
 showToast("Not connected to server");
 return;
 }

 JSONObject data = new JSONObject();

 try {
 data.put("targetScore", 20);
 } catch (Exception ignored) {}

 socket.emit("start_game", data, (Ack) args -> {
 if (args.length == 0) {
 return;
 }

 try {

```

```

JSONObject response;

if (args[0] instanceof JSONObject) {
 response = (JSONObject) args[0];
} else {
 response = new JSONObject(args[0].toString());
}

if (!response.optBoolean("ok", false)) {
 String error = response.optString(
 "error",
 "Could not start game"
);

 runOnUiThread(() -> showToast(error));
}

} catch (Exception error) {
 runOnUiThread(() ->
 showToast("Invalid server response")
);
}
});
}

private void leaveRoom() {
 Socket socket = SocketManager.getInstance().getSocket();

 if (socket == null || !socket.connected()) {
 openMain();
 return;
 }

 socket.emit("leave_room", new JSONObject(), (Ack) args ->
 runOnUiThread(this::openMain)
);
}

private void openMain() {
 Intent intent = new Intent(
 RoomActivity.this,
 MainActivity.class
);

```

```
intent.addFlags(
 Intent.FLAG_ACTIVITY_CLEAR_TOP
 | Intent.FLAG_ACTIVITY_SINGLE_TOP
);

startActivity(intent);
finish();
}

private void showToast(String message) {
 Toast.makeText(
 RoomActivity.this,
 message,
 Toast.LENGTH_LONG
).show();
}

@Override
protected void onDestroy() {
 super.onDestroy();

 Socket socket = SocketManager.getInstance().getSocket();

 if (socket != null) {
 socket.off("room_updated", roomUpdatedListener);
 socket.off("game_started", gameStartedListener);
 }
}
}
```

## מחלקה SocketManager

```
package me.meydan.dobble;

import android.util.Log;

import org.json.JSONObject;

import java.net.URI;
import java.util.HashMap;
import java.util.Map;

import io.socket.client.IO;
import io.socket.client.Socket;

public class SocketManager {

 private static final String TAG = "SocketManager";

 /*
 * Android:
 * http://10.0.2.2:5000
 *
 * טלפון אמיתי:
 * של המחשב, לדוגמה IPv4-החלף בכתובת ה
 * http://192.168.1.25:5000
 */
 private static final String SERVER_URL =
 "http://10.0.2.2:5000";

 private static SocketManager instance;

 private Socket socket;

 private SocketManager() {
 }

 public static synchronized SocketManager getInstance() {
 if (instance == null) {
 instance = new SocketManager();
 }

 return instance;
 }
}
```



```

}

public void connect(
 String firebaseToken,
 ConnectionListener listener
) {
 disconnect();

 try {
 Map<String, String> auth = new HashMap<>();
 auth.put("token", firebaseToken);

 IO.Options options = IO.Options.builder()
 .setAuth(auth)
 .setReconnection(true)
 .setReconnectionAttempts(10)
 .setReconnectionDelay(1000)
 .setTimeout(10000)
 .setForceNew(true)
 .build();

 socket = IO.socket(
 URI.create(SERVER_URL),
 options
);

 registerBaseListeners(listener);

 socket.connect();

 } catch (Exception error) {
 Log.e(TAG, "Failed to create socket", error);

 if (listener != null) {
 listener.onError(error.getMessage());
 }
 }
}

private void registerBaseListeners(
 ConnectionListener listener
) {
 socket.on(Socket.EVENT_CONNECT, args -> {

```

```
Log.d(TAG, "Socket connected: " + socket.id());

if (listener != null) {
 listener.onConnected();
}
});

socket.on("connected", args -> {
 if (args.length == 0 || args[0] == null) {
 Log.e(TAG, "Server sent empty connected event");

 if (listener != null) {
 listener.onError("Server sent empty connection data");
 }

 return;
 }

 try {
 JSONObject data;

 if (args[0] instanceof JSONObject) {
 data = (JSONObject) args[0];
 } else {
 data = new JSONObject(args[0].toString());
 }

 Log.d(TAG, "Server accepted connection: " + data);

 if (listener != null) {
 listener.onServerAccepted(data);
 }

 } catch (Exception error) {
 Log.e(TAG, "Failed to parse connected event", error);

 if (listener != null) {
 listener.onError(
 "Invalid server response: " + error.getMessage()
);
 }
 }
});
```

```

socket.on(Socket.EVENT_CONNECT_ERROR, args -> {
 String message = "Connection error";

 if (args.length > 0 && args[0] != null) {
 message = args[0].toString();
 }

 Log.e(TAG, message);

 if (listener != null) {
 listener.onError(message);
 }
});

socket.on(Socket.EVENT_DISCONNECT, args -> {
 String reason = "Disconnected";

 if (args.length > 0 && args[0] != null) {
 reason = args[0].toString();
 }

 Log.d(TAG, reason);

 if (listener != null) {
 listener.onDisconnected(reason);
 }
});
}

public Socket getSocket() {
 return socket;
}

public boolean isConnected() {
 return socket != null && socket.connected();
}

public void disconnect() {
 if (socket == null) {
 return;
 }
}

```

```
socket.off();
socket.disconnect();
socket.close();
socket = null;
}
```

```
public interface ConnectionListener {

 void onConnected();

 void onServerAccepted(JSONObject userData);

 void onError(String message);

 void onDisconnected(String reason);
}
}
```

**מחלקה app.py**

```

import os
from pathlib import Path
from typing import Any, Dict

import socketio
import uvicorn
from dotenv import load_dotenv

from managers.auth_manager import AuthManager
from managers.card_manager import CardManager
from managers.game_manager import GameManager
from managers.room_manager import RoomManager
from managers.stats_manager import StatsManager

ROOT = Path(__file__).resolve().parent
load_dotenv(ROOT / ".env")

card_manager = CardManager(ROOT / "data" / "Cards.json")
auth_manager = AuthManager(ROOT)
room_manager = RoomManager()
game_manager = GameManager(card_manager)
stats_manager = StatsManager()

sio = socketio.AsyncServer(
 async_mode="asgi",
 cors_allowed_origins="*",
 logger=True,
 engineio_logger=True,
)

app = socketio.ASGIApp(sio)

def success(data: Dict[str, Any] | None = None) -> Dict[str, Any]:
 response: Dict[str, Any] = {"ok": True}
 if data:
 response.update(data)
 return response

```

```

def failure(message: str) -> Dict[str, Any]:
 return {"ok": False, "error": message}

async def emit_private_game_states(room) -> None:
 for uid, player in room.players.items():
 private_state = game_manager.get_private_state(room, uid)
 await sio.emit("game_state", private_state, to=player.sid)

@sio.event
async def connect(sid: str, environ: dict, auth: Any):
 try:
 user = auth_manager.verify_connection(auth)
 except Exception as error:
 print(f"Connection rejected for {sid}: {error}")
 raise socketio.exceptions.ConnectionRefusedError(str(error))

 await sio.save_session(sid, user)
 await sio.emit(
 "connected",
 {
 "uid": user["uid"],
 "displayName": user["name"],
 "cardsLoaded": len(card_manager.cards),
 },
 to=sid,
)
 print(f"Connected: {user['uid']} ({sid})")

@sio.event
async def disconnect(sid: str, reason: str):
 room = room_manager.get_room_for_sid(sid)

 if room:
 was_playing = room.status == "playing"
 room_code = room.code
 leaving_user = next(
 (uid for uid, player in room.players.items() if player.sid == sid),
 None,
)

```

```
updated_room = room_manager.leave_by_sid(sid)
await sio.leave_room(sid, room_code)
```

```
if was_playing:
 game_manager.cancel_game(room_code)
 if updated_room:
 updated_room.status = "waiting"
 await sio.emit(
 "game_cancelled",
 {
 "reason": "opponent_disconnected",
 "disconnectedUid": leaving_user,
 },
 room=room_code,
)

 if updated_room:
 await sio.emit("room_updated", updated_room.to_dict(), room=room_code)

print(f'Disconnected: {sid}; reason={reason}')
```

```
@sio.event
async def ping_server(sid: str, data: Any = None):
 return success({"message": "pong", "received": data})
```

```
@sio.event
async def create_room(sid: str, data: Any = None):
 try:
 user = await sio.get_session(sid)
 display_name = str(
 user.get("name") or user["uid"]
).strip()

 room = room_manager.create_room(
 uid=user["uid"],
 sid=sid,
 display_name=display_name,
)
 await sio.enter_room(sid, room.code)
 await sio.emit("room_updated", room.to_dict(), room=room.code)
```

```

 return success({"room": room.to_dict()})
except Exception as error:
 return failure(str(error))

```

```

@sio.event
async def join_room(sid: str, data: Any = None):
 try:
 if not isinstance(data, dict):
 raise ValueError("Expected an object containing roomCode")

 code = str(data.get("roomCode", "")).strip()
 if not code:
 raise ValueError("roomCode is required")

 user = await sio.get_session(sid)
 display_name = str(
 user.get("name") or user["uid"]
).strip()

 room = room_manager.join_room(
 code=code,
 uid=user["uid"],
 sid=sid,
 display_name=display_name,
)
 await sio.enter_room(sid, room.code)
 await sio.emit("room_updated", room.to_dict(), room=room.code)

 return success({"room": room.to_dict()})
 except Exception as error:
 return failure(str(error))

```

```

@sio.event
async def leave_room(sid: str, data: Any = None):
 try:
 room = room_manager.get_room_for_sid(sid)
 if room is None:
 raise ValueError("You are not inside a room")

 code = room.code
 if room.status == "playing":

```



```

game_manager.cancel_game(code)

updated_room = room_manager.leave_by_sid(sid)
await sio.leave_room(sid, code)

if updated_room:
 updated_room.status = "waiting"
 await sio.emit("game_cancelled", {"reason": "player_left"}, room=code)
 await sio.emit("room_updated", updated_room.to_dict(), room=code)

return success()
except Exception as error:
 return failure(str(error))

@sio.event
async def start_game(sid: str, data: Any = None):
 try:
 room = room_manager.get_room_for_sid(sid)
 if room is None:
 raise ValueError("You are not inside a room")

 user = await sio.get_session(sid)
 if room.owner_uid != user["uid"]:
 raise ValueError("Only the room owner can start the game")

 payload = data if isinstance(data, dict) else {}
 target_score = int(payload.get("targetScore", 10))

 game_manager.start_game(room, target_score=target_score)

 await sio.emit(
 "game_started",
 {
 "roomCode": room.code,
 "targetScore": target_score,
 },
 room=room.code,
)
 await emit_private_game_states(room)

 return success()
except Exception as error:

```

```
return failure(str(error))
```

```
@sio.event
```

```
async def get_game_state(sid: str, data: Any = None):
 try:
 room = room_manager.get_room_for_sid(sid)
 if room is None:
 raise ValueError("You are not inside a room")

 user = await sio.get_session(sid)
 state = game_manager.get_private_state(room, user["uid"])
 return success({"game": state})
 except Exception as error:
 return failure(str(error))
```

```
@sio.event
```

```
async def symbol_click(sid: str, data: Any = None):
 try:
 if not isinstance(data, dict):
 raise ValueError("Expected symbol and cardId")

 symbol = str(data.get("symbol", "")).strip()
 card_id = str(data.get("cardId", "")).strip()

 if not symbol:
 raise ValueError("symbol is required")
 if not card_id:
 raise ValueError("cardId is required")

 room = room_manager.get_room_for_sid(sid)
 if room is None:
 raise ValueError("You are not inside a room")

 user = await sio.get_session(sid)
 correct, result = game_manager.submit_symbol(
 room=room,
 uid=user["uid"],
 symbol=symbol,
 card_id=card_id,
)
```

```

if not correct:
 await sio.emit(
 "wrong_symbol",
 {
 "uid": user["uid"],
 "submittedSymbol": result["submittedSymbol"],
 },
 to=sid,
)
 return success({"correct": False})

await sio.emit(
 "round_result",
 result,
 room=room.code,
)

if result["gameFinished"]:
 game = game_manager.get_game(room.code)

 game_over_data = game.to_public_dict(
 room.players
)

 try:
 match_id = stats_manager.save_match(
 room_code=room.code,
 players=room.players,
 scores=game.scores,
 winner_uid=game.winner_uid,
 target_score=game.target_score,
 round_number=game.round_number
)

 game_over_data["matchId"] = match_id
 game_over_data["statsSaved"] = True

 except Exception as error:
 print("Failed to save match:", error)

 game_over_data["statsSaved"] = False
 game_over_data["statsError"] = str(error)

```

```
 await sio.emit(
 "game_over",
 game_over_data,
 room=room.code,
)
 else:
 await emit_private_game_states(room)

 return success({"correct": True, **result})
except Exception as error:
 return failure(str(error))

if __name__ == "__main__":
 port = int(os.getenv("PORT", "5000"))
 print(f"Loaded and validated {len(card_manager.cards)} Dobble cards.")
 uvicorn.run("app:app", host="0.0.0.0", port=port, reload=True)
```

## auth\_manager.py מחלקה

```
import os
from pathlib import Path
from typing import Any, Dict

import firebase_admin
from firebase_admin import auth, credentials, firestore

class AuthManager:

 def __init__(self, project_root: Path):
 self.dev_skip_auth = (
 os.getenv("DEV_SKIP_AUTH", "false").lower() == "true"
)

 if self.dev_skip_auth:
 print(
 "WARNING: DEV_SKIP_AUTH is enabled. "
 "Do not use this in production."
)
 self.database = None
 return

 credentials_path = os.getenv(
 "FIREBASE_CREDENTIALS",
 "serviceAccountKey.json",
)

 credentials_file = Path(credentials_path)

 if not credentials_file.is_absolute():
 credentials_file = project_root / credentials_file

 if not credentials_file.exists():
 raise FileNotFoundError(
 "Firebase credentials file was not found: "
 f"{credentials_file}"
)

 if not firebase_admin._apps:
 firebase_admin.initialize_app(
```

```

 credentials.Certificate(
 str(credentials_file)
)
)

כדי שתמיד יהיה if-חשוב: מחוץ ל
self.database = firestore.client()

def verify_connection(
 self,
 auth_payload: Any
) -> Dict[str, str]:

 if not isinstance(auth_payload, dict):
 raise ValueError(
 "Missing Socket.IO authentication payload"
)

 if self.dev_skip_auth:
 uid = str(
 auth_payload.get("uid", "")
).strip()

 if not uid:
 raise ValueError(
 "DEV mode requires auth.uid"
)

 return {
 "uid": uid,
 "name": str(
 auth_payload.get("name", uid)
),
 "email": str(
 auth_payload.get("email", "")
),
 }

token = auth_payload.get("token")

if not isinstance(token, str) or not token.strip():
 raise ValueError(
 "Missing Firebase ID token"
)

```

```

)

 decoded = auth.verify_id_token(token)

 uid = decoded["uid"]
 email = decoded.get("email", "")

 display_name = None

 try:
 document = (
 self.database
 .collection("users")
 .document(uid)
 .get()
)

 print(
 f"Firestore user document exists "
 f"for {uid}: {document.exists}"
)

 if document.exists:
 user_data = document.to_dict() or {}

 print(
 f"Firestore user data for {uid}: "
 f"{user_data}"
)

 display_name = (
 user_data.get("displayName")
 or user_data.get("name")
 or user_data.get("username")
)

 except Exception as error:
 print(
 f"Failed to load Firestore user {uid}: "
 f"{type(error).__name__}: {error}"
)

 if display_name:

```

```
display_name = str(display_name).strip()

if not display_name:
 display_name = (
 decoded.get("name")
 or email
 or uid
)

print(
 f"Authenticated user {uid} "
 f"with display name: {display_name}"
)

return {
 "uid": uid,
 "name": display_name,
 "email": email,
}
```



**מחלקה card\_manager.py**

```

import json
from pathlib import Path
from typing import Dict, List, Set

class CardManager:
 VALID_REGIONS = {
 "top-left", "top-center", "top-right",
 "center-left", "center-center", "center-right",
 "bottom-left", "bottom-center", "bottom-right",
 }

 def __init__(self, cards_path: Path):
 with cards_path.open("r", encoding="utf-8") as file:
 self.cards: Dict[str, Dict[str, str]] = json.load(file)

 self._validate()

 def _unique_symbols(self, card: Dict[str, str]) -> Set[str]:
 return {symbol for symbol in card.values() if symbol}

 def _validate(self) -> None:
 if len(self.cards) != 55:
 raise ValueError(f"Expected 55 cards, found {len(self.cards)}")

 for card_id, regions in self.cards.items():
 unknown_regions = set(regions) - self.VALID_REGIONS
 if unknown_regions:
 raise ValueError(
 f"Card {card_id} contains invalid regions: {sorted(unknown_regions)}"
)

 symbols = self._unique_symbols(regions)
 if len(symbols) != 8:
 raise ValueError(
 f"Card {card_id} must contain 8 unique symbols, found {len(symbols)}"
)

 card_ids = list(self.cards.keys())
 for index, first_id in enumerate(card_ids):
 first_symbols = self._unique_symbols(self.cards[first_id])

```

```

for second_id in card_ids[index + 1:]:
 second_symbols = self._unique_symbols(self.cards[second_id])
 common = first_symbols & second_symbols

 if len(common) != 1:
 raise ValueError(
 f"Cards {first_id} and {second_id} share "
 f"{len(common)} symbols: {sorted(common)}"
)

def get_card(self, card_id: str) -> Dict[str, str]:
 card = self.cards.get(str(card_id))
 if card is None:
 raise KeyError(f"Card {card_id} does not exist")
 return card

def get_symbols(self, card_id: str) -> List[str]:
 return sorted(self._unique_symbols(self.get_card(card_id)))

def get_common_symbol(self, first_card_id: str, second_card_id: str) -> str:
 common = (
 set(self.get_symbols(first_card_id))
 & set(self.get_symbols(second_card_id))
)
 if len(common) != 1:
 raise ValueError("The selected cards do not share exactly one symbol")
 return next(iter(common))

```

**game\_manager.py מחלקה**

```

import random
from typing import Dict, Optional, Tuple

from managers.card_manager import CardManager
from models.game import GameState
from models.room import Room

class GameManager:
 def __init__(self, card_manager: CardManager):
 self.card_manager = card_manager
 self.games: Dict[str, GameState] = {}

 def start_game(self, room: Room, target_score: int = 10) -> GameState:
 if room.code in self.games:
 raise ValueError("A game is already active in this room")
 if len(room.players) != 2:
 raise ValueError("Exactly two players are required to start")
 if target_score < 1 or target_score > 50:
 raise ValueError("targetScore must be between 1 and 50")

 card_order = list(self.card_manager.cards.keys())
 random.shuffle(card_order)

 player_uids = list(room.players.keys())
 state = GameState(
 room_code=room.code,
 card_order=card_order,
 center_index=0,
 player_card_indexes={
 player_uids[0]: 1,
 player_uids[1]: 2,
 },
 scores={uid: 0 for uid in player_uids},
 target_score=target_score,
)

 self.games[room.code] = state
 room.status = "playing"
 return state

```

```

def get_game(self, room_code: str) -> GameState:
 game = self.games.get(room_code)
 if game is None:
 raise ValueError("No active game in this room")
 return game

def get_private_state(self, room: Room, uid: str) -> dict:
 game = self.get_game(room.code)
 if uid not in room.players:
 raise ValueError("Player is not in this room")

 result = game.to_public_dict(room.players)
 result["playerCardId"] = game.player_card_id(uid)
 return result

def submit_symbol(
 self,
 room: Room,
 uid: str,
 symbol: str,
 card_id: str,
) -> Tuple[bool, dict]:
 game = self.get_game(room.code)

 if game.winner_uid:
 raise ValueError("The game has already ended")
 if not game.round_open:
 raise ValueError("This round is already closed")
 if uid not in room.players:
 raise ValueError("Player is not in this room")

 expected_player_card = game.player_card_id(uid)
 if str(card_id) != str(expected_player_card):
 raise ValueError("The submitted card does not match the current player card")

 common_symbol = self.card_manager.get_common_symbol(
 game.center_card_id,
 expected_player_card,
)

 normalized_submitted = symbol.strip().casefold()
 normalized_expected = common_symbol.casefold()

```

```

if normalized_submitted != normalized_expected:
 return False, {
 "correct": False,
 "submittedSymbol": symbol,
 "expectedSymbol": common_symbol,
 }

game.round_open = False
game.scores[uid] += 1

if game.scores[uid] >= game.target_score:
 game.winner_uid = uid
 room.status = "finished"

 return True, {
 "correct": True,
 "winnerUid": uid,
 "correctSymbol": common_symbol,
 "gameFinished": True,
 }

self._advance_round(game, uid)

return True, {
 "correct": True,
 "roundWinnerUid": uid,
 "correctSymbol": common_symbol,
 "gameFinished": False,
}

def _advance_round(self, game: GameState, round_winner_uid: str) -> None:
 # The winning player's card becomes the next center card.
 new_center_index = game.player_card_indexes[round_winner_uid]
 game.center_index = new_center_index

 used_indexes = {game.center_index}
 for uid in list(game.player_card_indexes.keys()):
 next_index = self._find_next_unused_index(game, used_indexes)
 game.player_card_indexes[uid] = next_index
 used_indexes.add(next_index)

 game.round_number += 1
 game.round_open = True

```

```
def _find_next_unused_index(self, game: GameState, used_indexes: set[int]) -> int:
 total = len(game.card_order)
 start = max(used_indexes) + 1 if used_indexes else 0

 for offset in range(total):
 candidate = (start + offset) % total
 if candidate not in used_indexes:
 return candidate

 raise RuntimeError("No unused card index available")

def cancel_game(self, room_code: str) -> Optional[GameState]:
 return self.games.pop(room_code, None)
```

**room\_manager.py מחלקה**

```

import secrets
from typing import Dict, Optional

from models.room import Player, Room

class RoomManager:
 def __init__(self):
 self.rooms: Dict[str, Room] = {}
 self.sid_to_room: Dict[str, str] = {}

 def _generate_code(self) -> str:
 for _ in range(100):
 code = f"{secrets.randbelow(900000) + 100000}"
 if code not in self.rooms:
 return code
 raise RuntimeError("Could not generate a unique room code")

 def create_room(self, uid: str, sid: str, display_name: str) -> Room:
 if sid in self.sid_to_room:
 raise ValueError("You are already inside a room")

 code = self._generate_code()
 room = Room(code=code, owner_uid=uid)
 room.players[uid] = Player(
 uid=uid,
 sid=sid,
 display_name=display_name,
)

 self.rooms[code] = room
 self.sid_to_room[sid] = code
 return room

 def join_room(
 self,
 code: str,
 uid: str,
 sid: str,
 display_name: str,
) -> Room:

```

```

if sid in self.sid_to_room:
 raise ValueError("You are already inside a room")

room = self.rooms.get(code)
if room is None:
 raise ValueError("Room not found")
if room.status != "waiting":
 raise ValueError("The game has already started")
if len(room.players) >= 2:
 raise ValueError("Room is full")
if uid in room.players:
 raise ValueError("This user is already in the room")

room.players[uid] = Player(
 uid=uid,
 sid=sid,
 display_name=display_name,
)
self.sid_to_room[sid] = code
return room

def leave_by_sid(self, sid: str) -> Optional[Room]:
 code = self.sid_to_room.pop(sid, None)
 if code is None:
 return None

 room = self.rooms.get(code)
 if room is None:
 return None

 leaving_uid = next(
 (uid for uid, player in room.players.items() if player.sid == sid),
 None,
)
 if leaving_uid:
 room.players.pop(leaving_uid, None)

 if not room.players:
 self.rooms.pop(code, None)
 return None

 if room.owner_uid == leaving_uid:
 room.owner_uid = next(iter(room.players))

```



```
return room
```

```
def get_room_for_sid(self, sid: str) -> Optional[Room]:
 code = self.sid_to_room.get(sid)
 return self.rooms.get(code) if code else None
```

**stats\_manager.py מחלקה**

```

from datetime import datetime, timezone
from typing import Dict
from uuid import uuid4

from firebase_admin import firestore

class StatsManager:

 def __init__(self):
 self.database = firestore.client()

 def save_match(
 self,
 room_code: str,
 players: Dict[str, object],
 scores: Dict[str, int],
 winner_uid: str,
 target_score: int,
 round_number: int
) -> str:

 match_id = str(uuid4())

 player_uids = list(players.keys())

 player_names = {
 uid: players[uid].display_name
 for uid in player_uids
 }

 match_data = {
 "matchId": match_id,
 "roomCode": room_code,
 "players": player_uids,
 "playerNames": player_names,
 "scores": scores,
 "winnerUid": winner_uid,
 "targetScore": target_score,
 "roundsPlayed": round_number,
 "createdAt": datetime.now(timezone.utc)

```

```
}

batch = self.database.batch()

match_reference = (
 self.database
 .collection("matches")
 .document(match_id)
)

batch.set(match_reference, match_data)

for uid in player_uids:
 user_reference = (
 self.database
 .collection("users")
 .document(uid)
)

 score = scores.get(uid, 0)

 updates = {
 "gamesPlayed": firestore.Increment(1),
 "totalScore": firestore.Increment(score)
 }

 if uid == winner_uid:
 updates["wins"] = firestore.Increment(1)
 else:
 updates["losses"] = firestore.Increment(1)

 batch.set(
 user_reference,
 updates,
 merge=True
)

batch.commit()

return match_id
```

**game.py מחלקה**

```
from dataclasses import dataclass, field
from typing import Dict, List, Optional
```

```
@dataclass
class GameState:
 room_code: str
 card_order: List[str]
 center_index: int = 0
 player_card_indexes: Dict[str, int] = field(default_factory=dict)
 scores: Dict[str, int] = field(default_factory=dict)
 round_number: int = 1
 round_open: bool = True
 winner_uid: Optional[str] = None
 target_score: int = 10

 @property
 def center_card_id(self) -> str:
 return self.card_order[self.center_index]

 def player_card_id(self, uid: str) -> str:
 return self.card_order[self.player_card_indexes[uid]]

 def to_public_dict(self, players: Dict[str, object]) -> dict:
 return {
 "roomCode": self.room_code,
 "roundNumber": self.round_number,
 "centerCardId": self.center_card_id,
 "scores": self.scores,
 "targetScore": self.target_score,
 "status": "finished" if self.winner_uid else "playing",
 "winnerUid": self.winner_uid,
 "players": [
 {
 "uid": uid,
 "displayName": players[uid].display_name,
 "score": self.scores.get(uid, 0),
 }
 for uid in players
],
 }
```

**מחלקה room.py**

```
from dataclasses import dataclass, field
from typing import Dict
```

```
@dataclass
class Player:
 uid: str
 sid: str
 display_name: str
```

```
@dataclass
class Room:
 code: str
 owner_uid: str
 players: Dict[str, Player] = field(default_factory=dict)
 selected_box: str = "original"
 status: str = "waiting"

 def to_dict(self) -> dict:
 return {
 "code": self.code,
 "ownerUid": self.owner_uid,
 "selectedBox": self.selected_box,
 "status": self.status,
 "players": [
 {
 "uid": player.uid,
 "displayName": player.display_name,
 }
 for player in self.players.values()
],
 }
```